DPM Expression Language (DPM-XL) Documentation

None

EBA and EIOPA

None

Table of contents

1. DPM Expression Language (DPM-XL) Documentation 3
   - 1.1 What is DPM-XL? 3
   - 1.2 Key Features 3
   - 1.3 Quick Start 3
   - 1.4 Documentation Structure 3
   - 1.5 Who Should Use This Documentation? 4
   - 1.6 Getting Help 4
2. Information Model 5
   - 2.1 Introduction 5
   - 2.2 2 Information Model 7
   - 2.3 3 DPM-XL Data Selection 17
   - 2.4 4 Null in DPM Expression Language 19
   - 2.5 5 DPM-ML Metamodel 20
3. Operators 25
   - 3.1 1 Introduction 25
   - 3.2 2 Common General Behaviours for the Operators 26
   - 3.3 3 Selection Operators 29
   - 3.4 4 Numeric Operators 36
   - 3.5 5 Comparison Operators 45
   - 3.6 6 Logical Operators 52
   - 3.7 7 Aggregate Operators 55
   - 3.8 8 Conditional Operators 68
   - 3.9 9 String Operators 77
   - 3.10 10 Time Operators 80
   - 3.11 11 Clause Operators 90
   - 3.12 12 Assignment Operators 97
   - 3.13 13 Set Operators 99
4. Grammar 107
   - 4.1 Introduction 107
5. Script Format 108
   - 5.1 1 Introduction 108
   - 5.2 2 Structure and Concepts 111
   - 5.3 3 Document Sections 115
   - 5.4 4 Abstract Syntax Tree Reference 127
   - 5.5 5 Conformance 154

1. DPM Expression Language (DPM-XL) Documentation

Welcome to the comprehensive documentation for the **Data Point Model Expression Language (DPM-XL)**, a formal language for expressing data validation and calculation requirements in regulatory reporting.

1.1 What is DPM-XL?

The DPM Expression Language is a formal, business-user-friendly language that enables the specification of:

- **Data validation rules** for regulatory reporting
- **Calculation formulas** for derived data points
- **Data transformation operations** across reporting tables
- **Cross-module validations** and consistency checks

DPM-XL serves as the foundation for automated validation systems used by regulatory authorities such as EBA and EIOPA.

1.2 Key Features

1.2.1 🎯 Business-Oriented

Designed for business users who understand regulatory requirements but may not be programmers.

1.2.2 ⚡ Formally Specified

Fully formalized syntax and semantics enable automated compilation and execution.

1.2.3 🔄 Dual Representation

- **DPM-XL**: Human-readable expression language
- **DPM-ML**: Machine-readable metamodel representation

1.2.4 📊 Rich Data Model

Supports complex data structures including scalars, sets, and multi-dimensional recordsets.

1.2.5 🛡️ Robust

Comprehensive null handling, type system, and error management.

1.3 Quick Start

Here's a simple DPM-XL expression:

```
{tF_01.01, r0010, c0010} = {tF_01.01, r0020, c0010} + {tF_01.01, r0030, c0010}
```

This expression validates that the value in table F_01.01, row 0010, column 0010 equals the sum of rows 0020 and 0030 in the same column.

1.4 Documentation Structure

This documentation is organized to take you from basic concepts to advanced implementation:

1.4.1 🏗️ Information Model

Understand the core data structures: operations, artifacts (scalars, sets, recordsets), and data types.

1.4.2 ⚙️ Operators

Comprehensive reference for all operators, organized by category: - Selection, Numeric, Comparison, Logical - Aggregate, Conditional, String, Time, Clause

1.4.3 📖 Grammar

Quick reference materials, grammar specifications, and operator summaries.

1.5 Who Should Use This Documentation?

1.5.1 **Regulatory Specialists**

Writing validation rules and calculations for regulatory frameworks.

1.5.2 **Software Developers**

Building engines and tools that execute DPM-XL expressions.

1.5.3 **Implementation Teams**

Integrating DPM-XL into reporting systems and validation pipelines.

1.5.4 **Standard Bodies**

Understanding the formal specification for governance and evolution.

1.6 Getting Help

This documentation aims to be comprehensive and self-contained. Each section includes:

- Clear explanations of concepts
- Formal specifications where needed
- Practical examples
- Edge case handling

For the most current information and updates, please refer to the official DPM standards documentation.

Navigation Tip

Use the navigation menu on the left to explore different sections. Each section builds upon previous concepts, so we recommend following the order for first-time readers.

2. Information Model

2.1 Introduction

This is a technical document specifying the Information Model, the *Data Types* and other technical characteristics of the DPM *Operations*.

This documentation is mainly addressed to technical audiences with interest in developing engines or in having a better technical understanding of the DPM *Operations*. The document is not addressed to normal business users that write calculations, although it can be useful for those users with basic knowledge of information models and programming languages.

In any case, this document is complemented by the document describing the *Operators* used for the *Operations*, which should be the main reference for users writing calculations.

2.1.1 Why

EBA and EIOPA have been using for some years a semi-formal expression language for business users to write and share their data validation requirements.

The fact that is semi-formal allows automating some aspects for translating the expression into another language (e.g., XBRL Formula Link base). But because it is not completely formal, it is, by definition, not possible to create a compiler that fully automates the translation to another language or an interpreter for it.

This document, and its complementary document on the *Operators*, aims to provide this required formalization so that DPM *Operations* become fully formalized, therefore enabling the automation regarding the calculations written by EBA and EIOPA.

2.1.2 DPM-XL and DPM-ML

The DPM *Operations* have two representations: As expressions, with the DPM eXpression Language (DPM-XL) and as a structured representation in the database following a metamodel (DPM Metamodel Language or DPM-ML).

The reason for having **DPM-XL** is that it is the language business users write and can understand. Because it is the input of all the process, it is important to keep this language formal, so that it can be translated and executed. But, at the same time, it must serve for communication, so it has to be business users friendly, implying, among others, that the *Operands* of the language are referred to by using rendering artifacts (tables, row, columns and sheets).

A key requirement for the DPM-XL has been that the language that is currently used should change as little as possible. So, in practice, the formalization of the language has been a reverse-engineering process, where the starting point where the validations published by the EBA and EIOPA.

The main reasons for having **DPM-ML** are that it is providing a structured version of the calculations, with no need to parse expressions, and that it is referring to *Variables*, which are stable over time, because they represent business concepts, instead of rendering objects, which change often over time and do not hold any business meaning, but just a representation.

The DPM-ML is automatically derived from the DPM-XL and, eventually, can be also derived from languages other than the DPM-XL. DPM-ML relations to languages other than DPM-XL is out of the scope for this documentation. Instead, the documentation provides details on how *Operations* are represented both in DPM-XL and DPM-ML. The DPM-XL and DPM-ML share the same *Operators* and the same information model.

2.1.3 How

The formalization of the language has three pillars:

1. **Information model**: Specifies the artifacts that the language is using.
2. **Grammar**: Technical definition of the syntax of the language. Allows developers to build parsers for the language. It is provided in a separate file following the EBNF (Extended Backus-Naur form) notation (Only applicable to DPM-XL).
3. **Semantic specification**: Semantics for all *Operators* of the language, specifying formally their constraints and behaviour.

This document deals with the information model, while the grammar and semantic specification are provided as separate documents.

The Validation and Transformation Language specification has been used as an inspiration for this specification.

2.2 2 Information Model

2.2.1 2.1 Operations

classDiagram class Script class Operation class Result class Operand class Operator class LanguageArtifact class Output class Input Script *-- Operation Operation --> "1..1" Result : Produces Operation --> "0..n" Operand : Acts on Operation --> "0..n" Operator: Uses Result --|> LanguageArtifact Operand --|> LanguageArtifact Operator --> "1..1" Output : Produces Operator --> "0..n" Input : Uses Output --> Input Operand --> Input style Script fill:#0098f6 style Operation fill:#0098f6 style Result fill:#0098f6 style Operand fill:#0098f6 style Operator fill:#0098f6 style LanguageArtifact fill:#04f286 style Input fill:#0098f6 style Output fill:#0098f6

The DPM *Operations* serve to write *Scripts*, which are computer programs that constitute a run-time and are composed of *Operations*.

Operations are expressions that use input *Operands* and/or *Operators* to produce a result.

Expressions are finite combinations of symbols that are well-formed according to the syntactical rules of the language. Expressions compose some *Operands* in a certain order by means of the *Operators* of the language, to obtain the desired result. The symbols of the expression designate *Operators*, *Operands*, and the order of application of the *Operators*.

Operators specify a type of operation to be performed on some input *Operands* (exceptionally, there may be *Operators* that do not take *Operands* as input, e.g., an *Operator* to get the current time) to generate an output. The output produced by one *Operator* may be used as input for another *Operator* (i.e., *Operators* can be nested).

Operands are specific artifacts from the DPM Expression Language referenced in an expression as input.

The result produced by an *Operation* is also a specific artifact from the DPM Expression Language.

2.2.2 2.2 Language Artifacts

classDiagram class LanguageArtifact class Recordset class Scalar class ScalarSet class DataType Recordset --|> LanguageArtifact Scalar --|> LanguageArtifact ScalarSet --|> LanguageArtifact Recordset --> DataType : Defined on Scalar --> DataType : Defined on ScalarSet --> "1..n" Scalar style LanguageArtifact fill:#04f286 style Recordset fill:#04f286 style Scalar fill:#04f286 style ScalarSet fill:#04f286 style DataType fill:#ecdc0a

DPM Expression Language calculations can operate on and generate as results three different types of *Artifacts*.

2.2.1 Scalars

Scalars are individual values of a certain *Data Type*.

2.2.2 Scalar Sets

Scalar Sets are sets of *Scalar* values defined on the same *Data Type*. They are unordered and contain no duplicates. *Scalar Sets* are used with the in *Operator*, can be derived from a *Component* of a *Recordset* and combined through the Set *Operators* (see §13), and their cardinality is obtained with the count *Operator*.

2.2.3 Recordsets

classDiagram class Recordset class Records class Structure class Component class KeyComponent class FactComponent class AttributeComponent class DataType Recordset *-- Records Recordset --> Structure : Structured by Structure *-- Component Component --> DataType : Defined on KeyComponent --|> Component FactComponent --|> Component AttributeComponent --|> Component style Recordset fill:#04f286 style Records fill:#04f286 style Structure fill:#04f286 style Component fill:#04f286 style KeyComponent fill:#04f286 style FactComponent fill:#04f286 style AttributeComponent fill:#04f286 style DataType fill:#ecdc0a

Recordsets are collections of *Records* that share a same *Structure*. Technically, *Recordsets* are two-dimensional labelled data structures (tabular), which can be assimilated to Relational Tables or Data Frames. The columns (fields) of the *Recordset* are provided by the *Components* of its *Structure*. The rows of the *Recordset* are its composing *Records*.

The *Structure* of the *Recordset* is a collection of *Components*, which can have one of three roles: *Key*, *Fact* or *Attribute*. Each *Component* has a name, which must be unique within the *Recordset*. *Key*, *Fact* and *Attribute Components* are each defined on a single *Data Type*. For *Fact Components*, the *Data Type* is determined by the *Data Type* of the selected *Variables*, as described in §2.2.3.2.

Each *Record* of the *Recordset* is individually identified by the combination of the values for its *Key Components*.

A *Recordset* having no *Key Components* behaves like a *Scalar*.

A *Recordset* may have no records.

2.2.3.1 KEY COMPONENTS

classDiagram class KeyComponent class StandardKey class DpmKey class DpmXlStandardKey class DpmMlStandardKey class RowKey class ColumnKey class SheetKey class XIndexKey class YIndexKey class ZIndexKey StandardKey --|> KeyComponent DpmKey --|> KeyComponent DpmXlStandardKey --|> StandardKey DpmMlStandardKey --|> StandardKey RowKey --|> DpmXlStandardKey ColumnKey --|> DpmXlStandardKey SheetKey --|> DpmXlStandardKey XIndexKey --|> DpmMlStandardKey YIndexKey --|> DpmMlStandardKey ZIndexKey --|> DpmMlStandardKey style KeyComponent fill:#04f286 style StandardKey fill:#04f286 style DpmKey fill:#04f286 style DpmXlStandardKey fill:#04f286 style DpmMlStandardKey fill:#04f286 style RowKey fill:#04f286 style ColumnKey fill:#04f286 style SheetKey fill:#04f286 style XIndexKey fill:#04f286 style YIndexKey fill:#04f286 style ZIndexKey fill:#04f286

Key Components may be *Standard* or *DPM*. *Standard Key Components* are different for DPM-ML and DPM-XL, although their behaviour is the same.

Standard Key Components are common to all the *Recordsets*, independently on how the *Variables* are defined in the DPM. For each *Recordset*, there may be 0 or 1 occurrence of each subtype of *Standard Key Component*.

2.2.3.1.1 Standard DPM-XL Key Components

- **Row Key**: Identifies the Row Ordinate from a Report Table where the selected *Variable* is located. Arises in *Variable Set* Selections, when more than one Row for one Report Table is selected. The name for the *Component* is "r". It is defined on the String *Data Type*.
- **Column Key**: Identifies the Column Ordinate from a Report Table where the selected *Variable* is located. Arises in *Variable Set* Selections, when more than one Column for one Report Table is selected. The name for the *Component* is "c". It is defined on the String *Data Type*.
- **Sheet Key**: Identifies the Sheet Ordinate from a Report Table where the selected *Variable* is located. Arises in *Variable Set* Selections, when more than one Sheet for one Report Table is selected. The name for the *Component* is "s". It is defined on the String *Data Type*.

2.2.3.1.2 Standard DPM-ML Key Components

Regarding the DPM-ML, the *Standard Components* are three indexes, which do not represent any DPM object. They just serve to make sure that the right *Variables* are matched together when used with an *Operator*. The names for these *Components* are "x", "y" and "z". Note that these indexes do not necessarily correspond to rows, columns, and sheets. During the conversion from DPM-XL, the values of the indexes for each *Variable* shall be provided according to algorithms.

2.2.3.1.3 DPM Key Components

DPM Key Components are specific to how data is defined in the DPM. Arise when Open *Variables* are selected, and a *Recordset* will have one *DPM Key Component* per each *Key Variable* associated to the selected *Variables*.

The name for the *DPM Key Components* is the Code of the Property associated to the DPM *Key Variable*.

2.2.3.1.4 Contextual Key Components

There are two contextual dimensions that are always latent in every *Recordset*:

- **refPeriod**: Identifies the reference period of the data. It is defined on the *Date Data Type*.
- **entityID**: Identifies the reporting entity. It is defined on the *String Data Type*.

At compile-time, refPeriod and entityID are assumed to always exist, so they can always be referenced in expressions (e.g., as the `var` parameter in `time_shift`). At runtime, whether they effectively act as *Key Components* depends on the input data: if the data contains multiple reference periods or entities, they become distinguishing dimensions in the *Recordset*.

Being *Key Components*, refPeriod and entityID always have a value: a *Record* belonging to no reference period or to no entity cannot exist. The same contextual pattern applies to a set of *Attribute Components*, which are nullable instead; these are described in section 2.2.3.3.1.

A *Scalar* in DPM-XL is a *Recordset* with no *Key Components* other than the contextual ones. Within each instance (i.e., for each combination of refPeriod and entityID), it behaves as a single value. Therefore, "scalar" means "one value per instance", which is the only meaningful scope for a calculation.

2.2.3.2 FACT COMPONENT

A *Recordset* has one mandatory, and only one, *Fact Component*. The name for the *Component* is "f". Its *Data Type* depends on the *Data Type* of the selected *Variables*.

When a selection includes *Variables* of the same *Data Type*, the *Fact Component* has that *Data Type*. When a selection includes *Variables* of different *Data Types*, the *Data Type* of the *Fact Component* is the maximum common *Data Type*, determined by applying implicit casting (see section 2.3.2). The implicit casting hierarchy is as follows:

- Integer → Number → String
- Date → String
- Boolean → String
- Item → String
- Subcategory → String

The maximum common *Data Type* is the most specific type to which all the selected *Variables* can be implicitly cast. For example, if a selection includes Integer and Number *Variables*, the *Fact Component* has *Data Type* Number. If a selection includes Number and Boolean *Variables*, the *Fact Component* has *Data Type* String.

Selections resulting in implicit casting to a common *Data Type* should raise a warning, as the casting may cause a loss of operational capability (e.g., a *Recordset* cast to String cannot be used with arithmetic *Operators*).

If a DPM *Key Variable* is selected, the resulting *Recordset* will have the values for that *Variable* in the *Fact Component*, on top of having them in their corresponding *Key Component*. See example in section 2.2.4.4.

2.2.3.3 ATTRIBUTE COMPONENTS

Attribute Components provide additional information to the *Fact* value.

Attribute Components may be *DPM* or *Contextual*. Unlike *Key Components*, *Attribute Components* are nullable: they describe the *Fact*, they do not identify the *Record*.

DPM *Attribute Components* are specific to how data is defined in the DPM. Arise when a *Variable* with associated *Attribute Variable* is selected. A *Recordset* will have one DPM *Attribute Component* per each Property or Metric associated to any *Attribute Variable* associated to the selected *Variables*.

The name for the DPM *Attribute Components* is the Code of the Property associated to the DPM *Attribute Variable*.

2.2.3.3.1 Contextual Attribute Components

Besides those arising from the DPM, there are several contextual *Attribute Components* that are always latent in every *Recordset*. They follow the same pattern as the contextual *Key Components* of section 2.2.3.1.4: they are carried by the instance rather than by the individual *Record*, so no selection declares them, and within a single instance every *Record* shares the same value.

The contextual *Attribute Components* are:

Component	***Data Type***	**Meaning**
`baseCurrency`	Item	The reporting currency of the instance.
`decimalsInteger`	Integer	The declared precision of Integer *Facts* (DPM data type `i`).
`decimalsMonetary`	Integer	The declared precision of monetary *Facts* (DPM data type `m`).
`decimalsPercentage`	Integer	The declared precision of percentage *Facts* (DPM data type `p`).
`decimalsDecimal`	Integer	The declared precision of plain Number *Facts* (DPM data type `r`).

At compile-time, these *Attribute Components* are assumed to always exist, so they can always be referenced in any expression that takes a *Component* as input (e.g., in the condition of `where`, or as the component parameter of `get`). At runtime, a contextual *Attribute Component* is added to the data structure of the *Recordset* only where an expression references it; where no expression references it, it need not be loaded.

Being *Attribute Components*, they are nullable, and this is where their treatment differs from that of the contextual *Key Components*. A contextual *Key Component* always has a value, because a *Record* belonging to no reference period or to no entity cannot exist. A contextual *Attribute Component* only describes the *Fact*, and may be absent: where the instance declares no value for one of them, the *Component* takes the value null, which propagates according to the null semantics of part 4.

Within a single instance, a contextual *Attribute Component* carries one value repeated across every *Record*. Where the *Recordset* spans several instances, i.e., several combinations of refPeriod and entityID, the value may vary from *Record* to *Record*, exactly as refPeriod and entityID do.

The `decimals` *Components* give the declared precision of the reported *Fact*, one per numeric DPM data type (see section 2.3.1), following the convention that a negative value denotes precision to the left of the decimal point: `-3` means the value is accurate to thousands. They are the source of the precision used to derive the radius of a *Number* treated as an interval (see section 2.3), where interval arithmetics is requested. The precision is a property of the instance and is not overridden per *Fact*.

Some examples:

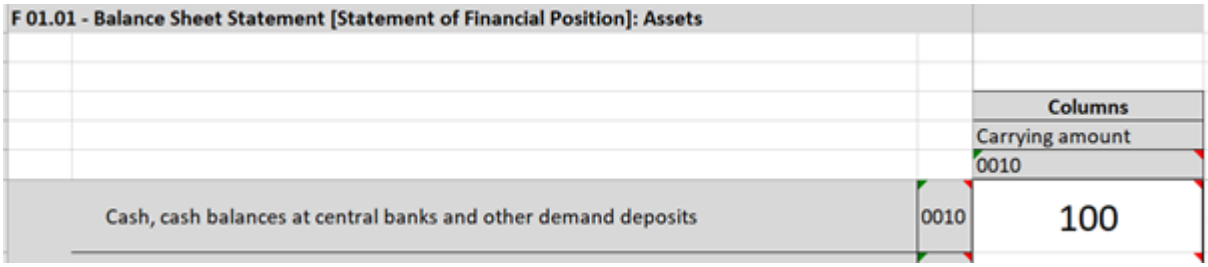
- `{tF_01.01}[where baseCurrency = [eba_CU:EUR]]` restricts the *Recordset* to the *Records* belonging to instances reported in euro. Evaluated over a single instance, this keeps either every *Record* or none.
- `{tM_01.01}[where decimalsMonetary >= -3]` keeps the monetary *Facts* reported at least to thousands precision.
- `{tF_01.01}[get baseCurrency]` promotes the reporting currency into the *Fact Component*, so that it can be compared or aggregated.

2.2.4 Examples

Recordsets arise from Open *Variable* selections or from selection of multiple *Variables*.

2.2.4.1 CLOSED VARIABLE SELECTION

For instance, taking as an example EBA's Table F 01.01:




```
{tF_01.01, r0010, c0010}
```

In the context of a single instance (i.e., disregarding the "refPeriod" and the "entityID"), selects a *Recordset* without *Key Components*, which works as a *Scalar* with value 100.

In the context of data with multiple dates and/or entities, and supposing that the data refer to the entity with code "id123" and to the date 31/12/2020, and that it has been reported with a 1 precision (attribute):

entityID	refPeriod	f	p
id123	2020-12-31	100	1

2.2.4.2 OPEN VARIABLE SELECTION

F 40.01 - Scope of the group: “entity-by-entity”

			Columns		
			Code	Type of code	Entity name
			0011	0015	0031
Rows	Investee	999	123456	LEI	Name1
			123456	ISIN	Name2
			1111	LEI	Name3
			LIN <Key value>	TYC <Key value>	

```
{tF_40.01, c0031}
```

Selects an Open *Variable*. In this illustrative example, yields a *Recordset* with three *Records*. The *Recordset*, in the context of a single instance, has two DPM *Key Components*, because the selected *Variable* (the one rendered in Column 0031 of Report Table F 40.01) is associated in the DPM to a *Key* with two *Key Variables*. These *Key Variables* are associated to Properties with codes "LIN" (rendered in Column 0011) and "TYC" (rendered in Column 0015).

The following table represents the *Recordset*:

LIN	TYC	f
123456	LEI	Name 1
123456	ISIN	Name 2
1111	LEI	Name 3

2.2.4.3 VARIABLE SET SELECTION

F 20.05.a - Geographical breakdown of off-balance sheet exposures by residence of the counterparty (a)

ES

			Columns
			Nominal amount
			0010
Rows	Loan commitments given	0010	100
	Financial guarantees given	0020	200
	Other commitments given	0030	300

PT

			Columns
			Nominal amount
			0010
Rows	Loan commitments given	0010	400
	Financial guarantees given	0020	500
	Other commitments given	0030	600

```
{tF_20.05, r0020-0030, c0010}
```

This case selects a set of *Variables* because the selection includes more than one Row. This implies that the *Row Key Component* applies to this *Recordset*. Besides, the selected *Variables* are associated to a *Key* with a single *Key Variable*, which is associated to a property with code "RCP".

RCP	r	f
ES	0020	200
ES	0030	300
PT	0020	500
PT	0030	600

In this case, the *Recordset* for the DPM-ML would be slightly different because of the change in the *Standard Key Component*. Instead of the row, we would have any of the three indexes. For instance, using the x index:

RCP	x	f
ES	1	200
ES	2	300
PT	1	500
PT	2	600

Note that the important thing for DPM-ML to be consistent is that all the *Records* with the same row share the same index.

2.2.4.4 KEY VARIABLE SELECTION

F 20.05.a - Geographical breakdown of off-balance sheet exposures by residence of the counterparty (a)

ES

			Columns
			Nominal amount
			0010
Rows	Loan commitments given	0010	100
	Financial guarantees given	0020	200
	Other commitments given	0030	300

PT

			Columns
			Nominal amount
			0010
Rows	Loan commitments given	0010	400
	Financial guarantees given	0020	500
	Other commitments given	0030	600

```
{tF_40.01, c0015}
```

Selects an Open *Key Variable*. The *Recordset*, in the context of a single instance, has two DPM *Key Components*, because the selected *Variable* (the one rendered in Column 0015 of Report Table F 40.01) is associated in the DPM to a *Key* with two *Key Variables* (in fact, it is part of the *Key*). These *Key Variables* are associated to Properties with codes "LIN" (rendered in Column 0011) and "TYC" (rendered in Column 0015).

The following table represents the *Recordset*:

LIN	TYC	f
123456	LEI	LEI
123456	ISIN	ISIN
1111	LEI	LEI

Note that the resulting *Recordset* has the values for the type of code two times, one as *Key Component* and another as *Fact Component*. This is necessary to allow doing calculations on the *Fact* while keeping all the *Records* in the *Recordset* uniquely identified.

2.2.3 2.3 Data Types

DPM *Operations* data types are defined with the principle: Create only a data type if they have different behaviour to other data types with at least one *Operator*.

The data types are:

- **String**: Sequence of alphanumeric characters of any length.
- **Number**: Is a rational number of any magnitude and precision, also used as approximation of a real number. Numbers can be treated as points or as intervals. Intervals are defined as a centre plus and minus a radius (c ± r). The centre is the input value provided. The radius is calculated based on the precision of the number as `10^-p/2`. This specification does not mandate a specific representation standard for numbers. The choice of representation (e.g., IEEE 754 binary or decimal floating-point, arbitrary-precision decimals) is left to implementers, who should balance precision, performance, and the requirements of their technology stack.
- **Integer**: Positive and negative integer numbers and zero. It Is a subtype of the type Number. As Number subtypes, Integers can be treated as intervals.
- **Date**: Represents a time reference following ISO 8601, written as a specific date (e.g., `#2022-01-15#`).
- **Boolean**: denotes a logical binary state, meaning either `true` or `false`.
- **Category items**: A reference to the code of a Category Item.

2.3.1 Correspondences with DPM Data Types

The DPM provides a wider list of data types. This is because greater granularity in the data typology may be useful for reporting or representation purposes (e.g., a monetary amount requires a currency attribute, while a pure number does not).

For each DPM data type there is one, and only one DPM-XL *Data type*.

Code	Name	DPM-XL type
i	integer	Integer
r	decimal	Number
s	string (non empty)	String
b	boolean	Boolean
t	true	Boolean
dt	date time	Date
d	date	Date
e	enumeration	Category Items
m	monetary	Number
p	percentage	Number
u	URI	String
o	ordinals	Integer
es	string (including empty string)	String

2.3.2 Casting

The casting between data types is possible. Casting can be done explicitly, if the cast *Operator* is used, or implicitly when it is allowed. The following table contains all the casting options.

	String	Number	Integer	Date	Boolean	Item
String		Explicit	Explicit	Explicit	Explicit	Explic
Number	Implicit		Explicit	Not possible	Not possible	Not po
Integer	Implicit	Implicit		Not possible	Not possible	Not po
Date	Implicit	Not possible	Not possible		Not possible	Not po
Boolean	Implicit	Not possible	Not possible	Not possible		Not po
Item	Implicit	Not possible	Not possible	Not possible	Not possible	
Subcategory	Implicit	Not possible	Not possible	Not possible	Not possible	Not po

Although the Integer → Date cast is marked *Not possible*, a *Date* can be built from three Integers using the `date(year, month, day)` *Operator* (see §10.3). Conversely, individual Integer components of a *Date* can be obtained with the extraction *Operators* in §10.2.

2.2.4 2.4 Language Conventions

2.4.1 Case Sensitivity

DPM-XL keywords are **lowercase**. The keywords listed in this specification (e.g., `if`, `then`, `sum`, `where`) must be written in lowercase as defined in the grammar.

Identifiers (table codes, property codes, operation names) are case insensitive: `CNT`, `cnt`, and `Cnt` refer to the same identifier.

2.4.2 Conventions to Write Scalars

To the effects of DPM-XL, when writing *Scalars*, the following conventions should be followed:

- **String**: Between double quotes ("this is a string") or single quotes ('this is also a string'). Empty strings are represented as `""` or `''`. An empty string is a distinct String value and is **not** equivalent to `null`.
- **Number**: Sequence of numeric digits. Dot (".") shall be used to separate the decimal and the integer part.
- **Integer**: Sequence of numeric digits.
- **Percent**: Written as an integer or decimal followed by `%` (e.g., `25%`, `0.5%`). A percent literal is stored as a Number equal to the written value divided by 100: `25%` is equivalent to `0.25`. There is no distinct Percent type.
- **Date**: Written between hashes (#), following ISO 8601. Example: `#2021-11-25#`.
- **Boolean**: true or false.
- **Items**: An item signature between square brackets, in the format [namespace:code]. E.g., [eba_RT:x11], [eba_CU:iso_currencies]. Item signatures are assumed to be unique within a given DPM release.

2.4.3 Reserved Words

The following words are reserved in DPM-XL and cannot be used as identifiers unless escaped:

Logical operators: `true`, `false`, `and`, `or`, `xor`, `not`

Arithmetic and numeric operators: `abs`, `exp`, `ln`, `sqrt`, `power`, `log`, `max`, `min`

Aggregate operators: `max_aggr`, `min_aggr`, `sum`, `count`, `avg`, `median`, `rank`

Analytical (windowing) keywords: `over`, `partition by`, `order by`, `asc`, `desc`, `data points`, `range`, `between`, `unbounded` (reserved within the `over` clause)

Comparison and set operators: `match`, `isnull`, `in`

Set operators: `set_of`, `union`, `intersect`, `setdiff`, `symdiff`

Conditional operators: `if`, `endif`, `then`, `else`, `nvl`, `filter`

Clause operators: `where`, `get`, `rename`, `to`, `sub`, `group by`

Selection and assignment: `with`, `null`

Time operators: `time_shift`, `annualise`, `year`, `semester`, `quarter`, `month`, `week`, `day`, `date`

String operators: `len`

Time period codes: `A`, `S`, `Q`, `M`, `W`, `D`

Contextual identifiers: `refPeriod` and `entityID` (contextual *Key Components*, see section 2.2.3.1.4) and `baseCurrency`, `decimalsInteger`, `decimalsMonetary`, `decimalsPercentage` and `decimalsDecimal` (contextual *Attribute Components*, see section 2.2.3.3.1) are reserved identifiers with special meaning. They cannot be used as regular property codes or operation names.

Selection mode keywords: `interval`, `default`, the *Parameter Selection* scalar type names `number`, `integer`, `string`, `date`, `boolean`, `item`, and the set type names `set-number`, `set-integer`, `set-string`, `set-date`, `set-boolean`, `set-item` are reserved within selection expressions (inside `{}`). Outside selections, these words have no reserved meaning.

Selection prefixes: The single letters `t`, `g`, `r`, `c`, `s`, `v`, `o`, `p` are reserved as selection prefixes (table, table group, row, column, sheet, variable, operation, parameter) at the start of an identifier inside `{}`. An optional underscore between the prefix and the code is allowed as a cosmetic separator (`tT1` ≡ `t_T1`). Codes that themselves start with an underscore are addressed by backtick-escaping the code, e.g. `` t`_meta` ``.

2.4.4 Escaping Reserved Words

If a DPM identifier (e.g., a property code or table code) conflicts with a reserved word, it can be escaped using backticks. For example, if a property is named `sum`, it can be referenced as `` `sum` `` to distinguish it from the `sum` operator.

Escaped identifiers are case sensitive: `` `Sum` `` and `` `sum` `` refer to different identifiers.

2.3 3 DPM-XL Data Selection

The data for the expression language is selected by referencing the DPM *Cells* and *Key Variables* of DPM Report Tables.

The references to the *Cells* and *Key Variables* are done by means of the table codes, ordinate codes and the codes of the Properties associated to the *Key Variables*.

The data selection is done by the interaction of the selection *Operator*, represented by the curly brackets symbol, and the general scope of an expression, provided by the with *Operator*. With this interaction, it is possible to individuate individual DPM *Variables* or sets of them. The actual data are referring to DPM *Variables*, thus making it possible to obtain the data required for the calculations.

The selection of data, together with the DPM definitions, make it possible to determine the *Structure* of the *Recordset* on compile-time.

2.3.1 3.1 Parameter Selection

A *Parameter* is an external input to an *Operation*: a value of a declared *Data Type* (or a set of such values) that is provided as input to the runtime alongside the data instances. Using a *Parameter* in an *Operation* therefore implies that the execution environment must supply a value for the *Parameter* together with the data instances; otherwise, the *Parameter* takes its declared default (or `null` when no default is declared).

Parameters let the same *Operation* be reused across executions that differ only in a threshold, a reference date, an enumeration member, an allowed set of codes, or any other configuration value that is not stored in the *Cells* of the input tables.

Parameters are referenced using the selection *Operator* with the `p` prefix, followed by the declared *Data Type*, and optionally followed by a default value.

3.1.1 Syntax

{ **p**[*]*parameter*, ***type*** *[, _default_value*] }

where:

- **parameter** is the parameter code, following the same conventions as a *Variable* code. As with the other selection prefixes, an optional underscore between `p` and the code is allowed as a cosmetic separator: `pthreshold` and `p_threshold` refer to the same *Parameter*. To reference a code that itself starts with an underscore, backtick-escape the code (e.g. `` p`_legacy_param` ``).
- **type** is the declared *Data Type* of the *Parameter*. Allowed values:
- *Scalar* types: `number`, `integer`, `string`, `date`, `boolean`, `item`.
- *Set* types: `set-number`, `set-integer`, `set-string`, `set-date`, `set-boolean`, `set-item` — a *Set* parameter takes a set of values of the corresponding *Scalar* type at execution time.
- **default_value** is an optional default, written as `default: <literal>` for *Scalar* types, `default: [namespace:code]` for parameters of type `item`, or `default: { <literal>, ... }` / `default: { [ns:c], ... }` for *Set* types. When omitted, the implicit default is `null`.

3.1.2 Semantics

A *Parameter Selection* evaluates to a *Scalar* (for the basic types) or a set value (for the `set-*` types), of the declared *Data Type*. The value is supplied by the execution environment at run time, as part of the input alongside the data instances. If no value is supplied:

- if a `default` is declared in the selection, the *Parameter* takes the default value;
- otherwise, the *Parameter* is `null`.

The declared type fixes the *Data Type* of the resulting value and is used at compile time to check that the *Parameter* is used in a position compatible with its type (e.g. an `integer` *Parameter* cannot be passed where a `date` is required, unless an explicit cast is applied; a `set-item` *Parameter* can be used as the right-hand side of an `in` *Operator*, but not where a *Scalar* is expected).

3.1.3 Additional Constraints

- The default value, when provided, must be compatible with the declared *Data Type*: an `integer` *Parameter* accepts an integer literal default; a `date` *Parameter* accepts a date literal default; an `item` *Parameter* accepts an item reference default; a `set-<type>` *Parameter* accepts a set literal whose elements are of `<type>`.
- The declared *Data Type* must be one of the twelve listed above. *Number Interval*, *Subcategory* and *Recordset*-shaped *Parameters* are not supported by this syntax.

3.1.4 Runtime Binding

Because a *Parameter* is not part of the data instances, **using a *Parameter* in an *Operation* changes the contract between the *Operation* and its execution environment**: the engine must accept the bindings for every *Parameter* referenced by the *Operation* (and by any other *Operation* it transitively depends on), in addition to the usual data instances. How those bindings are delivered (configuration file, command-line argument, REST parameter, etc.) is left to the implementer; this specification only requires that the engine resolve each *Parameter* code to a value of the declared type at evaluation time, falling back to the declared default — or to `null` — when no binding is supplied.

3.1.5 Examples

Declaring a *Parameter* with no default — evaluates to `null` if no binding is supplied at runtime:

```
{p_threshold, number}
```

Declaring a *Parameter* with an explicit default:

```
{p_threshold, number, default: 0}
{p_cutoff, date, default: #2025-01-01#}
{p_ccy, item, default: [eba_CU:EUR]}
```

Using a *Parameter* in an *Operation* — the same expression can be re-executed against different thresholds without modifying the DPM:

```
{tT1, r010, c010} > {p_threshold, number, default: 0}
```

Using a *Parameter* inside a `where` clause to filter *Records* by an externally-supplied reference date:

```
{tT1}[where refPeriod >= {p_cutoff, date}]
```

Using an Item-typed *Parameter* with a default — filters *Records* by currency, defaulting to `EUR` when no binding is supplied:

```
{tT1}[where ccy = {p_ccy, item, default: [eba_CU:EUR]}]
```

Declaring a *Set Parameter* and using it as the right-hand side of `in` — the runtime supplies the set of allowed currencies, falling back to a single-element default when unbound:

```
{tT1}[where ccy in {p_ccys, set-item, default: {[eba_CU:EUR]}}]
```

Referencing a parameter whose code starts with an underscore via backtick escaping:

```
{p`_legacy_param`, number}
```

2.4 4 Null in DPM Expression Language

2.4.1 4.1 Null and Data Types

All the *Data Types* are assumed to contain the conventional value null, which means "no value", or "absence of known value" or "missing value". Note that the null value, therefore, is the only value of multiple different types.

2.4.2 4.2 Implicit and Explicit Null Values

For *Recordsets*, nulls may arise in a selection for two reasons:

- **Explicit null**: In the input data for the engine, there is a *Record* without a value.
- **Implicit null**: In the input data for the engine, there is no *Record* for one of the *Variables* in the selection. In this case, the engine will generate a *Record* for that *Variable* with a null value, or another specified default (see Selection *Operator* in part 2).

In any case, there are no differences in how implicit or explicit nulls are treated.

4.2.1 Implicit Nulls Generation

Generation of implicit nulls is necessary because of the default values. Whenever a set of *Variables* is selected, all the selected *Variables* need to be created, even if they have not been reported (implicit null).

For **closed tables**, one *Record* shall be generated for each variable in the selection.

For **open tables**, for each combination of *Keys* that has at least one value reported (including explicit nulls), one *Record* for each non existing *Variable* shall be generated.

2.4.3 4.3 Null Treatment

In general, most of the *Operations* return null when any of their arguments is null.

- **Comparison Operators** (e.g., =, >): if a null is involved in the operation, then the result is null.
- **Arithmetic Operators** (e.g., + , -, *): if a null is involved in the operation, then the result is null.
- **String Operators**: if a null is involved in the operation, then the result is null.
- **Logical Operators** (and, or, xor, not): Three-value logic is adopted with the consideration that null means unknown. The concrete results for each *Operator* are specified in part 2, in the description of the *Operators*.
- **Conditional Operators**: null is considered equivalent to false.
- **Filtering**: null is considered equivalent to false (i.e., the *Records* with a null value are not selected in the filter).
- **Aggregations** (e.g., sum, avg, max): nulls are excluded from the calculations. If all input values are null, the result is null. The exception is count, which returns 0 in that case and never returns null.
- **Intervals**: If the centre is null, the radius is also null.

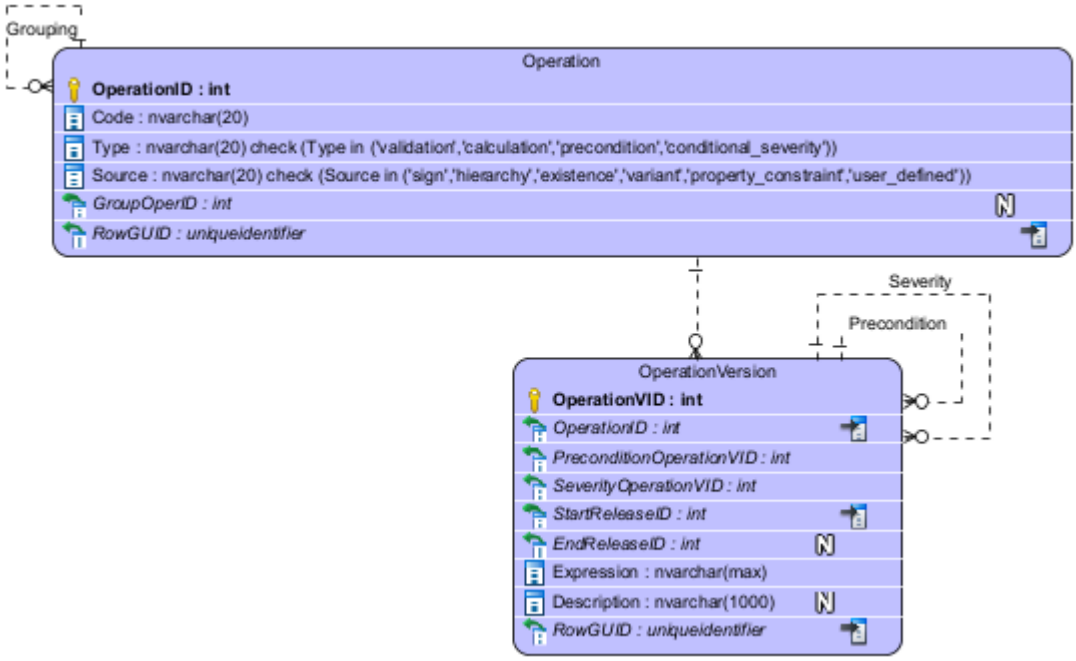
2.4.4 4.4 Overriding the Standard Null Treatment

The standard null treatment can be overridden by substituting the null values for other values. This can be done in two ways:

- **With the Selection Operator**: It is possible to set a default value in the selection for the null values.
- **With the nvl Operator**: serves to substitute nulls for any value.

2.5 5 DPM-ML Metamodel

2.5.1 5.1 Operation and OperationVersion



Operations have a code and can be grouped.

Groups of *Operations* refer to the cases where users define a parent validation and the system derives automatically children validations. There can only be two levels of *Operations* (i.e., a child validation cannot be the parent of another validation). The types of validations that generate children are Variant and Property constraint validations (see below).

Operations and may be of any of 4 types:

- **Validation**: *Operations* that serve to validate the consistency of data. They can provide as output a *Scalar* Boolean value or a *Recordset* with *Scalar* values.
- **Calculation**: *Operations* that serve to derive new data. They can provide as output any kind of data.
- **Precondition**: *Operations* that serve to determine whether one or more validations shall be applied. They can only provide as output a *Scalar* Boolean value.
- **Conditional_severity**: *Operations* that serve to determine the severity of an invalid result for a validation. It can yield a value 'error' or 'warning'

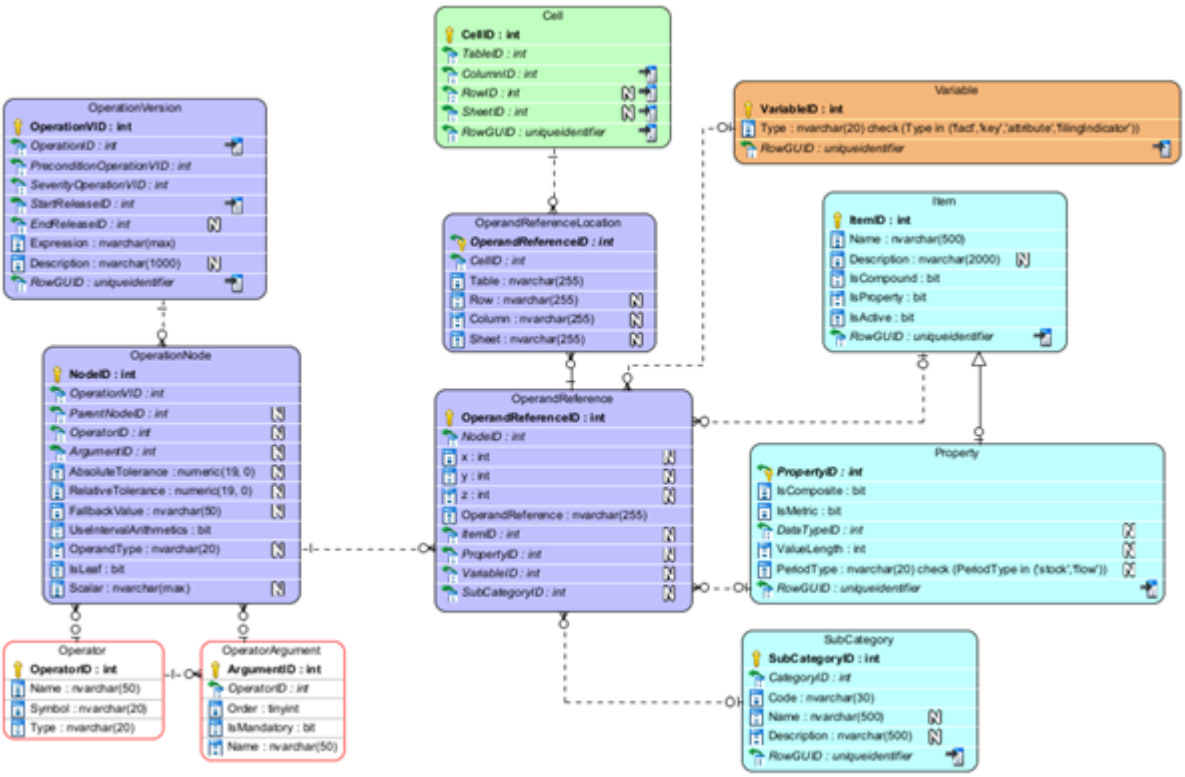
Operations are also classified based on their source for generation as:

- **User_defined**: *Operations* that have been defined by users.
- **Hierarchy**: Validations that have been automatically generated by the system based on the contents of subhierarchies.
- **Sign**: Validations that have been automatically generated by the system based on the contents of the Sign attribute for TableVersionCells.
- **Existence**: Validations that have been automatically generated by the system based on the IsNullable attribute for TableVersionCells.
- **Variant**: Validations that are defined at the level of TableGroups, including the individual validations generated from them.
- **Property_constraint**: Validations that are defined at the level of a property, and from which individual instances of validations based on *Variables* are generated.

Operations can have many versions. A new version of an *Operation* is required whenever any of the following attributes change:

- **Expression**: The DPM-XL expression of the version.
- **Description**: Natural language description of what the validation is aimed to check.
- **PreconditionOperation**: A link to an *Operation* version that acts as precondition.
- **SeverityOperation**: A link to an *Operation* that determines the conditional severity.

2.5.2 5.2 Operation Tree



5.2.1 Operation Nodes

The tree representation of an *OperationVersion* is provided in the *OperationNode* table.

Any *Operation* can be represented as a hierarchical tree, where the arguments of one *Operator* are represented.

For instance, the expression:

```
A = B + C
```

Can be represented as:

```
    =
   / \
  A   +
     / \
    B   C
```

Where "=" and "+" are *Operators*, and "A", "B" and "C" are *Operands*.

This kind of tree is represented in the *OperationNode* table.

5.2.2 *Operands*' Representation

All *Operands* are represented as *OperationNodes*.

Operands that are *Scalars* not representing a DPM Object (like an Item) are completely represented in the *OperationNode* table.

Scalar Operands that are referring to a DPM Object need to be further specified with the tables *OperandSource* and *OperandReference*, which provide the reference to the DPM Object.

Operands that are *Variables* and Sets of *Variables* need also to be further specified with the tables *OperandSource* and *OperandReference*. Concretely, for Sets of *Variables*, it is important to determine the index of each *Variable* in the context of the *Operand*, to be able to match the *Variables* to which the *Operator* applies.

5.2.3 Representation Example

For instance, suppose a table (Tab1) with three rows (100, 200 and 300), two columns (100 and 200) and two sheets.

Sheet A

	Column 100	**Column 200**
Row 100	1	4
Row 200	2	5
Row 300	3	6

Sheet B

	Column 100	**Column 200**
Row 100	7	10
Row 200	8	11
Row 300	9	12

Note that each number in the cells represents the ID of the corresponding *Variable*.

The following expression:

```
{tTab1, r100, c*, s*} = 2 * ({tTab1, r200, c*, s*} + {tTab1, r300, c*, s*})
```

Would be represented as the following tree:

```
                =
              /   \
{tTab1, r100, c*, s*}   *
                      /  \
                     2    +
                        /  \
                {tTab1, r200, c*, s*} {tTab1, r300, c*, s*}
```

The representation in the DB would be as follows (only the relevant fields are shown, and for *Operators* and argument symbols are used):

OperationNode Table

NodeId	ParentNodeID	*Operator*	Argument	OperandSourceId	*Scalar*
1		=			
2	1		left	oprnd1	
3	1	*	right		
4	3		left		2
5	3	+	right		
6	5		left	oprnd2	
7	5		right	oprnd3	

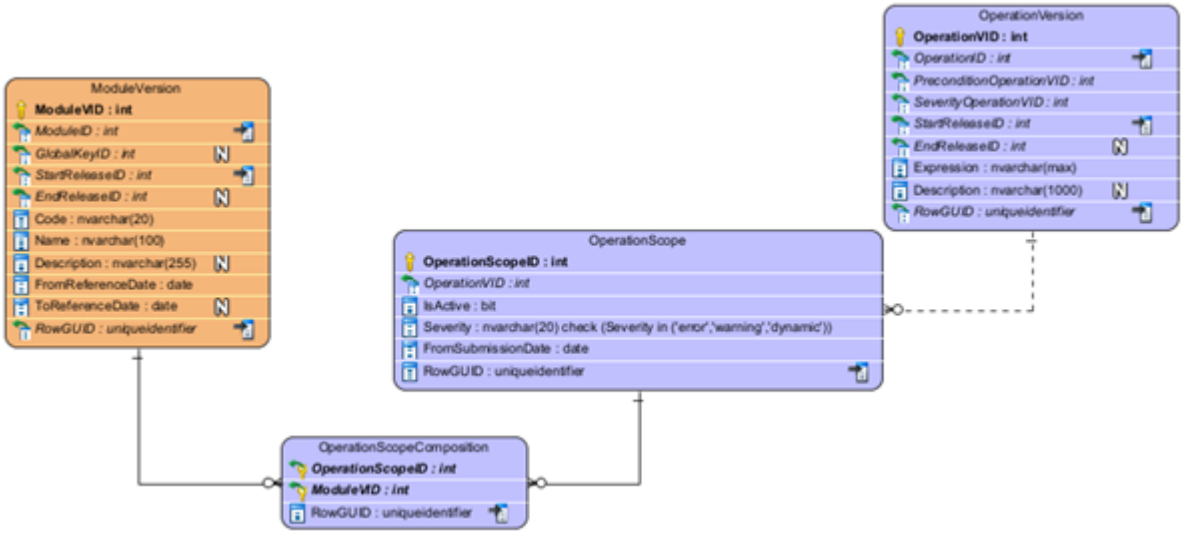
OperandReference Table

The `x`, `y` and `z` columns hold the index of the *Variable* on the row, column and sheet axis respectively. In this expression the row is fixed on each *Operand*, so `x` is not used; the column and sheet axes vary, and are indexed by `y` and `z`.

NodeID	x	y	z	VariableID
2		1	1	1
2		2	1	4
2		1	2	7
2		2	2	10
6		1	1	2
6		2	1	5
6		1	2	8
6		2	2	11
7		1	1	3
7		2	1	6
7		1	2	9
7		2	2	12

Note that the *Operations* shall be applied to the *Variables* having the same index. For instance, *Variable* 1 = 2 * (*Variable* 2 + *Variable* 3), because all of them have the index y=1 and z=1.

2.5.3 5.3 Operation Scope

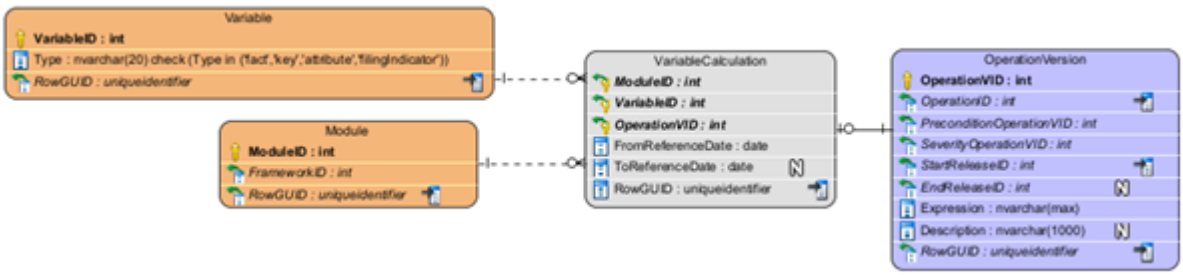


Each *Operation* version can be applied in different scopes. An *Operation* scope refers to the individual module versions or sets of module versions (for cross-module validations) to which the *Operation* logically applies.

For each *Operation* scope, there may be different values for the attributes:

- **IsActive**: Determines whether the *Operation* shall be run for a certain scope.
- **Severity**: Determines the severity of the error, if the validation is not passed.
- **FromSubmissionDate**: Sets a date from which the validation applies to subsequent submissions.

2.5.4 5.4 Variable Calculation



For calculations (i.e., *Operations* that serve to calculate values for *Variables*), it is necessary to link the *Operation* to the actual *Variable* that is generated. This is done with the *VariableCalculation* table, which links to the *Variable* and the Module. The link to the Module is necessary due to the fact that a *Variable* can be used in several Modules, being calculated in some cases and not calculated in other cases.

3. Operators

3.1 1 Introduction

This documentation provides the individual description of the semantics for all the *Operators* in the DPM Operations (DPM-XL and DPM-ML).

The document is structured from more generic to more concrete. It starts with general behaviours that apply to several *Operators* of different types. Then it follows with the common behaviour of the *Operators* of the same type, finalizing with the specific behaviour of the single *Operators*.

3.1.1 1.1 Operator Precedence

The table below lists operator precedence from lowest (level 1) to highest (level 12), derived from the order of alternatives in the grammar's `expression` rule. Operators at higher levels bind more tightly. When operators share a level they associate left-to-right. Use parentheses `()` to override any precedence.

Level	Operator(s)	Category
1	`or`, `xor`	Logical disjunction
2	`and`	Logical conjunction
3	`in`	Set membership
4	`=`, `<>`, `<`, `>`, `<=`, `>=`	Comparison
5	`||`	String concatenation
6	`+`, `-`	Additive
7	`*`, `/`	Multiplicative
8	`not(...)`	Logical negation
9	`+`, `-` (unary)	Unary sign
10	`[where ...]`, `[sub ...]`, `[rename ...]`, `[get ...]`	Clause operators (postfix)
11	Function calls, literals, selections, item references	Atomic expressions
12	`(...)`	Parenthesised grouping

`if ... then ... else ... endif` encloses a full expression and is always evaluated as a unit; its precedence is that of the enclosed expressions.

This ordering mirrors the precedence structure of VTL (Validation and Transformation Language), which DPM-XL is based on.

3.2 2 Common General Behaviours for the Operators

3.2.1 2.1 Unary Operators

This general behaviour is applied to *Operators* taking as an argument one single *Operand*.

Unless explicitly specified differently, the behaviour for the unary *Operators* is as follows:

2.1.1 Scalars

The *Operator* is applied on a *Scalar* value and returns a *Scalar* value.

2.1.2 Recordsets

The *Operator* is applied on a *Recordset* and returns a *Recordset*.

The *Operator* is applied to the values of all the facts of the *Recordset*.

2.1.2.1 CONSTRAINTS

1. The *Data Type* of the *Fact Component* of the *Operand Recordset* must be compatible with the *Operator*.

2.1.2.2 STRUCTURE

The structure of the resulting *Recordset* contains the *Key* and *Fact Components* of the *Operand Recordset*, the *Attribute Components* are not propagated.

2.1.2.3 RECORDS

The *Operators* produce one output *Record* per each input *Record*, which have:

- For *Key Components*, the values unchanged.
- For the *Fact* values, the *Operator* is applied to the input value and returns the corresponding value.

3.2.2 2.2 Binary Operators

This general behaviour is applied whenever an *Operator* takes as input two *Operands*, which includes the following *Operators*:

- Arithmetic Binary Operators
- Comparison Operators
- Logical Binary Operators

Unless explicitly specified differently, the behaviour for the binary *Operators* is as follows.

2.2.1 Scalars

If the two *Operands* of a binary *Operator* are *Scalars*, the result shall be the *Scalar* resulting of applying the *Operator* to the *Operands*.

2.2.2 Recordset and Scalar

A binary *Operator* applied to a *Recordset Operand* and a *Scalar*, will result in a *Recordset* with the same *Key* and *Fact Components* as the input *Recordset Operand* (*Attribute Components* are not propagated). The *Operator* shall be applied to every *Record* of the input *Recordset* and the *Scalar*.

2.2.2.1 EXAMPLES

```
0.25*{tS_26.01, r0600, cNNN}
```

Supposing that the selection yields the following *Recordset*:

c	f
0060	100
0080	200

The operation results in:

c	f
0060	25
0080	50

2.2.3 Recordsets

2.2.3.1 CONSTRAINTS

Binary *Operators* can only be applied to two *Recordsets Operands* if they have:

1. exactly the same *Key Components*; or
2. the *Key Components* of one *Recordset* (Reference *Recordset*) are a superset of the *Key Components* of the other *Recordset*.
3. *Standard Keys* contain the same values. E.g., summing `{tT1, r010-020}` and `{tT1, r030-040}` is not possible, because there are no matches in the operands.

2.2.3.2 STRUCTURE

The *Operator* yields a *Recordset* with the common *Key Components* in case 1, or the *Key Components* of the Reference *Recordset* in case 2. The resulting *Recordset* does not contain any *Attribute Component* from the *Operand Recordsets*.

2.2.3.3 RECORDS

The *Operator* applies to the pairs of values resulting from performing an inner join of the input *Recordsets* on the common *Key Components*.

2.2.3.4 EXAMPLES

2.2.3.4.1 Same Identifiers

```
{tC_28.00, c040} + {tC_28.00, c190}
```

Supposing that the selections yield:

{tC_28.00, c040}

INC	f
123	1000
456	2000
789	3000

{tC_28.00, c190}

INC	f
123	-100
456	-200
789	-300

The result would be:

INC	f
123	900
456	1800
789	2700

2.2.3.4.2 Subset of Identifiers

```
{tF_40.01, c0110} >= {tF_40.02, c0060}
```

Supposing that the selections yield:

{tF_40.01, c0110}

LIN	TYC	f
123	x1	1
456	x1	0.8
789	x1	0.4

{tF_40.02, c0060}

LIN	TYC	STC	LHC	LHO	f
123	x1	111	ABC	x1	0.3
123	x1	111	DEF	x1	0.7
456	x1	222	ABC	x1	0.85

The result would be:

LIN	TYC	STC	LHC	LHO	f
123	x1	111	ABC	x1	true
123	x1	111	DEF	x1	true
456	x1	222	ABC	x1	false

Note that:

- An inner join on the common *Key Components* is performed. This means that there is no comparison for the *Record* in F 40.01 with LIN = 789, because there is no match in F 40.02.
- The result of the operation has the *Structure* of the Reference *Recordset*, which is the one that has a superset of *Key Components*.
- For each match in the join the *Operator* `>=` is applied to the pairs of values for the facts.

2.2.4 Intervals Treatment

When one of two numeric *Operands* is an interval, and the other is a point, the point is transformed into an interval with centre value equal to the point value, and radius 0.

3.3 3 Selection Operators

3.3.1 3.1 Selection Operator

3.1.1 Syntax

All selection forms use a single-letter prefix (`t` , `g` , `v` , `o` , `p`) followed by an identifier code. An optional underscore between the prefix and the code is allowed as a **cosmetic separator** for readability: `tT1` and `t_T1` refer to the same table, `vfoo` and `v_foo` refer to the same *Variable*, etc.

Form 1 — Table reference

{ **t**[*]*table [, **r**row_args] [, **c**col_args] [, **s**sheet_args] [, _interval_arithmetics*] [, *default_value*] }

Form 2 — Table group reference

{ **g**[*]*table_group [, **r**row_args] [, **c**col_args] [, **s**sheet_args] [, _interval_arithmetics*] [, *default_value*] }

Form 3 — Variable reference

{ **v**[_]variable }

No row, column, sheet, or other arguments are permitted.

Form 4 — Operation reference

{ **o**[_]operation [, **r**row_args] [, **c**col_args] [, **s**sheet_args] }

Form 5 — Parameter reference

{ **p**[*]*parameter, **type** [, _default_value*] }

A *Parameter* is an execution-time value of a declared *Data Type* — either a *Scalar* (for the basic types) or a set of values (for the `set-*` types) — supplied as input to the runtime alongside the data instances. See §3.1 Parameter Selection for the full specification. Unlike Forms 1–4, a *Parameter Selection* produces a *Scalar* or a set, not a *Recordset*.

Escaping codes with leading underscores or other special characters

A code that starts with an underscore (or contains characters not allowed in the bare code) can be referenced by backtick-escaping the code part after the prefix:

`{ v`_internal` }` — selects the *Variable* whose code is `_internal` .

`{ t`_meta` }` — selects the table whose code is `_meta` .

The escape applies only to the code; the prefix stays outside the backticks. Without escaping, codes must start with a letter.

where *row_args*, *col_args*, and *sheet_args* follow the pattern:

rrow [, **r**row]* | **r**row_init–row_end | **r***

ccol [, **c**col]* | **c**col_init–col_end | **c***

ssheet [, **s**sheet]* | **s**sheet_init–sheet_end | **s***

3.1.2 Input Parameters

- **table**: The code of a DPM Report table.
- **variable**: the code of a DPM *Variable*.
- **operation**: The code of a DPM *Operation*.
- **row, row_init, row_end**: A Reference to the code of a row of a DPM table.

- **col, col_init, col_end**: A Reference to the code of a col of a DPM table.
- **sheet, sheet_init, sheet_end**: A Reference to the code of a sheet of a DPM table.
- **interval_arithmetics**: A Boolean value specifying whether interval arithmetics should apply. Only applicable to numeric types (Number, Integer). When true, numeric values are treated as intervals with centre equal to the value and radius calculated from the precision of the number. The default value, in case the parameter is omitted, is false.
- **default_value**: A *Scalar* value specifying the value to be used in case of nulls. The value must be compatible with the *Data Type* of the selected *Variable*. The default value, in case the parameter is omitted, is null.
- **parameter**: The code of a *Parameter* (back-door execution-time *Scalar*).
- **type**: The declared *Data Type* of the *Parameter*. One of the scalar types `number`, `integer`, `string`, `date`, `boolean`, `item`, or the set types `set-number`, `set-integer`, `set-string`, `set-date`, `set-boolean`, `set-item`. See §3.1 Parameter Selection.

3.1.3 Output

rset<*> for Forms 1–4

scal<> *for Form 5 with a scalar* `type` *(*`number`, `integer`, `string`, `date`, `boolean`, `item`*) — the* Data Type *is the one declared in the* Parameter Selection*.

set<> *for Form 5 with a* `set-*` `type` *(*`set-number`, `set-integer`, `set-string`, `set-date`, `set-boolean`, `set-item`*) — a set of values of the element type declared in the* Parameter Selection*.

3.1.4 Semantics

Serves to select data as defined in the DPM model. *Parameter Selections* (Form 5) additionally serve to inject *Scalar* values or sets of values supplied externally at execution time; see §3.1 Parameter Selection.

3.1.5 Additional Constraints

Constraints 1–5 apply to Forms 1–4. Constraint 6 applies to Form 5.

1. Existence of the parameters. The referred table, *Variable* or *Operation* need to be defined in the DPM.
2. *Operations* need to be defined in the same script.
3. Rows, columns and sheets need to exist in the table or *Operation* on which they are defined.
4. When the selection is done at *Variable* level, row, column and sheet parameters cannot exist.
5. The interval parameter can only be set to true when the selected *Variable* has a numeric *Data Type* (Number or Integer).
6. The declared **type** of a *Parameter Selection* must be one of the supported types and the optional default value must be compatible with that type:

- *Scalar* types: `number`, `integer`, `string`, `date`, `boolean`, `item`. The default is a single literal (or an item reference for `item`).
- *Set* types: `set-number`, `set-integer`, `set-string`, `set-date`, `set-boolean`, `set-item`. The default is a set literal whose elements are of the corresponding scalar type (e.g. `default: {"AAA", "BBB"}` for `set-string`, `default: {[eba_CU:EUR]}` for `set-item`).

3.1.6 Behaviour

Returns a *Recordset* with the following structure:

Key Components:

- DPM-XL
- Row: If the selected table has ordinate rows and more than one row is present in the selection. The name of the *Component* is "r".
- Column: If the selected table has ordinate columns and more than one column is present in the selection. The name of the *Component* is "c".
- Sheet: If the selected table has ordinate sheets and more than one sheet is present in the selection. The name of the *Component* is "s".
- DPM-ML
- X: If there is Row for DPM-XL.
- Y: If there is Column for DPM-XL.
- Z: If there is Sheet for DPM-XL.
- A DPM *Key Component* for each *Key Variable* belonging to the *Key* associated to the selected *Variable*. The name of the *Component* is the *Code* of the Property associated to the *Key Variable*. The *Data Type* of the *Component* shall be the correspondent *Data Type* of the Metric or Property associated to the respective *Key Variable*.

Fact Component: With the *Data Type* corresponding to the selected *Variable*.

Attribute Components: One *Component* for each *Attribute Variable* associated to the selected *Variable*. The name of the *Component* is the *Code* of the Property associated to the *Attribute Variable*. The *Data Type* of the *Component* shall be the correspondent *Data Type* of the Metric or Property associated to the respective *Attribute Variable*.

On top of these, the contextual *Components* are latent in every selection, whether or not the selected *Variables* declare them: the *Key Components* `refPeriod` and `entityID` (§2.2.3.1.4) and the contextual *Attribute Components* (§2.2.3.3.1). They are added to the data structure of the *Recordset* only where an expression references them.

The *Records* for the *Recordset* shall be obtained from the input data according to the selection.

Ranges (e.g., `r0010-0100`) select all *Headers* whose codes fall between the start and end codes, inclusive on both boundaries. The ordering is lexicographic, based on the string representation of the *Header* codes. In the example, all *Headers* with a code lexicographically between `0010` and `0100` are selected.

When a table has a *Header* that is not open, the selection needs to have an explicit value for that *Header*. The wildcard `*` (e.g., `r*`, `c*`, `s*`) is used to select all *Headers* of the corresponding dimension.

When an *Operation* is selected (using the `o` prefix), the resulting *Recordset* has the same structure as the output of the referenced *Operation*. The *Operation* must be defined in the same script, either as a named *Operation* in the DPM or via a non-persistent assignment (`:=`). Row, column, and sheet parameters can be used to further refine the selection on the *Operation* result.

A *Parameter Selection* (Form 5, `p` prefix) returns a *Scalar* (for the basic types) or a set of values (for the `set-*` types) of the declared *Data Type*, rather than a *Recordset*. The value is supplied at execution time as part of the runtime input; when unbound, the *Parameter* takes the declared default, or `null` if no default is declared. See §3.1 Parameter Selection for the full specification.

3.1.6.1 EXAMPLE: USING AN OPERATION AS INPUT

```
op1 := {tT1, r010-020, c010} * 10;
{t01, r010, c010} <- {o0p1, r010} + {tT1, r010, c020}
```

In this example, `op1` is assigned the result of multiplying the selection from table T1 by 10. The second statement selects row 010 from the result of `op1` (using the `o` prefix) and adds it to a selection from table T1. The structure of `{o0p1, r010}` is determined by the output of `op1`, filtered by row 010.

3.1.7 DPM-ML Metamodel Representation

There is no metamodel representation for the selection *Operator*. It is used to select, in last instance, *Variables*. So the result of the selection are processed to get the relevant *Variables* for DPM-ML, as well as the *DefaultValue* and *UseIntervalArithmetics* attributes.

3.3.2 3.2 With

3.2.1 Syntax

with partial_selection [where_expression][1]: *expression*

3.2.2 Input Parameters

- **partial_selection**: A selection expression (see Selection *Operator*)
- **expression**: An expression including selection *Operators*
- **where_expression**: An expression with a `where` operator

3.2.3 Output

Does not generate output; modifies the selections of the expression after the colon.

3.2.4 Semantics

Serves to simplify expressions by adding a single context that may apply to all the selection operators in the expression.

3.2.5 Additional Constraints

In case *where_expression* is used:

- All the operands rising from selection operators have a `where` or a `sub` clause applicable, the one in the with, or the one overriding it. When the *where_expression* references *Key Components*, this implies that it is not possible to have a selection with a table that does not have open keys. A *where_expression* that only references the *Fact Component* (e.g. `[where f != 0]`) is applicable to any selection.

3.2.6 Behaviour

The selection parameters in the partial selection applies to all the selections in the expression, unless they are overridden by a more specific parameter.

Therefore, in a selection inside the expression, whenever one of the parameters of the selection expression is not present, but that parameter is present on the partial selection, the parameter of the partial selection applies.

In case *where_expression* is used, the overriding mechanism works as follows:

- Any `where` clause used inside the expression fully overrides the *where_expression*, even if the properties used in the `where` clause inside the expression are different.
- A `sub` clause used inside the expression fully overrides the *where_expression*.

3.2.7 Examples

EXAMPLE 1

```
with {tF_01.01, c0010, default: 0, interval: false}:
{r0010} = {r0020} + {r0030} + {r0040}
```

The partial selection applies to all the selections in the expression

EXAMPLE 2

```
with {tF_01.01, c0010, default: 0, interval: false}:
{r0010} + {r0040} = {tF_04.01, r0010, c0010}
```

The partial selection applies to the two selections that do not refer to a table. Regarding the other selection, the default and intervals apply

EXAMPLE 3

```
with {tF_01.01, c0010, default: 0, interval: false}:
{tF_01.01, r0010} + {tF_01.01, r0040} = {tF_04.01, r0010, c0010, default: null}
```

The partial selection applies to the two selections that refer to the same table referred to in the partial selection. Regarding the other selection, interval argument in the partial selection applies, but the default is overridden.

EXAMPLE 4

```
with {c0010, default: 0, interval: false}:
{tF_01.01, r0010} + {tF_01.01, r0040} = {tF_04.01, r0010}
```

The partial selection applies to all the selections in the expression.

EXAMPLE 5

```
with {tEXM, r0010}[where id1="ABC" and id2=5]:
   {c0010} = {c0020} + {c0030}
```

Where:

- **tEXM** has two *Key Components*: *id1* and *id2*

The clauses applying to each selection are:

Selection	Clause
`{c0010}`	`[where id1="ABC" and id2=5]`
`{c0020}`	`[where id1="ABC" and id2=5]`
`{c0030}`	`[where id1="ABC" and id2=5]`

EXAMPLE 6

```
with {tEXM, r0010}[where id1="ABC" and id2=5]:
   {c0010} = {c0020} + {tEXM_02, c0030}[where id1="ABC"]
```

Where:

- **tEXM** has two *Key Components*: *id1* and *id2*
- **tEXM_02** has one *Key Component*: *id1*

The clauses applying to each selection are:

Selection	Clause
`{c0010}`	`[where id1="ABC" and id2=5]`
`{c0020}`	`[where id1="ABC" and id2=5]`
`{tEXM_02, c0030}`	`[where id1="ABC"]`

EXAMPLE 7

```
with {r0010}[where id1="ABC"]:
   {tEXM, c0010} = {tEXM, c0020} + {tEXM_02, c0030}
```

Where:

- **tEXM** has two *Key Components*: *id1* and *id2*
- **tEXM_02** has one *Key Component*: *id1*

The clauses applying to each selection are:

Selection	Clause
`{tEXM, c0010}`	`[where id1="ABC"]`
`{tEXM, c0020}`	`[where id1="ABC"]`
`{tEXM_02, c0030}`	`[where id1="ABC"]`

EXAMPLE 8

```
with {default: 0, interval: true}[where qEEA = [eba_qAE:qx2010]]:
if
     sum({tC_08.01.a, r0070, c0255})
         =
     sum({tC_08.01.a, r0010, c0255})
then
     sum
         ( filter ({tC_08.02, c0255}, {tC_08.02, c0010} = 1))
           =
     sum({tC_09.02, r0090, c0120} [where CEG = [eba_GA:qx2014]])
endif
```

Where:

- **C_08.01.a** has one *Key Component*: *qEEA*
- **C_08.02** has two *Key Components*: *qEEA* and *OGR*
- **C_09.02** has one*Key Component*: *CEG*

The clauses applying to each selection are:

Selection	Clause
`{tC_08.01.a, r0070, c0255}`	`[where qEEA = [eba_qAE:qx2010]]`
`{tC_08.01.a, r0010, c0255}`	`[where qEEA = [eba_qAE:qx2010]]`
`{tC_08.02, c0255}`	`[where qEEA = [eba_qAE:qx2010]]`
`{tC_08.02, c0010}`	`[where qEEA = [eba_qAE:qx2010]]`
`{tC_09.02, r0090, c0120}`	`[where CEG = [eba_GA:qx2014]]`

EXAMPLE 9

```
with {tEXM, r0010}[where id1="ABC" and id2=5]:
   {c0010} = {c0020} + {tEXM_02, c0030}[sub id1="ABC"]
```

Where:

- **tEXM** has two *Key Components*: *id1* and *id2*
- **tEXM_02** has one *Key Component*: *id1*

The clauses applying to each selection are:

Selection	Clause
`{c0010}`	`[where id1="ABC" and id2=5]`
`{c0020}`	`[where id1="ABC" and id2=5]`
`{tEXM_02, c0030}`	`[sub id1="ABC"]`

Note that the `sub` clause overrides the `where` in the `with` in the partial_selection.

EXAMPLE 10

```
with {tEXM, r0010}[where id1="ABC" and id2=5]:
   {c0010} = {c0020} + {tEXM_03, c0030}
```

Where:

- **tEXM** has two *Key Components*: *id1* and *id2*
- **tEXM_03** does not have any key component

This expression raises a *Semantic Error*, because the where clause applies to the operand `{tEXM_03, c0030}`, and this table doesnt have the referred *Key Components*.

3.2.8 Metamodel Representation

See selection *Operator*.

3.4 4 Numeric Operators

Numeric *Operators* describe operations involving *Operands* with data type Number, Integer or Number Interval.

3.4.1 4.1 Numeric Operators' General Behaviour

4.1.1 Input Parameters

Operands can be of *Recordset* or *Scalar* type. Must be defined as *Number interval*, *Number*, or *Integer*.

4.1.2 Result Type

Recordset or *Scalar* with type *Number interval*, *Number*, or *Integer*.

4.1.3 Behaviour

If any *Operand* is null, then the result is also null.

Numeric *Operators* can be applied to any numeric type (*Number interval*, *Number*, or *Integer*) and combination of them, in which case casting to the highest type shall apply.

If the type of all *Operands* is *Integer* then the result has type *Integer*, unless the *Operator*'s declared *Output* type does not include *Integer* (as is the case for division, `exp`, `ln`, `log`, `sqrt`, and `power`), in which case the result is implicitly promoted to *Number* (or *Number Interval*). For example, `7 / 2` returns `3.5`, not `3`. If any of the *Operands* is of type *Number*, and there are no *Number intervals*, then the result has type *Number*. If any *Operand* is of *Number Interval* type, the result has type *Number Interval*.

3.4.2 4.2 Unary Arithmetic Operators

4.2.1 Plus (+)

4.2.1.1 SYNTAX

```
+ op
```

4.2.1.2 INPUT PARAMETERS

Op: rset | scal <num + interval>

4.2.1.3 OUTPUT

rset | scal <num>

4.2.1.4 SEMANTICS

Returns the *Operand* unchanged.

For intervals, returns the interval unchanged (both centre and radius are preserved).

4.2.1.5 ADDITIONAL CONSTRAINTS

None

4.2.1.6 BEHAVIOUR

- Unary *Operators'* standard behaviour.
- Numeric *Operators'* standard behaviour.

4.2.1.7 EXAMPLES

- `+ 1` results in `1`
- `+ (-3)` results in `-3`

4.2.2 Minus (-)

4.2.2.1 SYNTAX

- op

4.2.2.2 INPUT PARAMETERS

Op: rset | scal <num + interval>

4.2.2.3 OUTPUT

rset | scal <num>

4.2.2.4 SEMANTICS

Inverts the sign of the *Operand*.

For intervals, inverts the sign of the centre, leaving the radius unchanged.

4.2.2.5 ADDITIONAL CONSTRAINTS

None

4.2.2.6 BEHAVIOUR

- Unary *Operators'* standard behaviour.
- Numeric *Operators'* standard behaviour.

4.2.2.7 EXAMPLES

- `- 1` results in `-1`
- `- (-3)` results in `3`

4.2.3 Absolute Value (abs)

4.2.3.1 SYNTAX

abs(op **)**

4.2.3.2 INPUT PARAMETERS

Op: rset | scal <num + interval>

4.2.3.3 OUTPUT

rset | scal <num>

4.2.3.4 SEMANTICS

Calculates the absolute value of a Number.

For intervals, return the absolute value of the centre, leaving the radius unchanged

4.2.3.5 ADDITIONAL CONSTRAINTS

None

4.2.3.6 BEHAVIOUR

- Unary *Operators'* standard behaviour.
- Numeric *Operators'* standard behaviour.

4.2.3.7 EXAMPLES

- `abs(-1)` results in `1`
- `abs(3)` results in `3`

4.2.4 Exponential (exp)

4.2.4.1 SYNTAX

exp(op **)**

4.2.4.2 INPUT PARAMETERS

Op: rset | scal <num>

4.2.4.3 OUTPUT

rset | scal

4.2.4.4 SEMANTICS

Returns `e` (base of the natural logarithm) raised to the op power.

4.2.4.5 ADDITIONAL CONSTRAINTS

Number Interval operands are not supported. The operator only accepts Number and Integer types.

4.2.4.6 BEHAVIOUR

- Unary *Operators'* standard behaviour.
- Numeric *Operators'* standard behaviour.

4.2.4.7 EXAMPLES

- `exp(2)` results in `7.38905`
- `exp(1)` results in `2.71828` (the `e` number)

4.2.5 Natural Logarithm (ln)

4.2.5.1 SYNTAX

ln(op **)**

4.2.5.2 INPUT PARAMETERS

Op: rset | scal <num>

4.2.5.3 OUTPUT

rset | scal <num>

4.2.5.4 SEMANTICS

Calculates the natural logarithm of a Number.

4.2.5.5 ADDITIONAL CONSTRAINTS

The numeric values must be greater than 0.

Number Interval operands are not supported. The operator only accepts Number and Integer types.

4.2.5.6 BEHAVIOUR

- Unary *Operators'* standard behaviour.
- Numeric *Operators'* standard behaviour.
- If any value is smaller than or equal to 0, generates a run-time error.

4.2.5.7 EXAMPLES

- `ln(1)` results in `0`
- `ln(148)` results in `4.997`

4.2.6 Square Root (sqrt)

4.2.6.1 SYNTAX

sqrt(op **)**

4.2.6.2 INPUT PARAMETERS

Op: rset | scal <num>

4.2.6.3 OUTPUT

rset | scal <num>

4.2.6.4 SEMANTICS

Calculates the square root of a Number.

4.2.6.5 ADDITIONAL CONSTRAINTS

The numeric values must be greater than or equal to 0.

Number Interval operands are not supported. The operator only accepts Number and Integer types.

4.2.6.6 BEHAVIOUR

- Unary *Operators'* standard behaviour.
- Numeric *Operators'* standard behaviour.
- If any value is smaller than 0, generates a run-time error.

4.2.6.7 EXAMPLES

- `sqrt(4)` results in `2`
- `sqrt(25)` results in `5`

4.2.7 Logarithm (log)

4.2.7.1 SYNTAX

log(op, base **)**

4.2.7.2 INPUT PARAMETERS

op: rset | scal <num>

base: scal <num>

4.2.7.3 OUTPUT

rset | scal <num>

4.2.7.4 SEMANTICS

Calculates the logarithm of op in the given base.

4.2.7.5 ADDITIONAL CONSTRAINTS

op numeric values must be greater than 0. base numeric values must be greater than 1.

Number Interval operands are not supported. The operator only accepts Number and Integer types.

4.2.7.6 BEHAVIOUR

- Unary *Operators'* standard behaviour.
- Numeric *Operators'* standard behaviour.
- If the base is 1 or smaller, generates a run-time error.
- If the number is 0 or smaller, generates a run-time error.

4.2.7.7 EXAMPLES

- `log(512, 2)` results in `9`
- `log(100, 10)` results in `2`

3.4.3 4.3 Binary Arithmetic Operators

This *Operators* group follows the General behavior for binary *Operators*.

4.3.1 Addition (+)

4.3.1.1 SYNTAX

left + right

4.3.1.2 INPUT PARAMETERS

left: rset | scal <num + interval>

right: rset | scal <num + interval>

4.3.1.3 OUTPUT

rset | scal <num + interval>

4.3.1.4 SEMANTICS

Returns the sum of two Numbers.

For intervals:

- Centre is calculated as `centre(left) + centre(right)`
- Radius is calculated `radius(left) + radius(right)`

4.3.1.5 ADDITIONAL CONSTRAINTS

None.

4.3.1.6 BEHAVIOUR

- Binary *Operators'* standard behaviour
- Numeric *Operators'* standard behaviour.

4.3.1.7 EXAMPLES

- `3 + 2` results in `5`
- `-7 + 3` results in `-4`

4.3.2 Subtraction (-)

4.3.2.1 SYNTAX

left - right

4.3.2.2 INPUT PARAMETERS

left: rset | scal <num + interval>

right: rset | scal <num + interval>

4.3.2.3 OUTPUT

rset | scal <num + interval>

4.3.2.4 SEMANTICS

Returns the difference of two Numbers.

For intervals:

- Centre is calculated as `centre(left) - centre(right)`
- Radius is calculated `radius(left) + radius(right)`

4.3.2.5 ADDITIONAL CONSTRAINTS

None.

4.3.2.6 BEHAVIOUR

- Binary *Operators'* standard behaviour
- Numeric *Operators'* standard behaviour.

4.3.2.7 EXAMPLES

- `3 - 2` results in `1`
- `-7 - 3` results in `-10`

4.3.3 Multiplication (*)

4.3.3.1 SYNTAX

left * right

4.3.3.2 INPUT PARAMETERS

left: rset | scal <num + interval>

right: rset | scal <num + interval>

4.3.3.3 OUTPUT

rset | scal <num + interval>

4.3.3.4 SEMANTICS

Returns the product of two Numbers.

For intervals:

- Centre is calculated as `centre(left) * centre(right)`
- Radius is calculated as `abs(centre(left) * radius(right)) + abs(radius(left) * centre(right)) + (radius(left) * radius(right))`

4.3.3.5 ADDITIONAL CONSTRAINTS

None.

4.3.3.6 BEHAVIOUR

- Binary *Operators'* standard behaviour
- Numeric *Operators'* standard behaviour.

4.3.3.7 EXAMPLES

- `4 * 6` results in `24`
- `-9 * 2` results in `-18`

4.3.4 Division (/)

4.3.4.1 SYNTAX

num / den

4.3.4.2 INPUT PARAMETERS

num: rset | scal <num + interval>

den: rset | scal <num + interval>

4.3.4.3 OUTPUT

rset | scal <num + interval>

4.3.4.4 SEMANTICS

Divides two Numbers.

For intervals: - Centre is calculated as `centre(num) / centre(den)` - Radius is calculated as:

```
max(
      abs(centre(num) + radius(num)) / (centre(den) + radius(den)),
      abs(centre(num) + radius(num)) / (centre(den) - radius(den)),
      abs(centre(num) - radius(num)) / (centre(den) + radius(den)),
      abs(centre(num) - radius(num)) / (centre(den) - radius(den))
) - abs(centre(num) / centre(den))
```

4.3.4.5 ADDITIONAL CONSTRAINTS

The denominator interval must not contain zero, i.e., `0` must not be in the range `[centre(den) - radius(den), centre(den) + radius(den)]`.

4.3.4.6 BEHAVIOUR

- Binary *Operators'* standard behaviour
- Numeric *Operators'* standard behaviour.
- If the denominator is 0, generates a run-time error.

4.3.4.7 EXAMPLES

- `24 / 6` results in `4`
- `7 / 2` results in `3.5`
- `-18 / 2` results in `-9`

4.3.5 Maximum (max)

4.3.5.1 SYNTAX

max(op1, op2 {, op} *)*

4.3.5.2 INPUT PARAMETERS

op1: rset | scal <num + interval>

op2: rset | scal <num + interval>

4.3.5.3 OUTPUT

rset | scal <num + interval>

4.3.5.4 SEMANTICS

Calculates the maximum value from a set of *Operands*.

For intervals:

- Centre is the maximum of the centres of all the *Operands*.
- Radius is the radius of the *Operand* with the maximum centre. If multiple *Operands* share the same maximum centre, the largest radius among them is taken.

4.3.5.5 ADDITIONAL CONSTRAINTS

None.

4.3.5.6 BEHAVIOUR

- For each pair of *Operands*, the standard behaviour for binary *Operators* applies.

4.3.5.7 EXAMPLES

- `max(1, 3, -5)` results in `3`
- `max(1, 3, null)` results in `null`

4.3.6 Minimum (min)

4.3.6.1 SYNTAX

min(op1, op2 {, op} *)**

4.3.6.2 INPUT PARAMETERS

op1: rset | scal <num + interval>

op2: rset | scal <num + interval>

4.3.6.3 OUTPUT

rset | scal <num + interval>

4.3.6.4 SEMANTICS

Calculates the minimum value from a set of *Operands*.

For intervals:

- Centre is the minimum of the centres of all the *Operands*.
- Radius is the radius of the *Operand* with the minimum centre. If multiple *Operands* share the same minimum centre, the largest radius among them is taken.

4.3.6.5 ADDITIONAL CONSTRAINTS

None.

4.3.6.6 BEHAVIOUR

- For each pair of *Operands*, the standard behaviour for binary *Operators* applies.

4.3.6.7 EXAMPLES

- `min(1, 3, -5)` results in `-5`
- `min(1, 3, null)` results in `null`

4.3.7 Power

4.3.7.1 SYNTAX

power(base, exponent **)**

4.3.7.2 INPUT PARAMETERS

base: rset | scal <num>

exponent: rset | scal <num>

4.3.7.3 OUTPUT

rset | scal <num>

4.3.7.4 SEMANTICS

Raises the power to the exponent.

4.3.7.5 ADDITIONAL CONSTRAINTS

Number Interval operands are not supported. The operator only accepts Number and Integer types.

4.3.7.6 BEHAVIOUR

- Binary *Operators'* standard behaviour
- Numeric *Operators'* standard behaviour.

4.3.7.7 EXAMPLES

- `power(5,2)` results in `25`
- `power(5,-1)` results in `0.2`
- `power(-5, 3)` results in `-125`

3.5 5 Comparison Operators

3.5.1 5.1 Comparison Operators' General Behaviour

Comparison *Operators* describe operations that compare the values of *Operands*.

This *Operator* group is based upon the General behavior for binary *Operators*.

5.1.1 Input Parameters

Operands that can be of *Recordset* or *Scalar* type. Must have the same *Data Type*. For all *Operators* that accept numeric *Operands*, intervals are allowed.

5.1.2 Result Type

Recordset or *Scalar* with type Boolean.

5.1.3 Constraints

The *Operands* for the comparison operations must be of the same type (considering implicit casting).

5.1.4 Behaviour

If any *Operand* is null, then the result is also null.

For comparison *Operators* implying an order (`>` , `>=` , `<` , `<=`), the following rules apply:

- Boolean values: `true` is considered greater than `false` .
- Strings: Alphabetic order is followed.

3.5.2 5.2 Equal (=)

5.2.1.1 Syntax

left = right

5.2.1.2 Input Parameters

left: rset | scal | scalar_set <*>

right: rset | scal | scalar_set <*>

5.2.1.3 Output

rset | scal <boo>

5.2.1.4 Semantics

Returns `true` if left is equal to right and `false` otherwise.

For intervals: `abs(centre(left) - centre(right)) <= radius(left) + radius(right)` .

When both *Operands* are set-valued (a set literal, a *Subcategory*, a `set-*` *Parameter*, or the result of a set *Operator*), `=` denotes set equality: it compares them as *Scalar Sets* and returns a single Boolean — see Set Equality.

5.2.1.5 Additional Constraints

- A *Scalar Set Operand* may only be compared with another *Scalar Set*: both `left` and `right` must be set-valued. Comparing a *Scalar Set* with a *Scalar* or *Recordset* is not allowed.

5.2.1.6 Behaviour

- Binary *Operators'* standard behaviour.
- Comparison *Operators'* standard behaviour.

5.2.1.7 Examples

- `1 = 2` results in `false`
- `3 = null` results in `null`
- `"4" = "4"` results in `true`

3.5.3 5.3 Not Equal (!=)

5.3.1.1 Syntax

left != right

5.3.1.2 Input Parameters

left: rset | scal | scalar_set <*>

right: rset | scal | scalar_set <*>

5.3.1.3 Output

rset | scal <boo>

5.3.1.4 Semantics

Returns `false` if left is equal to right and `true` otherwise.

For intervals: `abs(centre(left) – centre(right)) > radius(left) + radius(right)`.

When both *Operands* are set-valued (a set literal, a *Subcategory*, a `set-*` *Parameter*, or the result of a set *Operator*), `!=` denotes set inequality: it compares them as *Scalar Sets* and returns a single Boolean — see Set Equality.

5.3.1.5 Additional Constraints

- A *Scalar Set Operand* may only be compared with another *Scalar Set*: both `left` and `right` must be set-valued. Comparing a *Scalar Set* with a *Scalar* or *Recordset* is not allowed.

5.3.1.6 Behaviour

- Binary *Operators'* standard behaviour.
- Comparison *Operators'* standard behaviour.

5.3.1.7 Examples

- `1 != 2` results in `true`
- `3 != null` results in `null`

- `"4" != "4"` results in `false`

3.5.4 5.4 Greater than (>)

5.4.1.1 Syntax

```
left > right
```

5.4.1.2 Input Parameters

left: rset | scal <*>

right: rset | scal <*>

5.4.1.3 Output

rset | scal <boo>

5.4.1.4 Semantics

Returns `true` if left is greater than right and `false` otherwise.

For intervals: `centre(left) > centre(right) - (radius(left) + radius(right))`.

5.4.1.5 Additional Constraints

None.

5.4.1.6 Behaviour

- Binary *Operators'* standard behaviour.
- Comparison *Operators'* standard behaviour.

5.4.1.7 Examples

- `12 > 2` results in `true`
- `3 > null` results in `null`
- `true > false` results in `true`
- `"tez" > "test"` results in `true`

3.5.5 5.5 Greater Equal than (>=)

5.5.1.1 Syntax

```
left >= right
```

5.5.1.2 Input Parameters

left: rset | scal <*>

right: rset | scal <*>

5.5.1.3 Output

rset | scal <boo>

5.5.1.4 Semantics

Returns `true` if left is greater than or equal to right and `false` otherwise.

For intervals: `centre(left) >= centre(right) - (radius(left) + radius(right))`.

5.5.1.5 Additional Constraints

None.

5.5.1.6 Behaviour

- Binary *Operators'* standard behaviour.
- Comparison *Operators'* standard behaviour.

5.5.1.7 Examples

- `1 >= 2` results in `false`
- `3 >= null` results in `null`
- `"tez" >= "test"` results in `true`

3.5.6 5.6 Less than (<)

5.6.1.1 Syntax

left < right

5.6.1.2 Input Parameters

left: rset | scal <*>

right: rset | scal <*>

5.6.1.3 Output

rset | scal <boo>

5.6.1.4 Semantics

Returns `true` if left is smaller than right and `false` otherwise.

For intervals: `centre(left) – centre(right) < radius(left) + radius(right)`.

5.6.1.5 Additional Constraints

None.

5.6.1.6 Behaviour

- Binary *Operators'* standard behaviour.
- Comparison *Operators'* standard behaviour.

5.6.1.7 Examples

- `1 < 2` results in `true`
- `3 < null` results in `null`

- `"tea" < "test"` results in `true`

3.5.7 5.7 Less Equal than (<=)

5.7.1.1 Syntax

left <= right

5.7.1.2 Input Parameters

left: rset | scal <*>

right: rset | scal <*>

5.7.1.3 Output

rset | scal <boo>

5.7.1.4 Semantics

Returns `true` if left is smaller than or equal to right and `false` otherwise.

For intervals: `centre(left) - centre(right) <= radius(left) + radius(right)`.

5.7.1.5 Additional Constraints

None.

5.7.1.6 Behaviour

- Binary *Operators'* standard behaviour.
- Comparison *Operators'* standard behaviour.

5.7.1.7 Examples

- `3 <= 2` results in `false`
- `3 <= null` results in `null`
- `"tea" <= "test"` results in `true`

3.5.8 5.8 Element of (in)

5.8.1.1 Syntax

op **in** set

5.8.1.2 Input Parameters

op: rset | scal <*>

set: scalar_set <*> | subcategory

The `set` *Operand* may be any set-valued expression: a set literal (`{v1, v2, ...}`), a *Subcategory*, a *Parameter* of a `set-*` type, or the result of a Set *Operator* — `set_of`, `union`, `intersect`, `setdiff` or `symdiff` (see Set *Operators*).

5.8.1.3 Output

rset | scal <boo>

5.8.1.4 Semantics

Returns `true` if op belongs to the set and `false` otherwise.

`op in {v1, v2, ...}` is semantically equivalent to `op = v1 or op = v2 or ...`. Therefore, for intervals, this operator follows the interval semantics of the equality operator (`=`): the result is `true` if the interval overlaps (in the sense of `=`) with at least one element of the set.

Since set elements are typically points, and the point-to-interval promotion rule (§2.2.4) converts each point `v` to an interval with centre `v` and radius `0`, this reduces to:

```
abs(centre(op) - v) <= radius(op)
```

for each element `v` in the set. The overall result is `true` if this condition holds for any element.

5.8.1.5 Additional Constraints

op must be of the same *Data Type* as the values in the set (considering implicit casting)

5.8.1.6 Behaviour

- Binary *Operators'* standard behaviour.
- Comparison *Operators'* standard behaviour.

5.8.1.7 Examples

- `5 in {1,3, 5}` results in `true`
- `"abc" in {"def", "ghi"}` results in `false`
- `5 in union({1, 2}, {3, 5})` results in `true`
- `{tT1, r010, c010} in set_of({tT1, r010-020}[get CNT])` — tests membership against the set of distinct `CNT` values.

3.5.9 5.9 Match Characters (match)

5.9.1.1 Syntax

match(op, pattern **)**

5.9.1.2 Input Parameters

op: rset | scal <str>

pattern: scal <str>

5.9.1.3 Output

rset | scal <boo>

5.9.1.4 Semantics

Returns true if op matches the pattern, false otherwise.

5.9.1.5 Additional Constraints

pattern is a regex expression following the Python definition.

5.9.1.6 Behaviour

- Binary *Operators'* standard behaviour.
- Comparison *Operators'* standard behaviour.

5.9.1.7 Examples

- `match("test", "[0-9]+")` results in `false`
- `match("1234", "[0-9]+")` results in `true`
- `match("hello", "[a-z]+")` results in `true`

3.5.10 5.10 Is null (isnull)

5.10.1.1 Syntax

isnull(op **)**

5.10.1.2 Input Parameters

op: rset | scal <*>

5.10.1.3 Output

rset | scal <boo>

5.10.1.4 Semantics

Returns `true` if the value of op is `null`, `false` otherwise.

5.10.1.5 Additional Constraints

None.

5.10.1.6 Behaviour

- Unary *Operators'* standard behaviour.
- Comparison *Operators'* standard behaviour.

3.6 6 Logical Operators

Logical *Operators* describe operations involving two *Operands* with Boolean *Data Type*. To deal with null values, all operations implements Kleene logic for logical operations (also known as Three-valued logic).

3.6.1 6.1 Conjunction (and)

6.1.1.1 Syntax

left **and** right

6.1.1.2 Input Parameters

left: rset | scal <boo>

right: rset | scal <boo>

6.1.1.3 Output

rset | scal <boo>

6.1.1.4 Semantics

Returns `true` if both *Operands* are `true`, otherwise `false`.

6.1.1.5 Additional Constraints

None.

6.1.1.6 Behaviour

- Binary *Operators'* standard behaviour.

	false	**null**	**true**
false	false	false	false
null	false	null	null
true	false	null	true

3.6.2 6.2 Disjunction (or)

6.2.1.1 Syntax

left **or** right

6.2.1.2 Input Parameters

left: rset | scal <boo>

right: rset | scal <boo>

6.2.1.3 Output

rset | scal <boo>

6.2.1.4 Semantics

Returns `true` if any *Operand* is `true`, otherwise `false`.

6.2.1.5 Additional Constraints

None.

6.2.1.6 Behaviour

- Binary *Operators'* standard behaviour.

	false	**null**	**true**
false	false	null	true
null	null	null	true
true	true	true	true

3.6.3 6.3 Exclusive Disjunction (xor)

6.3.1.1 Syntax

left **xor** right

6.3.1.2 Input Parameters

left: rset | scal <boo>

right: rset | scal <boo>

6.3.1.3 Output

rset | scal <boo>

6.3.1.4 Semantics

Returns `true` if one *Operand* is `true` and the other is `false`, otherwise `false`.

6.3.1.5 Additional Constraints

None.

6.3.1.6 Behaviour

- Binary *Operators'* standard behaviour.

	false	**null**	**true**
false	false	null	true
null	null	null	null
true	true	null	false

3.6.4 6.4 Negation (not)

6.4.1.1 Syntax

not op

6.4.1.2 Input Parameters

op: rset | scal <boo>

6.4.1.3 Output

rset | scal <boo>

6.4.1.4 Semantics

Returns `true` if op is `false`, and `false` if op is `true`.

6.4.1.5 Additional Constraints

None.

6.4.1.6 Behaviour

- Unary *Operators'* standard behaviour.

Input	Result
false	true
null	null
true	false

3.7 7 Aggregate Operators

3.7.1 7.1 Aggregate Operators' General Behaviour

Aggregate *Operators* apply a summarising computation (sum, count, average, ...) to the *Fact* values of the *Operand Recordset*. They support two mutually exclusive invocation styles:

- A **reducing invocation** — the `group by` clause, or no clause at all — which collapses each group of *Records* into a single result *Record*: N *Records* per group become one. This is the classic aggregation and is the subject of §7.1.1–§7.1.7.
- An **analytical invocation** — the `over` clause — which computes the same aggregate over a *window* of related *Records* but returns **one output *Record per input Record***, so the *Recordset* keeps its cardinality. Analytical invocations are the unifying mechanism behind moving averages, running totals, shares of a parent total and ranks. They are described in §7.1.8.

The `group by` and `over` clauses are mutually exclusive: an invocation uses either `group by` (or nothing) or `over`, never both.

7.1.1 Syntax

aggregateOperator (op {**group by** groupingId {, groupingId}*} **)**

aggregateOperator (op **over (** analyticClause **))**

7.1.2 Input Parameters

op: rset <*>

groupingId: scal <prop>

7.1.3 Output

rset | scal <*>

7.1.4 Semantics

In the **reducing invocation**, aggregate *Operators* perform operations on the measures of the *Operand Recordset*, calculating the required aggregated values for groups of *Records*. The groups of *Records* to be aggregated are specified through the grouping clause. If no grouping clause is used, the operation shall be calculated on all the *Records*, resulting in a *Scalar*.

In the **analytical invocation**, the same computation is applied without reducing: for each input *Record* the *Operator* is evaluated over the *window* of *Records* defined by the `over` clause, and the result replaces that *Record*'s *Fact* value. The output has the same *Key Components* and the same number of *Records* as the input (see §7.1.8).

7.1.5 Additional Constraints

The allowed *Data Types* depend on the specific *Operator* according to the following table:

Operator	***Operand* Type**	**Result Type**
Sum	Number	Number
Count	Any	Integer
Min	Any	Any
Max	Any	Any
Average	Number	Number
Median	Number	Number
Rank	Any	Integer

For **Average** and **Median**, when all input values are *Integer*, the result is implicitly promoted to *Number* — the result type is always *Number*, even on integer-only inputs (e.g., `avg(1, 2)` returns `1.5`; `median(1, 2, 3, 4)` returns `2.5`). For **Sum**, an *Integer*-only input yields an *Integer* result. **Count** always returns *Integer* by definition. **Min** and **Max** return one of the input values, so an *Integer*-only input yields an *Integer* result. **Rank** (§7.8) exists only in the analytical invocation and always returns *Integer*.

The *Components* named in the grouping clause, and in the `partition by` and `order by` sub-clauses of the `over` clause, shall be present in the *Operand*.

7.1.6 Behaviour

In the **reducing invocation**, aggregate *Operations* generate a *Recordset* or a *Scalar*, depending on the grouping clause.

If the grouping clause exists, the structure of the resulting *Recordset* has as *Key Components* the *Components* in the group by

The result may be:

- A *Recordset*, for which the resulting structure contains as *Key* dimensions the *Components* included in the group by clause.
- A *Scalar*, if the grouping clause is omitted.

In the **analytical invocation** (`over` clause), the result is always a *Recordset* with the same *Key Components* and the same number of *Records* as the *Operand* (see §7.1.8).

Null values are excluded from the calculations. If all input values (in a group, or in a *Record*'s window) are null, the result is null. The only exception is **Count**, which returns 0 in that case (see below).

7.1.6.1 EMPTY OPERANDS

An aggregation has no values to work with when its *Operand* contains no *Records*, or when every one of its input values has been excluded as null. In that case:

- **Count** returns 0. Counting is a question about the collection rather than about the values, and the empty collection has a well-defined answer. **Count** therefore never returns null: its result is always a non-negative Integer.
- **Sum**, **Min**, **Max**, **Average** and **Median** return null. These *Operators* are defined over the input values, and with no values there is no result to report.

In particular, **Sum** over an empty *Operand* is null and not 0. Although 0 is the identity element of addition, returning it would state that a total of zero was reported, when in fact nothing was reported at all. This also keeps the empty case consistent with the all-null case: since null values are excluded before the calculation, an all-null *Operand* is an empty *Operand*, and both yield null.

When a grouping clause is present, the exclusion of null values takes place within each group and not before the grouping. The groups are derived from the *Records* present in the *Operand*, and a *Record* whose value is null still determines a group. A group all of whose values are null therefore does exist, and the rules above apply to it: **Count** returns 0 for that group and the other *Operators* return null for it.

Retaining the group preserves the information that the *Records* were there, so that the nvl *Operator* or any subsequent *Operation* can act on the result.

What cannot happen is a group emerging for a *Key* combination that is not present in the *Operand*. The values of the *Key Components* of a group have to be taken from a *Record*, and *Key Components* cannot be empty, so no *Record* can be produced for a combination that was never reported: DPM-XL does not build the Cartesian product of all the potential combinations of *Key* values (see Implicit Nulls Generation, where implicit nulls are generated only for the *Key* combinations having at least one value reported). Accordingly, an *Operand* with no *Records* at all yields an empty *Recordset*, and not a *Recordset* holding a *Record* with the value 0 or null.

Whether a null is explicit or implicit makes no difference to any of the above, in line with Implicit and Explicit Null Values. What determines the result is whether a *Record* exists in the *Operand*, not the reason why its value is null.

7.1.7 Examples

Supposing the following *Recordset* with name rs1:

r	c	CNT	f
010	010	PT	100
010	020	PT	200
020	010	PT	300
020	020	PT	400
010	010	DE	500
010	020	DE	600
020	010	DE	700
020	020	DE	800

`sum(rs1 group by r)` results in:

r	f
010	1400
020	2200

`sum(rs1 group by r, CNT)` results in:

r	CNT	f
010	PT	300
020	PT	700
010	DE	1100
020	DE	1500

`count(rs1)` results in: `8` (*Scalar*)

Supposing now a *Recordset* with name rs2 whose selection yields no *Records* at all:

- `count(rs2)` results in: `0` (*Scalar*)
- `sum(rs2)` results in: null (*Scalar*)
- `sum(rs2 group by r)` results in an empty *Recordset* with *Key Component* r and no *Records*.
- `count(rs2 group by r)` results in the same empty *Recordset*, and not in a *Record* holding the value 0. Under a grouping clause there is no *Record* to provide a value for r, so the 0 has nowhere to appear. **Count** and **Sum** thus give the same result here, and only differ in the *Scalar* case above.

Supposing instead a *Recordset* with name rs3 having the same *Records* as rs1, but with all the values of f null:

- `count(rs3)` results in: `0` (*Scalar*)
- `sum(rs3)` results in: null (*Scalar*)
- `sum(rs3 group by r)` results in:

r	f
010	null
020	null

- `count(rs3 group by r)` results in:

r	f
010	0
020	0

The groups for r are still formed, because the *Records* are present in rs3 and provide the values of r. This is the only way a **Count** of 0 can appear in a grouped result.

7.1.8 The Analytical Invocation (over clause)

The `over` clause turns an aggregate *Operator* into an **analytical invocation** (a *window function* in SQL terms). Instead of collapsing groups, the *Operator* computes its result over a *window* of *Records* defined relative to each *Record*, and emits one output *Record* per input *Record*.

7.1.8.1 SYNTAX

op **over (** {**partition by** partitionId {, partitionId}*} {**order by** orderId {**asc**|**desc**} {, orderId {**asc**|**desc**}}*} {windowClause} **)**

where

windowClause: **(data points** | **range) between** windowBoundary **and** windowBoundary

windowBoundary: **unbounded preceding** | **unbounded following** | **current data point** | int {time_period_code} **preceding** | int {time_period_code} **following**

partitionId, orderId: scal <prop>

int: A positive *Integer* literal.

time_period_code: One of the time-period codes `A`, `S`, `Q`, `M`, `W`, `D`.

All three inner sub-clauses are optional, subject to the constraints below. `asc` (ascending) is the default direction and may be omitted.

7.1.8.2 PARTITIONING (PARTITION BY)

`partition by` splits the *Operand Recordset* into groups (*partitions*) that share the same values for the listed *Components*, analogously to the *Key Components* of a reducing `group by`. Every *Record*'s window is confined to its own partition. When `partition by` is omitted, the whole *Recordset* is a single partition.

Unlike `group by`, `partition by` does **not** reduce: the partition *Components* are not the *Key* of the result — the result keeps the *Key Components* of the *Operand*.

7.1.8.3 ORDERING (ORDER BY)

Records are an unordered collection (see §2.2.3), so any order-dependent window needs an explicit ordering. `order by` establishes, within each partition, the order in which *Records* are considered. Each ordering *Component* may be followed by `asc` (ascending, the default) or `desc` (descending).

`order by` is:

- **required** for order-dependent *Operators* (`rank`) and whenever a `data points` or `range` window clause is used;
- **optional** otherwise — when omitted the whole partition forms one unordered window and every *Record* in the partition receives the same aggregate.

These requirements are stated as constraints in §7.1.8.6.

7.1.8.4 WINDOW CLAUSE (DATA POINTS / RANGE)

The window clause narrows the window to a band of *Records* around the current one, expressed as two bounds. A window clause requires an `order by`.

Each bound is a *windowBoundary*, written in the direction it travels from the current *Record*:

- **`current data point`** — the current *Record* itself.
- **`n preceding`** / **`n following`** — `n` units before or after the current *Record*, where `n` is a positive *Integer*. The zero offset is written `current data point`.
- **`unbounded preceding`** / **`unbounded following`** — the start or the end of the partition.

The unit of `n` depends on the keyword introducing the clause:

- **`data points`** counts **positional offsets**: `n` is a number of *Records* in the `order by` order, so `1 preceding` is the immediately preceding *Record*. Use `data points` when position — not the value of the order key — is what matters.
- **`range`** counts **value offsets** on a single `order by` *Component*: the window contains every *Record* whose order value lies within `n` of the current *Record*'s order value. Use `range` when the ordering has gaps, because the window is defined by the order value rather than by record position.

Where the `order by` *Component* is of *Date* type, `n` is followed by a time-period code giving the unit of the offset, so that `11 M preceding` is the *Record* eleven months before the current one. The offset is computed exactly as in time_shift: `n` units of the given period are subtracted from (`preceding`) or added to (`following`) the current *Record*'s order value, and the bound is that resulting date. Where the `order by` *Component* is numeric, `n` is a plain number of units of that *Component* and no period code is written.

`preceding` and `following` are always relative to the **`order by` order**, not to the natural order of the values: a `preceding` bound lies toward the start of the ordering and a `following` bound toward its end. Under a `desc` ordering the start of the ordering holds the *greatest* order values, so a `preceding` bound then has a greater order value than the current *Record* — a later date, where the order key is a *Date*. This is the same rule `data points` follows, where `1 preceding` is the previous *Record* of the ordering whichever direction it runs.

A **trailing** window therefore requires an **ascending** ordering. `order by refDate range between 11 M preceding and current data point` covers the twelve months ending at the current *Record*, whereas the same window clause under `order by refDate desc` reaches forward instead, over the twelve months beginning at it.

When the window clause is omitted, the window is the **whole partition** (still honouring `order by` where it matters, as for `rank`). A time-period code may be written only on a `range` bound whose `order by` *Component* is of *Date* type; it is not allowed on a `data points` bound, whose unit is always one *Record*. The bounds are further constrained as set out in §7.1.8.6.

7.1.8.5 STRUCTURE AND RECORDS

The result is a *Recordset* with the same *Key Components* and the same number of *Records* as the *Operand*. Each *Record*'s *Fact* value is replaced by the aggregate computed over that *Record*'s window; *Key Components* are unchanged and *Attribute Components* are not propagated. Null values inside a window are excluded from the computation, consistent with the reducing invocation; if a window contains only nulls, the result for that *Record* follows the empty-*Operand* rules of §7.1.6.1 — 0 for **Count**, null for the other *Operators*. The result *Data Type* of each *Operator* is the one given in §7.1.5.

7.1.8.6 ADDITIONAL CONSTRAINTS

The *Components* named in `partition by` and in `order by` shall be present in the *Operand*, as required by §7.1.5.

The offset `n`. The `n` of `n preceding` or `n following` shall be at least 1. The zero offset — the current *Record* itself — is written `current data point`, which is its only form: `0 preceding`, `0 following` and `0 M preceding` are not allowed.

When an `order by` is required. `order by` is required in two cases: when a window clause is used, and always with the `rank` operator. Both narrow or read a sequence, and *Records* are an unordered collection (§2.2.3), so without an explicit ordering neither has a determined result. This holds even for `between unbounded preceding and unbounded following`, whose meaning does not in fact depend on the ordering: the whole partition is expressed by omitting the window clause, which is the form to use.

`rank` takes no window clause. `rank` ranks by the `order by` value, and a window clause is defined along that same `order by`, positionally for `data points` and by value for `range`. A frame can therefore never give `rank` a different comparison group to measure against. To compare a *Record* against a window of its neighbours, use an aggregate over that window and compare it with the *Record*'s own value, as in `max_aggr(rs1 over (partition by qInstrument order by refDate range between 11 M preceding and current data point))`.

The `order by` of a `range` clause. Where the window clause is a `range`, the `order by` shall name exactly one *Component*, and that *Component* shall be of *Date*, *Number* or *Integer* type. A `range` bound measures an offset against the order *value*, and a lexicographic ordering over several *Components* offers no single value to offset from, while a *String* or *Boolean Component* offers no notion of lying within `n` of a value. Neither restriction applies to `data points`, which needs only a sequence, nor to `rank`; both accept several ordering *Components*.

Ordering of the window bounds. The bounds of a window clause are ordered by the position they denote relative to the current *Record*:

`unbounded preceding` < `n preceding` (decreasing in `n`) < `current data point` < `n following` (increasing in `n`) < `unbounded following`

- The first bound shall not be after the second in this order. Where the two bounds denote the same position, the window is that single position — as in `between current data point and current data point`.
- `unbounded following` shall not be the first bound, and `unbounded preceding` shall not be the second. Such a window is either empty or a single *Record* of the partition repeated for every *Record*, which is never the intent: the whole partition is written `between unbounded preceding and unbounded following`, and a single aggregate over the partition is a reducing invocation (§7.1.6).

Commensurable period codes. Where both bounds of a `range` clause carry a time-period code, the two codes shall be commensurable: either both drawn from `A`, `S`, `Q`, `M`, or both from `W`, `D`. Within each of these groups every code is a whole number of the same base unit (twelve, six, three and one *months*; seven and one *days*), so the bounds can always be ordered. Across the groups they cannot: `1 M preceding` is earlier than `30 D preceding` in some months and later in others, so a frame such as `range between 3 M preceding and 40 D preceding` has no determinate meaning and is not allowed.

7.1.8.7 EXAMPLES

Considering the following *Recordset* with name rs1, holding a monthly balance per instrument:

qInstrument	refDate	f
A	2022-01-01	100
A	2022-02-01	200
A	2022-03-01	300
B	2022-01-01	10
B	2022-02-01	20
B	2022-03-01	30

Trailing 2-month moving average — `avg(rs1 over (partition by qInstrument order by refDate data points between 1 preceding and current data point))` averages each *Record* with the immediately preceding one in its partition:

qInstrument	refDate	f
A	2022-01-01	100
A	2022-02-01	150
A	2022-03-01	250
B	2022-01-01	10
B	2022-02-01	15
B	2022-03-01	25

(The first *Record* of each partition has no predecessor, so its window is just itself.)

Running total — `sum(rs1 over (partition by qInstrument order by refDate data points between unbounded preceding and current data point))` accumulates from the start of each partition up to the current *Record*:

qInstrument	refDate	f
A	2022-01-01	100
A	2022-02-01	300
A	2022-03-01	600
B	2022-01-01	10
B	2022-02-01	30
B	2022-03-01	60

12-month moving average, gap-tolerant — `avg(rs1 over (partition by qInstrument order by refDate range between 11 M preceding and current data point))` averages every *Record* whose `refDate` falls within the 12 calendar months ending at the current *Record*. Because the bound is value-based, a missing month simply contributes no *Record* to the window; the positional form `data points between 11 preceding and current data point` would instead always reach 11 *Records* back, ignoring calendar gaps.

3.7.2 7.2 Sum (sum)

7.2.1 Syntax

sum(op {**group by** groupingId {, groupingId}* | **over (** analyticClause **)** } **)**

7.2.2 Input Parameters

op: rset <num + interval>

groupingId: scal <prop>

7.2.3 Output

rset | scal <num + interval>

7.2.4 Semantics

Returns the sum of the input values. Follows the general semantics of aggregate *Operators*.

For intervals:

- The centre is calculated as the sum of the all the centers of the *Operands*.
- The radius is calculated as the sum of all the radiuses of the *Operands*.

7.2.5 Additional Constraints

None.

7.2.6 Behaviour

Aggregate *Operators'* general behaviour.

3.7.3 7.3 Count (count)

7.3.1 Syntax

count(op {**group by** groupingId {, groupingId}* | **over (** analyticClause **)** } **)**

7.3.2 Input Parameters

op: rset <> | *scalar_set* <>

groupingId: scal <prop>

7.3.3 Output

rset | scal <int>

7.3.4 Semantics

Returns the number of non-null values in the *Recordset* or in each group of *Records*. Follows the general semantics of aggregate *Operators*: null values are excluded from the calculation, so *Records* whose value is null do not contribute to the count.

Unlike the other aggregate *Operators*, **count** never returns null. Its result is always a non-negative Integer: an *Operand* with no *Records*, and an *Operand* whose values are all null, both yield 0 (see Empty *Operands*).

When op is a `scalar_set`, `count` returns the **cardinality** of the set — the number of distinct elements it contains. As sets are duplicate-free and never contain `null` (see Set *Operators*' general behaviour), the count is taken over the distinct, non-null values. The `group by` clause is inapplicable when op is a `scalar_set`.

7.3.5 Additional Constraints

None.

7.3.6 Behaviour

Aggregate *Operators*' general behaviour.

Note: Aggregate *Operators* generally ignore null values. This behavior can be overridden by using the nvl *Operator*. Applied to **count**, `count(nvl(op, 0))` counts every *Record* of op, including those whose value is null.

7.3.7 Examples

- `count({1, 2, 3, 3, 2})` results in `3` — the set literal de-duplicates first, leaving three distinct elements.
- `count(set_of({tT1, r010-020}[get CNT]))` results in `4` — the number of distinct `CNT` values (see set_of).
- `count(union({1, 2}, {2, 3}))` results in `3`.
- `count({})` results in `0` — the cardinality of the empty set.

3.7.4 7.4 Minimum Value (min_aggr)

7.4.1 Syntax

min_aggr(op {**group by** groupingId {, groupingId}* | **over (** analyticClause **)** } **)**

7.4.2 Input Parameters

op: rset <*>

groupingId: scal <prop>

7.4.3 Output

rset | scal <*>

7.4.4 Semantics

Returns the minimum value of the input values. Follows the general semantics of aggregate *Operators*.

For intervals:

- Centre is the minimum of the centres of all the *Operands*.
- Radius is the radius of the *Operand* with the minimum centre. If multiple *Operands* share the same minimum centre, the largest radius among them is taken.

7.4.5 Additional Constraints

None.

7.4.6 Behaviour

Aggregate *Operators*' general behaviour.

3.7.5 7.5 Maximum Value (max_aggr)

7.5.1 Syntax

max_aggr(op {**group by** groupingId {, groupingId}* | **over (** analyticClause **)** } **)**

7.5.2 Input Parameters

op: rset <*>

groupingId: scal <prop>

7.5.3 Output

rset | scal <*>

7.5.4 Semantics

Returns the maximum value of the input values. Follows the general semantics of aggregate *Operators*.

For intervals:

- Centre is the maximum of the centres of all the *Operands*.
- Radius is the radius of the *Operand* with the maximum centre. If multiple *Operands* share the same maximum centre, the largest radius among them is taken.

7.5.5 Additional Constraints

None.

7.5.6 Behaviour

Aggregate *Operators'* general behaviour.

3.7.6 7.6 Average (avg)

7.6.1 Syntax

avg(op {**group by** groupingId {, groupingId}* | **over (** analyticClause **)** } **)**

7.6.2 Input Parameters

op: rset <num + interval>

groupingId: scal <prop>

7.6.3 Output

rset | scal <num + interval>

7.6.4 Semantics

Returns the average of the input values. Follows the general semantics of aggregate *Operators*.

For intervals:

- The centre is calculated as the average of the centres of all the *Operands*.
- The radius is calculated as the average of the radii of all the *Operands*.

7.6.5 Additional Constraints

None.

7.6.6 Behaviour

Aggregate *Operators'* general behaviour.

3.7.7 7.7 Median Value (median)

7.7.1 Syntax

median(op {**group by** groupingId {, groupingId}* | **over (** analyticClause **) })**

7.7.2 Input Parameters

op: rset <num + interval>

groupingId: scal <prop>

7.7.3 Output

rset | scal <num + interval>

7.7.4 Semantics

Returns the median of the input values. Follows the general semantics of aggregate *Operators*.

For intervals, the centre is used as the ordering criterion. The result is defined as follows:

- **Centre**: the median of the centres of all the *Operands* (using standard median arithmetic on the centre values).
- **Radius**:
- If the median position corresponds to a single *Operand*, its radius is used.
- If multiple *Operands* share the same median centre (tie at the median position), the largest radius among them is taken.
- If the median is the arithmetic average of two *Operands* with distinct centres (even number of *Operands*), the radius is the arithmetic average of their radii.

7.7.5 Additional Constraints

None.

7.7.6 Behaviour

Aggregate *Operators'* general behaviour.

3.7.8 7.8 Rank (rank)

`rank` is an **analytical-only** *Operator*: it has no reducing counterpart and is defined solely through the `over` clause (§7.1.8).

7.8.1 Syntax

rank(op **over (** analyticClause **))**

7.8.2 Input Parameters

op: rset <*>

partitionId, orderId: scal <prop>

7.8.3 Output

rset <int>

7.8.4 Semantics

Returns, for each *Record*, the 1-based rank of its `order by` value within its partition, following the `order by` direction (`asc`/`desc`). *Records* that tie on the order value receive the same rank, and the next distinct value's rank is increased by the number of tied *Records* (standard competition ranking: 1, 2, 2, 4, …).

`rank` is order-dependent, so the `over` clause **must** contain an `order by`, and it takes no window clause: `rank` always considers the whole partition. The reason is that the `order by` is both the ranking key and the axis along which a window clause would be defined, so a frame could only report the *Record*'s position inside the frame rather than its standing within a comparison group (see §7.1.8.6).

Records whose order value is null sort last and share the final rank group.

7.8.5 Additional Constraints

- The `over` clause must contain an `order by`.
- `partition by` is optional; without it the whole *Recordset* is ranked as a single partition.
- A window clause (`data points` / `range`) is not allowed with `rank`.

7.8.6 Behaviour

Aggregate *Operators'* analytical invocation behaviour: one output *Record* per input *Record*, with the *Fact Component* replaced by the Integer rank. *Key Components* are unchanged and *Attribute Components* are not propagated.

7.8.7 Examples

Considering the following *Recordset* with name rs1, holding total assets per entity within each country:

qCountry	id	f
PT	X	300
PT	Y	100
PT	Z	300
DE	P	500
DE	Q	200

Rank within country by total assets (descending) — `rank(rs1 over (partition by qCountry order by f desc))` returns:

qCountry	id	f
PT	X	1
PT	Z	1
PT	Y	3
DE	P	1
DE	Q	2

(Within PT, the two *Records* valued `300` tie at rank `1`, so the `100` *Record* takes rank `3`. The *Fact* value is replaced by the rank.)

3.8 8 Conditional Operators

3.8.1 8.1 If-then-else

8.1.1 Syntax

if conditionExpression **then** thenExpression {**else** elseExpression} **endif**

8.1.2 Input Parameters

conditionExpression: rset | scal <boo>

thenExpression: rset | scal <*>

elseExpression: rset | scal <*>

8.1.3 Output

rset | scal <*>

8.1.4 Semantics

The result combines values selected from the thenExpression where the conditionExpression evaluates to `true`, with values selected from the elseExpression where the conditionExpression evaluates to `false` or `null`. *Records* are matched between the conditionExpression and each branch by inner join. When the elseExpression is omitted, *Records* where the conditionExpression is not `true` are excluded from the result. The formal evaluation is given in §8.1.6.

8.1.5 Additional Constraints

The additional constraints ensure that the structure of the output is known at compile time and that no *Record* in the result can have a `null` value in any *Key Component*.

To the effects of these additional constraints, *Scalars* are treated as *Recordsets* with zero *Key Components*.

- **Data Type compatibility**: The thenExpression and elseExpression (when provided) must be of the same *Data Type*, or of *Data Types* that admit implicit casting per the casting rules in §2.3.2. When implicit casting applies, the resulting *Data Type* is that common *Data Type*.
- **Structure compatibility with the condition**: The *Key Components* of the thenExpression must be compatible with those of the conditionExpression — that is, one set must be equal to, or a subset of, the other, following the same rule as for binary *Operators* on two *Recordsets*. If the elseExpression is provided, the same restriction applies to the elseExpression together with the conditionExpression.
- **Matched join structure**: When the elseExpression is provided, the *Key Components* of the thenExpression and the elseExpression must satisfy one of: a) they are the same; or b) both are equal to, or a subset of, those of the conditionExpression.
- **Null literal exception**: When thenExpression or elseExpression is `null` (explicit or, in the case of elseExpression, implicit because not used), it is possible for the other expression to have a set of key components that is a superset of the conditionExpression. This is allowed because null records are not added to the output, and therefore no records with null value for *Key components* may arise.

8.1.6 Behaviour

Throughout this section, `join(x, y)` denotes the inner join of §2.2.3.3; its result is a *Recordset* whose *Key Components* are the superset of those of x and y (which, under the *Structure compatibility with the condition* constraint, equals the *Key Components* of whichever operand is the superset). This makes the *Matched join structure* constraint of §8.1.5 equivalent to requiring that `join(conditionExpression, thenExpression)` and `join(conditionExpression, elseExpression)` have the same

structure: case (a) gives both joins the *Key Components* `conditionExpression ∪ thenExpression = conditionExpression ∪ elseExpression`; case (b) gives both joins the *Key Components* of the conditionExpression.

- **Structure**:
- If all operands are *Scalars* (zero *Key Components*), the result is a *Scalar*; otherwise the result is a *Recordset*.
- The structure of the output shall contain the largest set of *Key Components* of the input structures.
- **Evaluation**:
- The expression is evaluated as the disjoint union of two sets of *Records*:
- The *Records* of `join(conditionExpression, thenExpression)` for which the conditionExpression evaluates to `true`, with the value of the *Fact Component* taken from the thenExpression.
- When the elseExpression is provided: the *Records* of `join(conditionExpression, elseExpression)` for which the conditionExpression evaluates to `false` or `null`, with the value taken from the elseExpression.
- The two sets are disjoint by construction (they select complementary *Records* of the conditionExpression) and share the same structure by the *Matched join structure* constraint.
- **Omitted else**: *Records* where the conditionExpression evaluates to `false` or `null` are not included in the result.

The following table summarises the allowed combinations, the additional constraint that applies beyond pairwise structure compatibility, and the resulting structure:

To the effects of this table:

- An explicit `else null` is treated as a missing else.
- When thenExpression or elseExpression is the literal `null`, the *Null literal exception* of §8.1.5 may further relax the constraints in the corresponding row, since no *Records* are contributed by a null branch.

Condition	Then	Else	Additional constraint	Result structure
Scalar	*Scalar*	—	—	*Scalar*
Scalar	*Scalar*	*Scalar*	—	*Scalar*
Scalar	*Recordset*	—	—	*Recordset* (keys of then)
Scalar	*Recordset*	*Recordset*	then and else must have the same *Key Components*	*Recordset* (common keys of then and else)
Scalar	*Scalar*	*Recordset*	Forbidden — join structures differ	—
Scalar	*Recordset*	*Scalar*	Forbidden — join structures differ	—
Recordset	*Scalar*	—	—	*Recordset* (keys of condition)
Recordset	*Scalar*	*Scalar*	—	*Recordset* (keys of condition)
Recordset	*Scalar*	*Recordset*	*Key Components* of else $\subseteq$ those of condition	*Recordset* (keys of condition)
Recordset	*Recordset*	—	*Key Components* of then and condition: equal or one a subset of the other	*Recordset* (superset of keys)
Recordset	*Recordset*	*Scalar*	*Key Components* of then $\subseteq$ those of condition	*Recordset* (keys of condition)
Recordset	*Recordset*	*Recordset*	`join(condition, then)` and `join(condition, else)` must share the same structure	*Recordset* (common join structure)

Relation to other operators: Under this definition, `if-then-else` subsumes several other operators as equivalent idioms:

- `if A then B` (no else), when the *Key Components* of A are a subset of those of B, is equivalent to `filter(B, A)`.
- `if isnull(A) then B else A` is equivalent to `nvl(A, B)` when A and B share the same data structure (the case in which `nvl` is applicable).
- `if A = x then y else A` is a general pattern to replace a value `x` with `y`.

8.1.7 Examples

EXAMPLE 1: SCALAR CONDITION, SCALAR BRANCHES

```
if 10 > 5 then 100 else 200 endif
```

Result: `100` (scalar). The condition is true, so the then branch is returned.

EXAMPLE 2: SCALAR CONDITION, RECORDSET BRANCHES

Supposing `{tT1, r010-020}` yields:

r	f
010	100
020	200

And `{tT2, r010-020}` yields:

r	f
010	900
020	800

```
if 10 > 5 then {tT1, r010-020} else {tT2, r010-020} endif
```

Result:

r	f
010	100
020	200

The condition is true, so the entire then recordset is returned.

EXAMPLE 3: RECORDSET CONDITION, SCALAR BRANCHES

Supposing `{tT1, r010-020} > 150` yields:

r	f
010	false
020	true

```
if {tT1, r010-020} > 150 then 1 else 0 endif
```

Result:

r	f
010	0
020	1

The condition is evaluated per record. The scalar branches are applied to each record based on its condition value.

EXAMPLE 4: RECORDSET CONDITION, RECORDSET BRANCHES WITH SAME KEYS

Supposing `{tT1, r010-020}` yields:

r	f
010	100
020	200

And `{tT2, r010-020}` yields:

r	f
010	900
020	800

```
if {tT1, r010-020} > 150 then {tT1, r010-020} else {tT2, r010-020} endif
```

Result:

r	f
010	900
020	200

Record 010 takes the else value (100 is not > 150), record 020 takes the then value (200 > 150).

EXAMPLE 5: FORBIDDEN — SCALAR CONDITION WITH MIXED BRANCHES

The following expression is **not allowed**:

```
if {tT1, r010} > 150 then {tT2, r010-020} else 0 endif
```

The condition `{tT1, r010} > 150` is a *Scalar* Boolean. The then branch is a *Recordset* with *Key Components* `{r}` and the else branch is a *Scalar* (zero *Key Components*). Applying the *Matched join structure* constraint: `join(condition, then)` has keys `{r}` while `join(condition, else)` has no keys. Since these structures differ, the expression is rejected. To fix this, either omit the else branch (records where the condition is `false` or `null` will not appear in the result) or replace the else branch with a *Recordset* sharing the same *Key Components* as the then branch.

EXAMPLE 6: RECORDSET CONDITION WITH MORE KEYS THAN THE BRANCHES

Supposing `{tT1, r010-020}` is an open table with a *Key Component* CNT, and the condition `{tT1, r010-020} > 0` yields:

r	CNT	f
010	ES	true
010	PT	false
020	ES	true
020	PT	true

And the then branch `{tT2, r010-020}` has only the r key:

r	f
010	100
020	200

```
if {tT1, r010-020} > 0 then {tT2, r010-020} else 0 endif
```

The condition has keys {r, CNT} and the then branch has keys {r}. The result has the structure of the superset (the condition), and the then/else values are matched via inner join on the common key (r):

r	CNT	f
010	ES	100
010	PT	0
020	ES	200
020	PT	200

EXAMPLE 7: OMITTED ELSE

Supposing `{tT1, r010-020}` yields:

r	f
010	100
020	200

```
if {tT1, r010-020} > 150 then {tT1, r010-020} endif
```

Result:

r	f
020	200

Record `010` is excluded because its condition value is `false` (100 is not > 150). Record `020` is included with its then value.

EXAMPLE 8: FORBIDDEN — BRANCHES WITH INCOMPATIBLE KEY STRUCTURES

Supposing `{tT1, r010-020}` has *Key Component* `r` and yields:

r	f
010	200
020	100

And `{tT3, r010-020}` has *Key Components* `r` and `CNT` and yields:

r	CNT	f
020	ES	500
020	PT	600

The following expression is **not allowed**:

```
if {tT1, r010-020} > 150 then 0 else {tT3, r020} endif
```

`join(condition, then)` has keys `{r}` while `join(condition, else)` has keys `{r, CNT}`. Since the structures differ, the *Matched join structure* constraint rejects the expression at compile time. This prevents results where records of the condition would silently disappear because one branch cannot supply values for the key tuples required by the other (while the same records would be included if the elseExpression was omitted). To fix this, ensure both branches produce results with the same *Key Components* after joining with the condition.

EXAMPLE 9: NULL LITERAL EXCEPTION — THEN BRANCH WITH MORE KEYS THAN THE CONDITION

Supposing the condition `{tT1, r010-020} > 0` yields a *Recordset* with key `{r}`:

r	f
010	true
020	false

And the then branch `{tT2, r010-020}` is an open table with keys `{r, CNT}`:

r	CNT	f
010	ES	100
010	PT	200
020	ES	300
020	PT	400

```
if {tT1, r010-020} > 0 then {tT2, r010-020} endif
```

With no `else` clause, the elseExpression is implicitly the literal `null`. The *Matched join structure* constraint would normally reject this — `join(condition, then)` has *Key Components* `{r, CNT}` while `join(condition, null)` has `{r}` — but the *Null literal exception* of §8.1.5 admits the case because the implicit-null branch contributes no *Records*, so no *Record* with a `null` value in a *Key Component* can arise.

Result:

r	CNT	f
010	ES	100
010	PT	200

The result takes the structure of the then branch, `{r, CNT}`. *Records* `(010, ES)` and `(010, PT)` are included (the condition is `true` for r = 010); *Records* with r = 020 are excluded because the condition is `false` and the implicit-null else branch contributes nothing.

3.8.2 8.2 Null Substitute (nvl)

8.2.1 Syntax

nvl(op1, op2 **)**

8.2.2 Input Parameters

op1: rset | scal <*>

op2: rset | scal <*>

8.2.3 Output

rset | scal <*>

8.2.4 Semantics

Returns op2 when op1 is `null`, otherwise op1.

8.2.5 Additional Constraints

op1 and op2 need to be of the same type (*Scalar* or *Recordset*) and same *Data Type*.

If op1 and op2 are of the *Recordset* type, they need to have the same data structure.

8.2.6 Behaviour

Binary *Operators'* standard behaviour.

8.2.7 Examples

Scalar example

`nvl({tT1, r010, c010, s01}, 0)` returns `0` when the selected cell is null, otherwise returns the cell value.

Recordset example

Considering the following *Recordset* from `{tT1, r010}`:

CNT	f
ES	100
PT	null
DE	null
IT	400

And the substitution *Recordset* (same structure, all zeros):

CNT	f
ES	0
PT	0
DE	0
IT	0

`nvl({tT1, r010}, {tT1_default, r010})` returns:

CNT	f
ES	100
PT	0
DE	0
IT	400

Records where op1 is not null are kept as-is; *Records* where op1 is null take the value from op2.

3.8.3 8.3 Filter

8.3.1 Syntax

filter(filteredOp, filteringOp **)**

8.3.2 Input Parameters

filteredOp: rset <*>

filteringOp: rset <boo>

8.3.3 Output

rset <*>

8.3.4 Semantics

Returns the *Records* of the filteredOp *Recordset* for which their matching *Record* in filteringOp evaluates to `true`. *Records* whose matching value in filteringOp is `false` or `null` are excluded (per the null-as-false rule for filtering in §4.3).

8.3.5 Additional Constraints

filteringOp must have the same *Key Components* as filteredOp or a subset of them.

8.3.6 Behaviour

Returns a *Recordset* with the same data structure as filteredOp.

An inner join between filteredOp and filteringOp is performed. The *Records* in filteredOp that match to a *Record* in filteringOp with true value are returned.

8.3.7 Examples

Considering the following *Recordset* from `{tT1, r010}`:

CNT	f
ES	100
PT	200
DE	300
IT	400

`filter({tT1, r010}, {tT1, r010} > 150)` keeps only the *Records* whose fact value is greater than 150:

CNT	f
PT	200
DE	300
IT	400

The filtering *Operand* `{tT1, r010} > 150` evaluates to a Boolean *Recordset* with the same *Key Components*. Only the *Records* where the Boolean value is `true` are retained; `false` and `null` values cause the corresponding *Record* to be excluded.

3.9 9 String Operators

3.9.1 9.1 Length (len)

9.1.1 Syntax

len(op **)**

9.1.2 Input Parameters

op: rset | scal <str>

9.1.3 Output

rset | scal <int>

9.1.4 Semantics

Returns the number of characters of the op string.

9.1.5 Additional Constraints

None.

9.1.6 Behaviour

- Unary *Operators'* standard behaviour.

9.1.7 Examples

- `len("test")` results in `4`
- `len(null)` results in `null`

3.9.2 9.2 Concatenate (&)

9.2.1 Syntax

op1 **&** op2

9.2.2 Input Parameters

op1: rset | scal <str>

op2: rset | scal <str>

9.2.3 Output

rset | scal <str>

9.2.4 Semantics

Concatenates two strings.

9.2.5 Additional Constraints

None.

9.2.6 Behaviour

- Binary *Operators'* standard behaviour.

9.2.7 Examples

- `"hello" & "world"` results in `"helloworld"`
- `"test" & null` results in `null`

3.9.3 9.3 Substring (substr)

9.3.1 Syntax

substr(op, {start}, {length} **)**

9.3.2 Input Parameters

op: rset | scal <str>

start: scal <int>

length: scal <int>

9.3.3 Output

rset | scal <str>

9.3.4 Semantics

Extracts the substring of op that begins at the character in position start (the first character of op is in position 1) and runs for length characters.

start and length are optional. If start is omitted, extraction begins at the first character of op. If length is omitted, the substring runs to the end of op. If both are omitted, the whole op string is returned.

9.3.5 Additional Constraints

- start must be greater than or equal to 1.
- length must be greater than or equal to 0. A length of 0 is permitted (and returns an empty string) so that a length derived from another expression, such as the difference between two positions, can collapse to an empty span without raising an error.

9.3.6 Behaviour

- Unary *Operators'* standard behaviour.
- If start is greater than the number of characters of op, an empty string is returned.
- If start + length goes past the end of op, the substring ends at the last character of op.

9.3.7 Examples

- `substr("Hello World", 7)` results in `"World"`
- `substr("Hello World", 1, 5)` results in `"Hello"`
- `substr("Hello World", 7, 100)` results in `"World"`

- `substr("Hello World")` results in `"Hello World"`
- `substr("Hello World", 20)` results in `""`
- `substr("Hello World", 7, 0)` results in `""` (a length of 0 extracts no characters)
- `substr(null, 2, 3)` results in `null`

3.10 10 Time Operators

3.10.1 10.1 Time Shift

10.1.1 Syntax

time_shift(op, period, numberPeriods, {var} **)**

10.1.2 Input Parameters

op: rset <*> | scal <date>

period: time_period_code

numberPeriods: scal <int>

var: scal <prop>

where `time_period_code` is one of the following bare keyword tokens:

Code	Period
`A`	Annual (year)
`S`	Semi-annual (semester)
`Q`	Quarterly
`M`	Monthly
`W`	Weekly
`D`	Daily

10.1.3 Output

rset <*> | scal <date>

10.1.4 Semantics

Changes the dates of the var *Component* of the *Recordset* op by adding (or subtracting) numberPeriods of the given period unit.

10.1.5 Additional Constraints

- If op is a *Recordset*, then
- var must be provided,
- must belong to the *Recordset* op, and
- must be of *Date* type.
- If op is a *Scalar*, then var must be omitted.

10.1.6 Behaviour

Returns a *Recordset* with the same data structure and number of *Records* as the input op. The dates for the *Component* var are modified by adding the numberPeriods (subtracting, if negative) of the given period unit.

10.1.7 Examples

Considering the following *Recordset*, generated from the selection {tT1, r010-020, c010-020}:

refPeriod	r	c	f
2022-01-01	010	010	100
2022-01-01	010	020	200
2022-01-01	010	010	300
2022-01-01	010	020	400
2022-04-01	020	010	500
2022-04-01	020	020	600
2022-04-01	020	010	700
2022-04-01	020	020	800

`time_shift({tT1, r010-020, c010-020}, Q, 1, refPeriod)` returns:

refPeriod	r	c	f
2022-04-01	010	010	100
2022-04-01	010	020	200
2022-04-01	010	010	300
2022-04-01	010	020	400
2022-07-01	020	010	500
2022-07-01	020	020	600
2022-07-01	020	010	700
2022-07-01	020	020	800

`time_shift({tT1, r010-020, c010-020}, Q, -1, refPeriod)` returns:

refPeriod	r	c	f
2021-10-01	010	010	100
2021-10-01	010	020	200
2021-10-01	010	010	300
2021-10-01	010	020	400
2022-01-01	020	010	500
2022-01-01	020	020	600
2022-01-01	020	010	700
2022-01-01	020	020	800

3.10.2 10.2 Date-component Extraction

The *Operators* in this section extract a single calendar component (year, semester, quarter, month, week or day) from a *Date*. They all share the same syntax, input/output types and null-propagation behaviour; they differ only in the component returned and in how they handle *Date* notations that do not pin down the requested component.

10.2.1 Common Syntax, Typing and Behaviour

10.2.1.1 SYNTAX

\<**op**>**(** op **)**

where `<op>` is one of `year`, `semester`, `quarter`, `month`, `week`, `day`.

10.2.1.2 INPUT PARAMETERS

op: rset <date> | scal <date>

10.2.1.3 OUTPUT

rset <int> | scal <int>

If op is a *Recordset*, the result is a *Recordset* with the same *Key Components* and *Records* as op, whose *Fact Component* is the extracted Integer value (or null). If op is a *Scalar*, the result is an Integer *Scalar*.

10.2.1.4 COMMON SEMANTICS

Week numbers follow the ISO-8601 convention (week 1 is the week containing the first Thursday of the year; weeks range 1..53).

10.2.1.5 COMMON NULL SEMANTICS

- If op is null, the result is null.

10.2.1.6 COMMON ADDITIONAL CONSTRAINTS

- op must be of *Date* type.

10.2.1.7 COMMON BEHAVIOUR

- Unary *Operators'* standard behaviour.
- If op is a *Recordset*, the *Operator* is applied to every *Record*'s *Fact* value (a *Date*) and the result is a *Recordset* with the same *Key Components* and *Records* whose *Fact Component* is the extracted Integer. *Attribute Components* are not propagated.

10.2.2 Year (year)

10.2.2.1 SYNTAX

year(op **)**

10.2.2.2 SEMANTICS

Returns the calendar year of op (e.g. `2022`).

10.2.2.3 BEHAVIOUR

- Date-component Extraction *Operators*' common behaviour.

10.2.2.4 EXAMPLES

- `year(#2022-03-15#)` → `2022`
- `year(null)` → `null`
- `{tT1}[where year(refPeriod) = 2022]` — filters *Records* whose `refPeriod` falls in 2022.

Considering the following *Recordset* (the selection of a *Variable* whose *Fact* is the maturity date of each loan):

`{vmaturity}`

id	f
L001	#2024-03-15#
L002	#2025-06-30#
L003	#2023-12-31#

`year({vmaturity})` returns:

id	f
L001	2024
L002	2025
L003	2023

10.2.3 Semester (semester)

10.2.3.1 SYNTAX

semester(op **)**

10.2.3.2 SEMANTICS

Returns the semester index (1 for January–June, 2 for July–December).

10.2.3.3 BEHAVIOUR

- Date-component Extraction *Operators*' common behaviour.

10.2.3.4 EXAMPLES

- `semester(#2022-03-15#)` → `1`
- `semester(#2022-08-01#)` → `2`

For the *Recordset* `{vmaturity}` of §10.2.2.4, `semester({vmaturity})` returns:

id	f
L001	1
L002	2
L003	2

10.2.4 Quarter (quarter)

10.2.4.1 SYNTAX

quarter(op **)**

10.2.4.2 SEMANTICS

Returns the quarter index (1..4).

10.2.4.3 BEHAVIOUR

- Date-component Extraction *Operators*' common behaviour.

10.2.4.4 EXAMPLES

- `quarter(#2022-03-15#)` → `1`

- `{tT1}[where quarter(refPeriod) in {1, 2}]` — first-half-year *Records*.

For the *Recordset* `{vmaturity}` of §10.2.2.4, `quarter({vmaturity})` returns:

id	f
L001	1
L002	2
L003	4

10.2.5 Month (month)

10.2.5.1 SYNTAX

month(op **)**

10.2.5.2 SEMANTICS

Returns the month of year (1..12).

10.2.5.3 BEHAVIOUR

- Date-component Extraction *Operators*' common behaviour.

10.2.5.4 EXAMPLES

- `month(#2022-03-15#)` → `3`

For the *Recordset* `{vmaturity}` of §10.2.2.4, `month({vmaturity})` returns:

id	f
L001	3
L002	6
L003	12

10.2.6 Week (week)

10.2.6.1 SYNTAX

week(op **)**

10.2.6.2 SEMANTICS

Returns the ISO-8601 week number (1..53).

10.2.6.3 BEHAVIOUR

- Date-component Extraction *Operators*' common behaviour.

10.2.6.4 EXAMPLES

- `week(#2022-03-15#)` → `11`
- `week(#2022-01-01#)` → `52` (the first day of 2022 belongs to ISO week 52 of 2021)

For the *Recordset* `{vmaturity}` of §10.2.2.4, `week({vmaturity})` returns:

id	f
L001	11
L002	26
L003	52

10.2.7 Day (day)

10.2.7.1 SYNTAX

day(op **)**

10.2.7.2 SEMANTICS

Returns the day of month (1..31).

10.2.7.3 BEHAVIOUR

- Date-component Extraction *Operators*' common behaviour.

10.2.7.4 EXAMPLES

- `day(#2022-03-15#)` → `15`
- `day(#2022-01-01#)` → `1`
- `day(null)` → `null`

For the *Recordset* `{vmaturity}` of §10.2.2.4, `day({vmaturity})` returns:

id	f
L001	15
L002	30
L003	31

3.10.3 10.3 Date Constructor (date)

10.3.1 Syntax

date(year, month, day **)**

10.3.2 Input Parameters

year: rset <int> | scal <int>

month: rset <int> | scal <int>

day: rset <int> | scal <int>

10.3.3 Output

rset <date> | scal <date>

10.3.4 Semantics

Builds a *Date* (notation `#YYYY-MM-DD#`, see §2.4.2) from the three Integer parts.

This *Operator* is the sanctioned path to obtain a *Date* from Integers: the casting table in §2.3.2 marks Integer → Date as *Not possible*.

10.3.5 Additional Constraints

- year, month and day must be of *Integer* type.
- Any combination of *Scalar* and *Recordset* operands is allowed, provided the *Recordset* operands satisfy the constraints in §2.2 Binary Operators, generalised to three operands: *Recordset* operands must share *Key Components* (or one's *Key Components* must be a superset of the others'), and *Standard Keys* must contain matching values.

10.3.6 Behaviour

- Binary *Operators*' standard behaviour, generalised to three operands.
- If all three operands are *Scalars*, the result is a *Scalar Date* built from the three values.
- If any operand is a *Recordset*, the result is a *Recordset* whose *Key Components* are the common *Key Components* of the *Recordset* operands (or those of the Reference *Recordset* if one's *Keys* are a superset of the others'), and whose *Fact Component* is the Date built per *Record*. Per-*Record* construction uses the inner join of the *Recordset* operands on their common *Key Components*; *Scalar* operands are broadcast to every *Record*. *Attribute Components* are not propagated.
- If any of year, month or day is null on a given *Record* (or *Scalar*), the resulting *Date* for that *Record* (or *Scalar*) is null.
- If the three values do not form a valid calendar date — month outside 1..12, or day outside the valid range for that month and year (accounting for leap years; e.g. `date(2025, 2, 30)`) — the resulting *Date* is null.

10.3.7 Examples

- `date(2025, 12, 31)` → `#2025-12-31#`
- `date(2024, 2, 29)` → `#2024-02-29#` (2024 is a leap year)
- `date(2025, 2, 30)` → `null` (not a valid date)
- `date(2025, 13, 1)` → `null` (month out of range)
- `date(null, 1, 1)` → `null`
- `date(year(refPeriod), 1, 1)` — first day of each *Record*'s reference year (round-trips with the extraction family of §10.2).

Considering the following *Recordsets* (selections of three *Variables* sharing the same *Key Components*):

`{vyyyy}`

id	f
A	2024
B	2025

`{vmm}`

id	f
A	2
B	12

`{vdd}`

id	f
A	29
B	31

`date({vyyyy}, {vmm}, {vdd})` returns:

id	f
A	#2024-02-29#
B	#2025-12-31#

Mixing *Recordsets* and *Scalars* is also allowed; the *Scalar* is broadcast. `date({vyyyy}, 1, 1)` returns:

id	f
A	#2024-01-01#
B	#2025-01-01#

3.10.4 10.4 Annualise

10.4.1 Syntax

annualise(op, fyEnd, var **)**

10.4.2 Input Parameters

op: rset <num> | scal <num>

fyEnd: scal <int>

var: scal <prop>

10.4.3 Output

rset <num> | scal <num>

10.4.4 Semantics

Annualise scales each numeric value in op up to its full-year equivalent based on where the *Record*'s reference month sits within the surrounding *financial year*. The scaling is computed in **whole months**: a *financial year* always spans 12 months, and each value is grossed up by the ratio of 12 to the number of months elapsed since the start of the *financial year*.

The relevant quantities are:

flowchart LR subgraph FY["Financial Year — 12 months"] direction LR fyStart["fyStart (month after fyEnd)"] -- "elapsed months" --> m["reference month m"] m -- "remaining months" --> fyEndMonth["fyEnd"] end

where, for each *Record*:

- `m` is the month of the *Record*'s reference date — the month component (1..12) of the value of `var` for that *Record*. Only the month is used; the day within the month does not affect the result.
- `fyEnd` is the month code of the last month of the *financial year*: an integer in 1..12 (e.g., `12` for a calendar *financial year*, `3` for a *financial year* ending in March).
- `elapsed` is the number of months from the start of the *financial year* up to and including month `m`. The *financial year* starts in the month following `fyEnd`, so:

```
elapsed = 12 - (fyEnd - m)    if fyEnd >= m
elapsed = m - fyEnd           if fyEnd <  m
```

The annualised value for each *Record* is then:

```
annualised = value * (12 / elapsed)
```

10.4.5 Additional Constraints

- `fyEnd` must be an integer in the range 1..12. Any other value raises a runtime error.
- `var` must be provided, must belong to op, as a *Key* or *Attribute Component* of the *Recordset*, and must be of *Date* type. Only its month component is used.
- The *Fact Component* of op (or the *Scalar*'s value) must be numeric.
- If the reference date for a *Record* is `null`, the annualised value for that *Record* is `null`..

10.4.6 Behaviour

Returns a *Recordset* (or *Scalar*) with the same *Structure* and number of *Records* as op. The *Fact Component* of each *Record* is replaced by its annualised value, computed independently per *Record*. *Key* and *Attribute Components* are preserved unchanged.

10.4.7 Examples

Considering the following *Recordset*, generated from the selection `{tP01, r010, c010}` and reporting partial-year balances at different cut-off dates within a calendar *financial year*:

refPeriod	f
2022-03-31	100
2022-06-30	250
2022-09-30	400
2022-12-31	600

`annualise({tP01, r010, c010}, 12, refPeriod)` — calendar *financial year* (ends in December) — returns:

refPeriod	f
2022-03-31	400.00
2022-06-30	500.00
2022-09-30	533.33
2022-12-31	600.00

(For the first *Record*: `m = 3`, `fyEnd = 12`, and since `fyEnd >= m`, `elapsed = 12 - (12 - 3) = 3` months, so `100 * 12 / 3 = 400.00`. The last *Record* sits in month `fyEnd`, so `elapsed = 12` and the value is unchanged.)

`annualise({tP01, r010, c010}, 3, refPeriod)` — *financial year* ending in March — returns:

refPeriod	f
2022-03-31	100.00
2022-06-30	1000.00
2022-09-30	800.00
2022-12-31	800.00

(For 2022-03-31: `m = 3`, `fyEnd = 3`, and since `fyEnd >= m`, `elapsed = 12 - (3 - 3) = 12` months — it sits in month `fyEnd`, so the value is unchanged. For 2022-06-30: `m = 6`, `fyEnd = 3`, and since `fyEnd < m`, `elapsed = 6 - 3 = 3` months, so `250 * 12 / 3 = 1000.00`. Note: with `fyEnd = 3`, the first cut-off belongs to a different *financial year* from the other three.)

For a *Scalar* operand, the contextual `refPeriod` of the instance (always latent — see §2.2.3.1.4) is passed explicitly as `var`:

```
annualise({tP01, r010, c010}, 12, refPeriod)
```

3.11 11 Clause Operators

Clause *Operators* serve to perform operations on the *Components* of *Recordsets*. Which roles of *Component* (*Key*, *Fact* or *Attribute*) each clause *Operator* accepts is specified in its Additional Constraints: `where` may reference any of them, while `rename`, `get` and `sub` apply only to *Key* and, where indicated, *Attribute Components*. Use of clause *Operators* with Standard *Key Components* ("r", "c" or "s") is not allowed.

3.11.1 11.1 where

11.1.1 Syntax

op **[where** condition **]**

11.1.2 Input Parameters

op: rset <*>

condition: expression <boo>

11.1.3 Output

rset <*>

11.1.4 Semantics

Filters a *Recordset* based on the values of its *Components*.

11.1.5 Additional Constraints

Condition must be a Boolean expression using as input *Components* of op — any of its DPM *Key Components*, its *Fact Component*, or its *Attribute Components*. The contextual *Components* are latent in every *Recordset* and may therefore always be referenced: `refPeriod` and `entityID` (§2.2.3.1.4), and the contextual *Attribute Components* (§2.2.3.3.1).

Standard *Key Components* ("r", "c" or "s") cannot be referenced in the condition.

11.1.6 Behaviour

Returns a *Recordset* with the same data structure as the input *Operand*, and with the *Records* resulting from the evaluation of the filtering condition. When the condition evaluates to true, the *Record* is kept, otherwise (including null), the *Record* is not kept.

11.1.7 Examples

Considering the following *Recordset*, generated from the selection `{tT1, r010-020}` :

refPeriod	r	CNT	f
2022Q1	010	ES	100
2022Q1	010	PT	200
2022Q1	010	DE	300
2022Q1	010	IT	400
2022Q2	020	ES	500
2022Q2	020	PT	600
2022Q2	020	DE	700
2022Q2	020		800

`{tT1, r010-020}[where CNT in {"ES", "PT"}]` returns:

refPeriod	r	CNT	f
2022Q1	010	ES	100
2022Q1	010	PT	200
2022Q2	020	ES	500
2022Q2	020	PT	600

`{tT1, r010-020}[where f != 0]` keeps only the *Records* with a non-zero *Fact* value. In the example above, all the *Records* are kept, as none of them has a zero or null *Fact*.

`{tT1, r010-020}[where f > 300 and CNT = "PT"]` combines *Fact* and *Key* filtering, and returns:

refPeriod	r	CNT	f
2022Q2	020	PT	600

`{tT1, r010-020}[where not isnull(f)]` keeps only the *Records* with a non-null *Fact* value.

3.11.2 11.2 rename

11.2.1 Syntax

op **[rename** compFrom to compTo {, compFrom to compTo} *]**

11.2.2 Input Parameters

op: rset <*>

compFrom: scal <prop>

compTo: scal <prop>

11.2.3 Output

rset <*>

11.2.4 Semantics

Changes the name of a *Component*.

11.2.5 Additional Constraints

compFrom must belong to op.

compFrom may refer to a DPM *Key Component* or to an *Attribute Component*, including a contextual *Attribute Component* (§2.2.3.3.1), which is latent in every *Recordset* and therefore always belongs to op. It cannot refer to a Standard *Key Component* (i.e., cannot refer to row, column or sheet), nor to the *Fact Component* (the name of the *Fact Component* is always "f", as specified in §2.2.3.2).

compTo name cannot be already used in op.

compTo name cannot be the name of a standard *Key Component* ("r", "c" or "s"), nor the name of a contextual *Component* (§2.4.3), which would make the contextual *Component* unaddressable.

11.2.6 Behaviour

Returns a *Recordset* with the same data structure as the input *Operand*, except the renamed *Components*, which change name. The role of each renamed *Component* (*Key* or *Attribute*) is preserved. All the *Records* are kept.

11.2.7 Examples

Considering the following *Recordset*, generated from the selection `{tT1, r010-020}` :

refPeriod	r	CNT	f
2022Q1	010	ES	100
2022Q1	010	PT	200
2022Q1	010	DE	300
2022Q1	010	IT	400
2022Q2	020	ES	500
2022Q2	020	PT	600
2022Q2	020	DE	700
2022Q2	020		800

`{tT1, r010-020}[rename CNT to country]` returns:

refPeriod	r	country	f
2022Q1	010	ES	100
2022Q1	010	PT	200
2022Q1	010	DE	300
2022Q1	010	IT	400
2022Q2	020	ES	500
2022Q2	020	PT	600
2022Q2	020	DE	700
2022Q2	020		800

Considering now a *Recordset* with an *Attribute Component* "p" (precision), generated from the selection `{tT2, r010}` :

refPeriod	CNT	f	p
2022Q1	ES	100	1
2022Q1	PT	200	0

`{tT2, r010}[rename p to precision]` renames the *Attribute Component*, and returns:

refPeriod	CNT	f	precision
2022Q1	ES	100	1
2022Q1	PT	200	0

3.11.3 11.3 get

11.3.1 Syntax

op **[get** component **]**

11.3.2 Input Parameters

op: rset <*>

component: scal <prop>

11.3.3 Output

rset <*>

11.3.4 Semantics

Returns a *Recordset* where the *Fact* is the value of one of the *Key* or *Attribute Components*.

11.3.5 Additional Constraints

component must belong to op.

component may refer to a DPM *Key Component* or to an *Attribute Component*, including a contextual *Attribute Component* (§2.2.3.3.1), which is latent in every *Recordset* and therefore always belongs to op. It cannot refer to a Standard *Key Component* (i.e., cannot refer to row, column or sheet), nor to the *Fact Component* (the operation would have no effect).

11.3.6 Behaviour

Returns a *Recordset* with the same data structure as the input *Operand*. The *Fact* values the *Records* are substituted for the value of the selected *Component*. The *Data Type* of the *Fact Component* of the result is the *Data Type* of the selected *Component*.

Records with null *Fact* value will be omitted.

11.3.7 Examples

Considering the following *Recordset*, generated from the selection `{tT1, r010-020}` :

refPeriod	r	CNT	f
2022Q1	010	ES	100
2022Q1	010	PT	200
2022Q1	010	DE	300
2022Q1	010	IT	400
2022Q2	020	ES	500
2022Q2	020	PT	600
2022Q2	020	DE	700
2022Q2	020	IT	800

`{tT1, r010-020}[get CNT]` returns:

refPeriod	r	CNT	f
2022Q1	010	ES	ES
2022Q1	010	PT	PT
2022Q1	010	DE	DE
2022Q1	010	IT	IT
2022Q2	020	ES	ES
2022Q2	020	PT	PT
2022Q2	020	DE	DE
2022Q2	020	IT	IT

Considering now a *Recordset* with an *Attribute Component* "p" (precision), generated from the selection `{tT2, r010}` :

refPeriod	CNT	f	p
2022Q1	ES	100	1
2022Q1	PT	200	0

`{tT2, r010}[get p]` promotes the value of the *Attribute Component* into the *Fact Component*, so that it can be further aggregated or compared, and returns:

refPeriod	CNT	f	p
2022Q1	ES	1	1
2022Q1	PT	0	0

3.11.4 11.4 sub

11.4.1 Syntax

op **[sub** key_component = value {, key_component = value} *]**

11.4.2 Input Parameters

op: rset <*>

key_component: scal <prop>

The right-hand side `value` accepts three forms:

Form	Syntax	Description
Literal	A scalar literal value	A constant of a basic type (String, Number, Boolean, Date, etc.) written directly in the expression.
Select	`{` selectOperand `}`	A selection expression that resolves to a *Scalar* value (e.g., a single-cell table selection or a *Variable* reference).
Item reference	`[` dimension `:` member `]`	A dimension member reference using the `dimension:member` notation.

11.4.3 Output

rset | scal <*>

11.4.4 Semantics

Filters a *Recordset* based on the value of a *Key Component*, and then drops the *Key Component*. If all *Key Components* are substituted, the result is a *Scalar*.

11.4.5 Additional Constraints

key_component must belong to op, and must be a DPM *Key Component*. It cannot refer to the *Fact Component* (substituting it would leave the *Recordset* without a value) nor to an *Attribute Component*.

The value needs to have the same *Data type* as the key_component.

11.4.6 Behaviour

Returns a *Recordset* with the same data structure as the input *Operand*, except the *Key Components* dropped, and with the *Records* resulting from the evaluation of the filtering condition. When the condition evaluates to true, the *Record* is kept, otherwise (including null), the *Record* is not kept. If no *Key Components* remain after the substitution, the result is a *Scalar*.

11.4.7 Examples

Considering the following *Recordset*, generated from the selection `{tT1, r010-020}`:

refPeriod	r	CNT	f
2022Q1	010	ES	100
2022Q1	010	PT	200
2022Q1	010	DE	300
2022Q1	010	IT	400
2022Q2	020	ES	500
2022Q2	020	PT	600
2022Q2	020	DE	700
2022Q2	020		800

Form 1 — Literal

`{tT1, r010-020}[sub CNT = "PT"]` returns:

refPeriod	r	f
2022Q1	010	200
2022Q2	020	600

Form 2 — Select

Where `{vCountry}` is a *Variable* that resolves to the String `"ES"`:

`{tT1, r010-020}[sub CNT = {vCountry}]` returns:

refPeriod	r	f
2022Q1	010	100
2022Q2	020	500

Form 3 — Item reference

`{tT1, r010-020}[sub CNT = [CNT:ES]]` returns:

refPeriod	r	f
2022Q1	010	100
2022Q2	020	500

3.12 12 Assignment Operators

Assignment *Operators* serve to assign the result of an operation to a name or to a *DPM Variable*.

3.12.1 12.1 Non-persistent assignment (:=)

12.1.1 Syntax

name := op

12.1.2 Input Parameters

name: the name assigned to the result

op: the operand, which can be any dpm-xl expression

12.1.3 Output

Assigns the name to the op.

12.1.4 Semantics

Allows a *DPM-XL* operation to use the result of another operation as input. By assigning a name to the result of an operation, this result can be selected using the selection operator.

12.1.5 Additional Constraints

name must be a valid temporary identifier as defined in the grammar.

12.1.6 Behaviour

The value of the *Operand* op is assigned the name *name*, which is non-persistent and therefore is not stored. The operand op can be obtained through an expression as complex as needed (for example: `op1 := {tT1, r010-020} * 10`).

3.12.2 12.2 Persistent assignment (<-)

12.2.1 Syntax

var_sel <- op

12.2.2 Input Parameters

var_sel: a selection operator with the output variable

op: the operand, which can be any dpm-xl expression

12.2.3 Output

Assigns the result of op to the var_sel.

12.2.4 Semantics

Links the calculation of a *DPM Variable* to an expression.

12.2.5 Additional Constraints

var_sel is a selection *Operator* that refers to a *Variable* or set of *Variables* that are defined in the *DPM*. The resulting *Structure* in op needs to be equal to the *Structure* of var_sel.

12.2.6 Behaviour

The value of the *Operand* op is assigned the *Variables* selected in var_sel, with the objective of storing the result. The operand op can be obtained through an expression as complex as needed (for example: `{t0T1, r010-020} <- {tT1, r010-020} * 10`).

3.13 13 Set Operators

Set *Operators* construct and combine *Scalar Sets* (see §2.2.2). A *Scalar Set* is an unordered, duplicate-free collection of *Scalar* values that share a single *Data Type*. Until now, *Scalar Sets* could only be written as literals (`{v1, v2, ...}`, or `{}` for the empty set) or obtained from a *Subcategory*, and were consumed only by the `in` *Operator* (§5.8). The *Operators* in this section make *Scalar Sets* first-class:

- `set_of` derives a *Scalar Set* from the *Fact Component* of a *Recordset*.
- `union`, `intersect`, `setdiff` and `symdiff` combine *Scalar Sets*.

The cardinality of a *Scalar Set* (its number of distinct elements) is obtained with the `count` *Operator* (§7.3), which accepts a `scalar_set` in addition to a *Recordset*.

Each combining *Operator* (`union`, `intersect`, `setdiff`, `symdiff`) is illustrated below with a Venn diagram of two example sets, `set 1 = {1, 2, 3}` and `set 2 = {3, 4, 5}`, in which the shaded region marks the elements that end up in the result.

A set-valued expression — a literal, a *Subcategory*, a `set-*` *Parameter*, or the result of any *Operator* in this section — may appear as the right-hand side of `in`, as the *Operand* of `count`, as an *Operand* of the binary and variadic set *Operators* (so they nest freely, e.g. `union(set_of(A), setdiff(B, C))`), and as an *Operand* of the equality *Operators* `=` and `!=` (see Set Equality).

Where a set *Operand* is written as a *Recordset* or *Subcategory* selection, it is implicitly read as the *Scalar Set* of its values — equivalent to wrapping it in `set_of` (§13.2). So `union({tC_28.00, c020}[get qINC], {tC_29.00, c0030}[get qINC])` is accepted directly, without an explicit `set_of` around each selection.

These are **semantic** rules, not syntactic ones: the grammar accepts an ordinary expression in every set position, and semantic analysis is responsible for the type restrictions. In particular, set-valued expressions are **not** *Scalars* or *Recordsets*, so — apart from set equality — they cannot be combined with scalar/recordset *Operators*: arithmetic and the ordering comparisons (`<`, `>`, `<=`, `>=`) are not defined on *Scalar Sets*, and a comparison such as `{c020} = setdiff(A, B)` (a *Recordset* against a *Scalar Set*) is rejected by semantic analysis even though it parses.

3.13.1 13.1 Set Operators' General Behaviour

13.1.1 Syntax

setOperator (s1, s2 {, s3}* **)**

13.1.2 Input Parameters

s1, s2, ...: scalar_set <> | *subcategory* | *rset* <>

13.1.3 Output

scalar_set <T>

13.1.4 Semantics

The binary and variadic set *Operators* combine their *Operands* into a new *Scalar Set*. Membership is decided element-wise using the equality *Operator* (`=`), so two elements are "the same" exactly when they compare equal under `=`; for intervals, this inherits the interval semantics of `=` (see §5.2). The result is always a *Scalar Set*: unordered and duplicate-free.

13.1.5 Additional Constraints

- All elements within a *Scalar Set* must share a single *Data Type*, considering implicit casting.
- For a binary or variadic set *Operator*, the *Operands* must have compatible element *Data Types*. The element *Data Type* of the result is the maximum common *Data Type* of the *Operands*, determined by the implicit casting hierarchy (§2.3.2) — e.g. combining an Integer set with a Number set yields a Number set.

- A *Subcategory Operand* is implicitly treated as a *Scalar Set* of its member items, consistently with the `in` *Operator* (§5.8).
- A *Recordset Operand* is implicitly treated as the *Scalar Set* of its *Fact Component* values, exactly as if wrapped in `set_of` (§13.2). To take the set of a *Key* or *Attribute Component* instead, move it into the *Fact* first with the get clause — e.g. `union(A[get CNT], B[get CNT])`.
- `null` is **not** a valid set element. Set *Operators* never introduce `null`; any `null` arising at construction time (e.g. from `set_of`) is dropped.

13.1.6 Behaviour

Set *Operators* operate purely on the values of their *Operands*; they do not carry *Key* or *Attribute Components*, since their result is a *Scalar Set*, not a *Recordset*. The result contains each distinct value at most once.

3.13.2 13.2 Set of (set_of)

13.2.1 Syntax

set_of(op **)**

13.2.2 Input Parameters

op: rset <*>

13.2.3 Output

scalar_set <T> — where T is the *Data Type* of the *Fact Component* of op.

13.2.4 Semantics

Returns the *Scalar Set* of distinct, non-null values taken by the *Fact Component* across the *Records* of op.

To obtain the set of values of a *Key* or *Attribute Component*, first move that *Component* into the *Fact* with the get clause *Operator*, then apply `set_of` — e.g. `set_of(op[get CNT])`.

13.2.5 Additional Constraints

None.

13.2.6 Behaviour

Each *Record* of op contributes its *Fact* value. Duplicate values collapse to a single element, and *Records* whose *Fact* value is `null` contribute nothing. The result is a *Scalar Set* whose element *Data Type* is that of the *Fact Component* of op.

13.2.7 Examples

Considering the following *Recordset*, generated from the selection `{tT1, r010-020}` :

refPeriod	r	CNT	f
2022Q1	010	ES	100
2022Q1	010	PT	200
2022Q1	010	DE	300
2022Q1	010	IT	400
2022Q2	020	ES	500
2022Q2	020	PT	600
2022Q2	020	DE	700
2022Q2	020	IT	800

`set_of({tT1, r010-020})` results in `{100, 200, 300, 400, 500, 600, 700, 800}` — the distinct *Fact* values.

`set_of({tT1, r010-020}[get CNT])` results in `{"ES", "PT", "DE", "IT"}` — the `get` clause first moves `CNT` into the *Fact*.

3.13.3 13.3 Union (union)

13.3.1 Syntax

union(s1, s2 {, s3}* **)**

13.3.2 Input Parameters

s1, s2, ...: scalar_set <*> | subcategory

13.3.3 Output

scalar_set <T>

13.3.4 Semantics

Returns the *Scalar Set* of elements present in **any** of the *Operands*. `union` accepts two or more *Operands*.

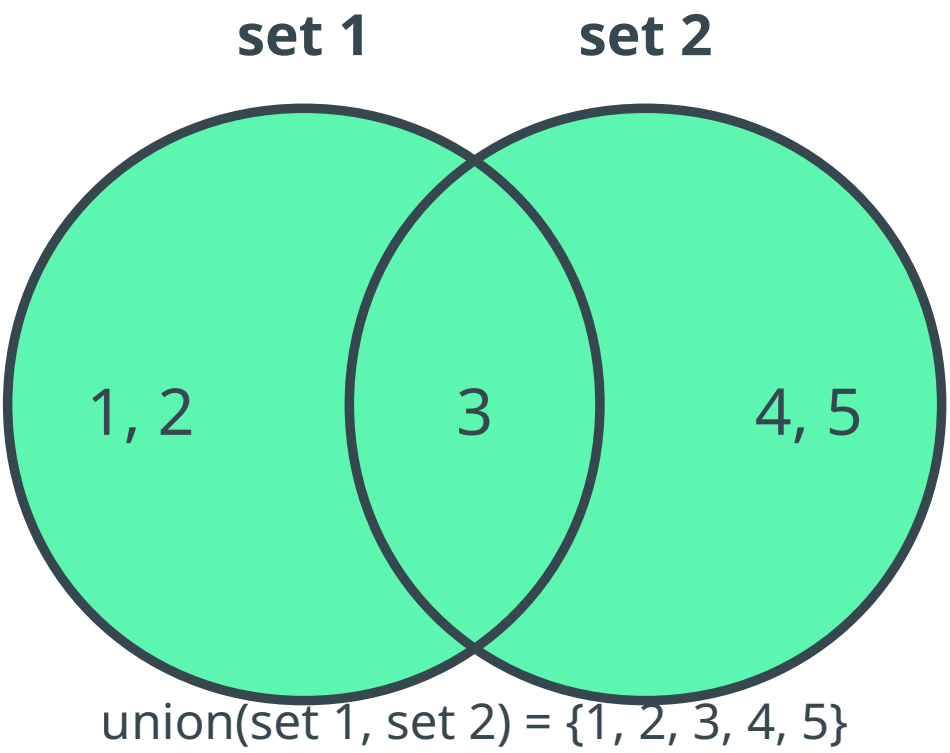
13.3.5 Additional Constraints

- Set *Operators*' general behaviour.

13.3.6 Behaviour

- Set *Operators*' general behaviour.

13.3.7 Examples



- `union({1, 2, 3}, {3, 4, 5})` results in `{1, 2, 3, 4, 5}`.
- `union({1}, {2}, {3})` results in `{1, 2, 3}`.

3.13.4 13.4 Intersection (intersect)

13.4.1 Syntax

intersect(s1, s2 {, s3}* **)**

13.4.2 Input Parameters

s1, s2, ...: scalar_set <*> | subcategory

13.4.3 Output

scalar_set <T>

13.4.4 Semantics

Returns the *Scalar Set* of elements present in **all** of the *Operands*. `intersect` accepts two or more *Operands*.

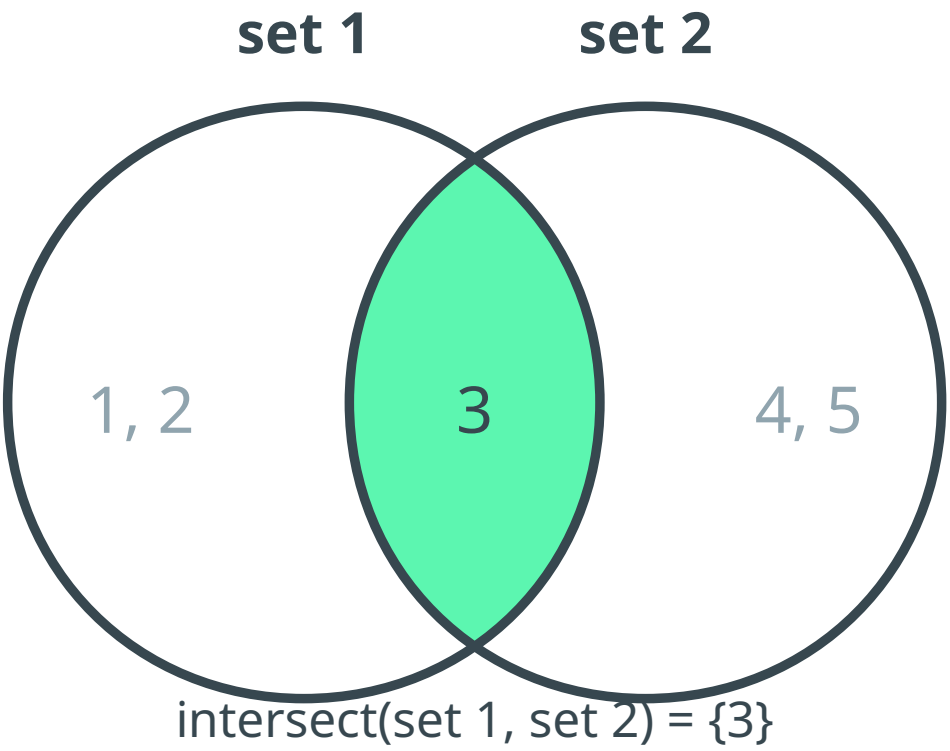
13.4.5 Additional Constraints

- Set *Operators*' general behaviour.

13.4.6 Behaviour

- Set *Operators*' general behaviour.

13.4.7 Examples



- `intersect({1, 2, 3}, {3, 4, 5})` results in `{3}`.
- `intersect({1, 2, 3}, {2, 3, 4}, {3, 4, 5})` results in `{3}`.

3.13.5 13.5 Set Difference (setdiff)

13.5.1 Syntax

setdiff(s1, s2 **)**

13.5.2 Input Parameters

s1, s2: scalar_set <*> | subcategory

13.5.3 Output

scalar_set <T>

13.5.4 Semantics

Returns the *Scalar Set* of elements in `s1` that are **not** in `s2`. `setdiff` is binary and **not** symmetric: the order of the *Operands* matters.

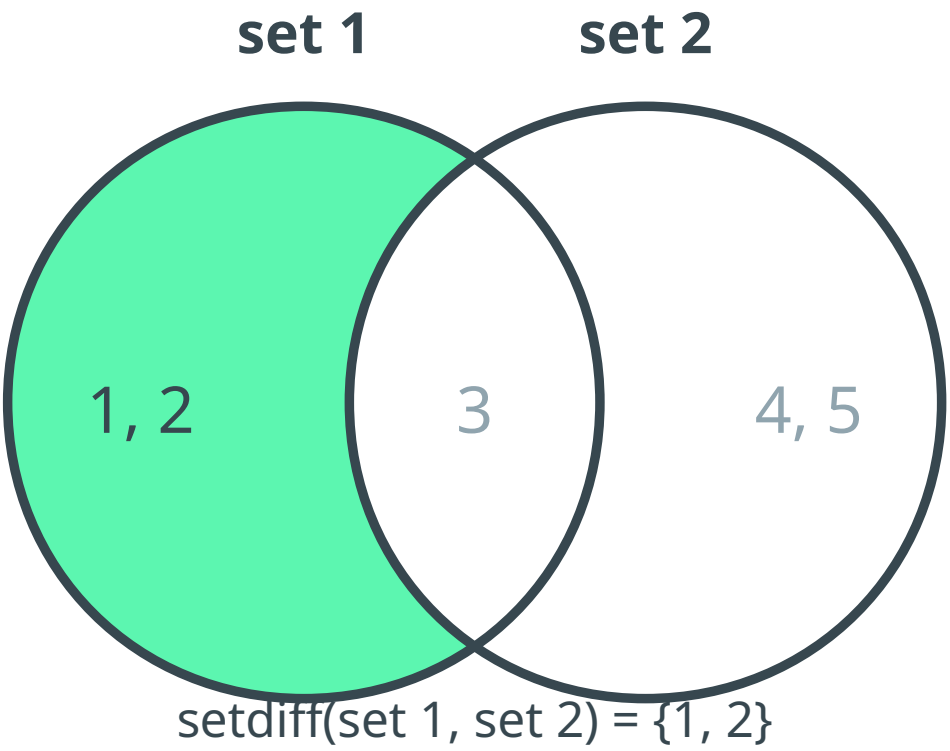
13.5.5 Additional Constraints

- Set *Operators*' general behaviour.

13.5.6 Behaviour

- Set *Operators*' general behaviour.

13.5.7 Examples



- `setdiff({1, 2, 3}, {3, 4, 5})` results in `{1, 2}`.
- `setdiff({3, 4, 5}, {1, 2, 3})` results in `{4, 5}`.

3.13.6 13.6 Symmetric Difference (symdiff)

13.6.1 Syntax

symdiff(s1, s2 **)**

13.6.2 Input Parameters

s1, s2: scalar_set <*> | subcategory

13.6.3 Output

scalar_set <T>

13.6.4 Semantics

Returns the *Scalar Set* of elements in **exactly one** of `s1` or `s2` — equivalently, `setdiff(union(s1, s2), intersect(s1, s2))`. `symdiff` is binary and symmetric.

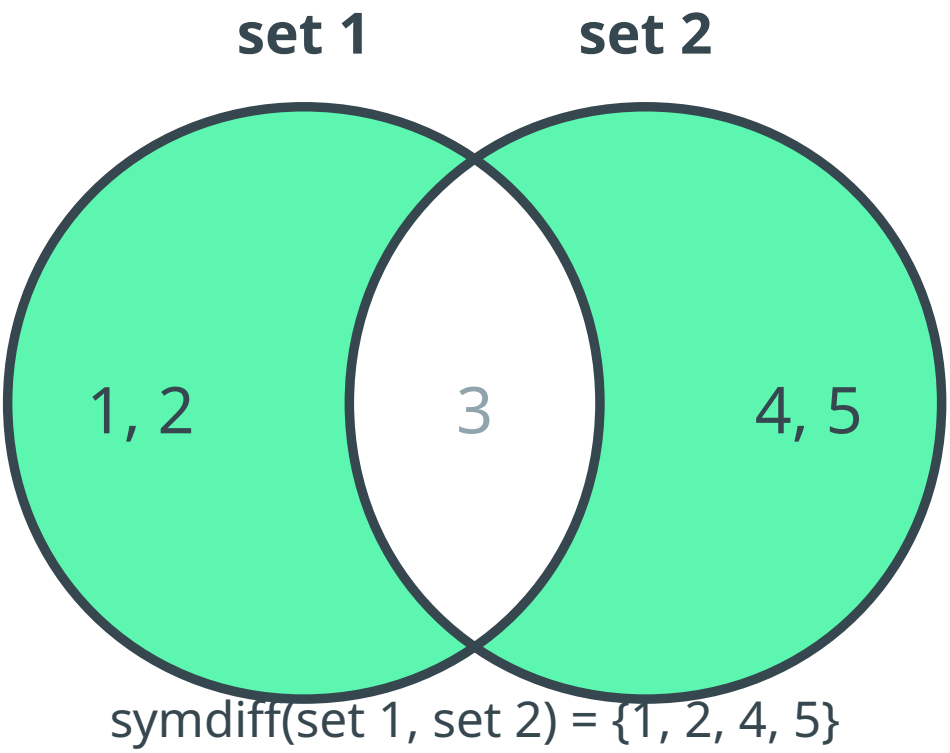
13.6.5 Additional Constraints

- Set *Operators*' general behaviour.

13.6.6 Behaviour

- Set *Operators*' general behaviour.

13.6.7 Examples



- `symdiff({1, 2, 3}, {3, 4, 5})` results in `{1, 2, 4, 5}`.
- `symdiff({1, 2}, {1, 2})` results in `{}` (the empty set).

3.13.7 13.7 Set Equality (=, !=)

The equality *Operators* `=` and `!=` accept two set-valued *Operands* and compare them as *Scalar Sets*, returning a single Boolean. This is the only comparison defined on *Scalar Sets*: the ordering *Operators* (`<`, `>`, `<=`, `>=`) and the arithmetic *Operators* are **not** defined on sets.

13.7.1 Syntax

```
s1 = s2
```

```
s1 != s2
```

13.7.2 Input Parameters

```
s1, s2: scalar_set <*> | subcategory
```

13.7.3 Output

```
scal <boo>
```

13.7.4 Semantics

`s1 = s2` returns `true` when the two *Scalar Sets* contain exactly the same elements — every element of `s1` is in `s2` and every element of `s2` is in `s1` — and `false` otherwise. `s1 != s2` is its negation. Membership is decided with the same element-wise equality as the other set *Operators* (§13.1.4); order and duplicates are irrelevant, since both *Operands* are *Scalar Sets*.

The empty set is written `{}`. Comparing against it tests emptiness: `s = {}` is `true` exactly when `s` has no elements.

13.7.5 Additional Constraints

- The two *Operands* must have compatible element *Data Types*, following the same rule as the combining set *Operators* (§13.1.5).
- Set equality applies when **both** *Operands* are set-valued (a set literal, a set *Operator* result, a `set-*` *Parameter*, or a *Subcategory*). When both *Operands* are instead *Scalars* or *Recordsets*, `=` and `!=` denote the ordinary comparison (§5.2, §5.3); a comparison that mixes a *Scalar Set* with a *Scalar* or *Recordset* is rejected by semantic analysis. The grammar does not distinguish these cases — the choice is made from the *Operand* types. For instance, `{c020} = {c030}` is a *Recordset* comparison when `c020`/`c030` are ordinary *Recordset* selections; wrap an *Operand* in `set_of` (or use a set literal or set *Operator*) to make the comparison set-valued.

13.7.6 Behaviour

Both *Operands* are evaluated to *Scalar Sets* and compared by value; the result is a single Boolean *Scalar*.

13.7.7 Examples

- `{1, 2, 3} = {3, 2, 1}` results in `true` (order does not matter).
- `{1, 2} = {1, 2, 3}` results in `false`.
- `symdiff({1, 2}, {1, 2}) = {}` results in `true` (equal sets have an empty symmetric difference).
- `setdiff({tT1, r010}[get CNT], {tT2, r020}[get CNT]) = {}` results in `true` when every `CNT` value of the first selection also appears in the second — i.e. the first set is a subset of the second. Each selection is coerced to its *Scalar Set* (§13.1.5), so no explicit `set_of` is needed.

4. Grammar

4.1 Introduction

The DPM-XL grammar provides the technical definition of the syntax of the language. It allows developers to build parsers for the language. It is provided in two .g4 files. The grammar uses the Extended Backus-Naur Form (EBNF)

Grammar files:

- Lexer
- parser

5. Script Format

5.1 1 Introduction

5.1.1 1.1 Scope

This part specifies the **DPM-XL Script Document**: a JSON serialisation of the complete set of DPM-XL *Operations* applicable to one module version, together with the metadata required to evaluate them.

A *Script Document* is the deliverable that a DPM authority may publish and that a validation engine is expected to be able to consume. It is generated from the DPM-ML metamodel (see Information Model §5 DPM-ML) and is intended to be the **only** input an engine needs, besides the reported data itself.

This part is **normative**. It occupies the same position as the Grammar: where the grammar defines what a well-formed DPM-XL expression looks like, this part defines what an engine actually receives and how it must be interpreted.

What this part specifies is an **encoding**, and only an encoding. Three things are therefore outside its scope, and are named here because each has at some point been mistaken for its business:

- **What an expression means.** The semantics of every *Operator* belong to the Operators part and the Information Model. This part encodes *Operations* whose meaning is settled elsewhere; where it appears to constrain meaning, it is citing those parts, not adding to them.
- **Whether an expression is meaningful.** Type agreement between operands, the admissibility of an argument to an *Operator*, and every other question of semantic analysis are settled **before** a *Script Document* is produced. A *Script Document* is the output of a pipeline whose semantic analysis has already passed, so this part neither restates those rules nor asks a Consumer to re-derive them.
- **What a Consumer produces.** The result of evaluating a document — its outcome categories, counts, messages and report structure — is implementation-specific by design. What is standardised here is the **input** an engine receives, so that independent engines consume the same documents; what they emit is their own concern.

The structural validation rules of §5.3 mark the boundary precisely: each is decidable from the document alone, without knowing what any *Operator* means, without a DPM database, and without an instance.

5.1.2 1.2 Why a Serialised Script Format

A DPM-XL expression is not self-contained. The expression

```
with {tC_28.00, default: 0, interval: true}: {c040} >= {c050}
```

denotes nothing until `C_28.00` is resolved to a table version, `c040` and `c050` to sets of *Variables*, and the enclosing `with` context to a fallback value and an interval-arithmetics setting. All of that lives in the DPM. Publishing expression strings alone would therefore oblige every engine to embed a copy of the DPM database and reimplement its resolution logic.

The *Script Document* exists to remove that obligation. Five properties motivate it:

1. **Self-containment.** Every *Operand* carries its own fully resolved coordinates and *Variable* identifiers, including data types. Evaluation requires no lookup outside the document.
2. **Decoupling.** Engines should be able to run the validations without needing to access the DPM database.
3. **Determinism.** Range expansion, `with` propagation and *Operand* index assignment are performed once, by the authority. Two conforming engines therefore combine exactly the same *Variables* in exactly the same pairs.
4. **Auditability.** Because the expansion is published rather than recomputed, a validation result can be traced to a specific *Variable* set, and two releases of the same module can be diffed mechanically.
5. **Deployment.** One file per module version, transferable and cacheable, with no database dependency.

5.1.3 1.3 Relationship to the Other Parts of This Specification

Part	Governs
Information Model	The data structures DPM-XL operates on: *Recordsets*, *Records*, *Components*, *Variables*, null treatment.
Operators	The semantics of each *Operator* — signature, output type, behaviour. **Authoritative for meaning.**
Grammar	The concrete surface syntax of DPM-XL expressions.
Script Format (this part)	The serialised, resolved form in which *Operations* are published and consumed. **Authoritative for encoding.**

This part defines no semantics of its own. For every *Operator* it specifies the encoding and refers to the Operators part for behaviour. Where an encoding rule here appears to imply a semantics that the Operators part contradicts, the Operators part prevails.

5.1.4 1.4 Conformance

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY** and **OPTIONAL** are to be interpreted as described in RFC 2119.

Two conformance targets are defined:

- A **conforming Producer** is a process that emits *Script Documents*. It MUST satisfy every requirement in §5.1 and MUST emit only documents that satisfy the structural validation rules of §5.3.
- A **conforming Consumer** is a process that reads *Script Documents* and evaluates the *Operations* they contain. It MUST satisfy every requirement in §5.2.

A *Script Document* is **valid** if and only if it satisfies every rule in §5.3.

Encoding, media type and file naming are specified in §3.1.

5.1.5 1.5 Terminology

Term	Definition
Script Document	A single JSON document conforming to this part.
Module Object	The value of the single member of the root object; carries all content for one module version (§3.2).
Operation Object	The serialised form of one *OperationVersion* for this module version (§3.10).
Node	One JSON object in an abstract syntax tree, discriminated by `class_name` (§4.1).
Operand Node	A *Node* denoting data, a literal or a named value rather than an *Operator*: `VarID`, `Constant`, `Scalar`, `Set`, `Dimension`, `ParameterRef`, `PreconditionItem`.
Operator Node	Any *Node* that is not an *Operand Node*. This covers the *Operator* classes proper, the parenthesised expression `ParExpr` (§4.10), and the two clause classes `GroupingClause` and `AnalyticClause` (§4.19, §4.20). The "Kind" column of §4.1.2 names these three groups separately, for orientation; wherever this part says *Operator Node* it means the whole complement of *Operand Node*.
Datapoint Reference	One entry of a `VarID.data` array: a *Variable* identifier, the resolved coordinates of the cell holding it, and a *Position Index* component on each axis that varies within the *Operand* (§4.3).
Position Index	The `(x, y, z)` triple locating a *Datapoint Reference* on the row, column and sheet axes of its *Operand*. A component is present only on an axis that varies within the *Operand* (§2.4).
Module Parameter	A named execution-time value — a *Scalar* or a *Scalar Set* — declared in `parameters` (§3.9) and read by a `ParameterRef` *Node* (§4.8), which carries its fallback.
Resolved	Said of an *Operand* that carries its own complete coordinates, fallback value and interval setting, independent of any enclosing lexical context (§2.3).

Throughout this part, "*Variable*" means a DPM *Variable* as defined in the Information Model, identified by its `VariableID`. JSON member names are written in `code style`; DPM-ML table and column names are written in *italics*.

A cross-reference of the form §X.Y always denotes a section of **this** part. References to other parts always name the part, as in "Operators §7" or "Information Model §5.3".

5.2 2 Structure and Concepts

This chapter explains the shape of a *Script Document* and the five concepts needed to read one. It is expository; the normative field-by-field rules are in §3 and §4.

5.2.1 2.1 The Document at a Glance

A *Script Document* is a JSON object with exactly one member. Its name is the taxonomy module URI; its value is the *Module Object*.

```
{
  "http://www.eba.europa.eu/eu/fr/xbrl/crr/fws/corep/4.2/mod/corep_le": {

    "format_version": "1.0",                // — —  identification
    "module_code": "COREP_LE",
    "module_version": "3.3.0",
    "framework_code": "COREP",
    "dpm_release":   { "release": "4.3", "publication_date": "2026-06-28" },
    "dates":         { "from": "2026-03-31", "to": null },

    "operations":    { "EGDQ_0053": { … } },   // — —  the rules
    "preconditions":{ "p_395095":   { … } },

    "variables":                 { "5488018": "m" },   // — —  dictionaries
    "precondition_variables":    { "395095": "b" },
    "tables":          { "C_28.00": { "variables": {…}, "open_keys": {…} } },
    "parameters":      { "WAIVCONS_C10": "String" },

    "dependency_information": { … },          // — —  execution planning
    "dependency_modules":     { … }
  }
}
```

The fourteen members of the *Module Object* fall into four groups:

Group	Members	Purpose
Identification	`format_version`, `module_code`, `module_version`, `framework_code`, `dpm_release`, `dates`	What this document is, which revision of this specification it was written to, and when it applies.
Rules	`operations`, `preconditions`	The *Operations*, as resolved abstract syntax trees.
Dictionaries	`variables`, `precondition_variables`, `tables`, `parameters`	Type and structure information for everything the rules reference, and the *Module Parameters* they read.
Execution planning	`dependency_information`, `dependency_modules`	Which rules can run on a single instance, and what the others need.

5.2.2 2.2 Keying by Module URI

The document is keyed by the XBRL taxonomy module URI rather than by module code. This makes a *Script Document* addressable directly from a reported instance: an engine that has opened an instance knows its entry-point URI and can select the applicable document without consulting a code mapping. The redundant `module_code` and `module_version` members are retained for human use and for cross-checking.

The same URI serves as the key in `dependency_modules`, so cross-module references are expressed in exactly one namespace.

5.2.3 2.3 Concept 1 — Resolved *Operands*

In DPM-XL surface syntax a `with` clause factors out coordinates shared by several *Operands*:

```
with {tC_28.00, default: 0, interval: true}: {c040} >= {c050}
```

In a *Script Document* no such context exists. The `with` clause is **pushed down** into every leaf, and each `VarID` *Node* independently carries its table, its coordinates, its fallback value and its interval-arithmetics setting:


```
{ "class_name": "BinOp", "op": ">=",
  "left":  { "class_name": "VarID", "table": "C_28.00", "column": "040",
             "default": 0, "interval": true, "data": [ … ] },
  "right": { "class_name": "VarID", "table": "C_28.00", "column": "050",
             "default": 0, "interval": true, "data": [ … ] } }
```


The consequence is that **the abstract syntax tree has no scoping rules**. A *Node* can be evaluated, cached, relocated or reported on in isolation, and a Consumer needs no environment or symbol table.

5.2.4 2.4 Concept 2 — Range Expansion and *Position Indices*

Surface DPM-XL selects ranges: `{c070-190}`, `{r*}`, `{(r0010, r0015)}`. A *Script Document* contains no ranges. Every selection is expanded into an explicit list of *Datapoint References* in `VarID.data`.

Each entry of that list carries two different kinds of information, and keeping them apart is the whole of this concept:

- **Where the cell is** — its `row`, `column` and `sheet` **codes**. Every entry carries these, on every axis the table version has, whatever the selection happened to fix. They identify the cell.
- **Where the cell sits in the *Operand*** — its *Position Index*, the components `x`, `y` and `z`. These appear only on an axis that **varies** within the *Operand*. They are what aligns one *Operand* against another.

The `VarID` itself carries a third thing: the coordinate the *selection* pinned, if any. `{tTab1, r100, c*}` pins the row, so the `VarID` carries `row: "100"`. Whether each entry then repeats `row: "100"` is a Producer's choice — both conventions are in circulation (§4.3.2) — so a Consumer reads a cell's position from its entry and falls back to the `VarID` for an axis the entry omits.

Consider a table `Tab1` with rows 100/200/300, columns 100/200 and sheets A/B, whose cells hold the *Variables* shown:

Sheet A

	Column 100	Column 200
Row 100	1	4
Row 200	2	5
Row 300	3	6

Sheet B

	Column 100	Column 200
Row 100	7	10
Row 200	8	11
Row 300	9	12

The *Operand* `{tTab1, r100, c*, s*}` fixes the row and varies the column and sheet axes. It expands to a `VarID` carrying `row: "100"` once, and four entries indexed on `y` and `z`:


```
{ "class_name": "VarID", "table": "Tab1", "row": "100",
  "interval": true, "default": 0,
  "data": [
    { "y": 1, "z": 1, "row": "100", "column": "100", "sheet": "A", "datapoint": 1,
"operand_reference_id": … },
    { "y": 2, "z": 1, "row": "100", "column": "200", "sheet": "A", "datapoint": 4,
"operand_reference_id": … },
    { "y": 1, "z": 2, "row": "100", "column": "100", "sheet": "B", "datapoint": 7,
"operand_reference_id": … },
    { "y": 2, "z": 2, "row": "100", "column": "200", "sheet": "B", "datapoint": 10,
"operand_reference_id": … }
  ] }
```

`x` is absent from the entries because the row axis does not vary; `row` is present on them all the same.

> **Axis assignment**
>
> `x` indexes **rows**, `y` indexes **columns**, `z` indexes **sheets**. This is normative and matches the DPM-ML *OperandReference* table.

A wildcard that resolves to a single code produces a fourth case, and it is the one that catches implementations out: nothing is pinned onto the `VarID`, and no index is emitted either, because the axis does not vary. §4.3.2 specifies all four states; the practical rule is that a Consumer MUST read an entry's position from the entry, never from the `VarID`.

Positional matching

Position Indices are what make a resolved tree evaluable. When an *Operator* combines two *Operands* that each expand to several *Datapoint References*, it combines the entries **that share the same *Position Index***, and produces one result per index. The index is therefore the format's substitute for the *Key Components* that would align *Records* in the Information Model.

Given `{tTab1, r100, c*, s*} = 2 * ({tTab1, r200, c*, s*} + {tTab1, r300, c*, s*})`, the index `(y=1, z=1)` selects *Variable* 1 on the left and *Variables* 2 and 3 on the right, yielding `1 = 2 * (2 + 3)`; index `(y=2, z=2)` yields `10 = 2 * (11 + 12)`; and so on for four results in total.

Alignment is on the *axes* the two *Operands* share, not on the arity of their index sets. An *Operand* that varies over sheets alone combines quite happily with one that varies over columns **and** sheets: it joins on the sheet and broadcasts along the column, giving one result per column-and-sheet. An *Operand* with no varying axis at all — a single cell — broadcasts across everything. §4.1.5 gives the rule.

5.2.5 2.5 Concept 3 — Preconditions Are Operations

A precondition is an *Operation* like any other. DPM-ML classifies *Operations* by type (Information Model §5.1), and a *Precondition* is the type whose job is to decide whether one or more validations apply. Its **only** constraint is on the output: where a Validation may yield a *Scalar* Boolean **or** a *Recordset* of Booleans, a Precondition may yield only a *Scalar* Boolean — one verdict per instance, because a rule is either applied or not.

Everything else about it is unrestricted. **Any DPM-XL expression may be a precondition** provided it reduces to a scalar Boolean: it may select facts, aggregate them, compare them, apply clauses, read a *Module Parameter*. A precondition tree is subject to §4 in full, exactly as an *Operation* tree is.

`preconditions` is nonetheless a separate section, for two reasons that have nothing to do with the language:

- **The linkage is inverted relative to DPM-ML.** Instead of each *Operation* naming its precondition, each precondition lists the *Operations* it gates in `affected_operations`. One precondition therefore serves many *Operations* without duplication, and a Consumer evaluates each precondition once per instance rather than once per gated rule.
- **A false precondition means *skipped*, not *failed*.** The two sections drive different reporting outcomes, so keeping them apart keeps the outcome unambiguous.

Filing indicators do get their own dictionary, `precondition_variables`, and that separation *is* structural: DPM-ML types every *Variable* as `fact`, `filingindicator` or `key`, and a filing indicator is not a fact. The dictionaries are therefore disjoint by construction (§3.8). A precondition that selects facts types them in `variables` like anything else.

5.2.6 2.6 Concept 4 — Scope Attributes

In DPM-ML an *OperationVersion* applies in many scopes, each with its own `IsActive`, `Severity` and `FromSubmissionDate` (see Information Model §5.3 Operation Scope). A *Script Document* covers exactly one module version, so the scope for that module version is resolved and flattened onto the *Operation Object* as `severity` and `from_submission_date`. Inactive scopes are omitted entirely: **presence in a *Script Document* is the assertion that the *Operation* is active.**

The same *Operation* may therefore appear with `severity: "error"` in one module's document and `severity: "warning"` in another's. Severity is a property of the scope, never of the rule. Three values occur — `error`, `warning` and `info` — and all three are reportable outcomes (§3.10).

One case the flattening cannot express is an *Operation* whose severity depends on **which** of several mutually exclusive dependencies the reporter filed, because there the document's own module version appears in more than one active scope. §5.7.2 records the limitation.

5.2.7 2.7 Concept 5 — Instance Dependencies

`dependency_information` partitions the *Operations* into those evaluable from the instance at hand and those that are not:

- `intra_instance_validations` — evaluable against a single reported instance of this module version.
- `cross_instance_dependencies` — each entry names the module versions required, the *Operations* affected, and the reference-date window over which the dependency holds.

The two groups cover every *Operation* and never overlap (§5.3, rule V-14). An *Operation* missing from both is the failure mode §5.7.1 records, and leaves the rule unevaluable.

Two subtleties matter. Within one entry the required modules are **conjunctive**. They are alternatives only where `alternative_dependencies` declares their modules mutually exclusive, never by inference (§3.11.2). And an entry MAY name the document's own module at a different reference period — a self-dependency, which is how `time_shift` is declared, forwards as well as backwards (§3.11.3).

`dependency_modules` then supplies the table structures of those foreign modules, so that cross-module *Operands* remain resolvable without loading the foreign *Script Document*.

It is worth noting that dependencies work somehow similarly to preconditions, in the sense that if an instance for a dependency module is not provided, the validations with that dependency are skipped.

5.2.8 2.8 What Is Deliberately Absent

A *Script Document* is not a substitute for the DPM. It omits, by design:

- **Labels and descriptions.** No natural-language names for *Variables*, tables or rules. Reporting messages are the consumer's concern.
- **Dimensional composition.** A `datapoint` is an opaque *Variable* identifier; the *Property*/*Category*/*Context* composition behind it is not serialised.
- **Rendering.** Row, column and sheet *codes* are present as coordinates; header structure, ordering and presentation are not.
- **Item enumerations.** `Scalar` *Nodes* carry item QNames; the *SubCategories* they belong to are not serialised.
- ***Parameter* bindings.** `parameters` (§3.9) declares which *Module Parameters* exist and what type each has, and each `ParameterRef` carries the fallback for its own reference (§4.8.1). What the document cannot carry is the **binding**: the value itself comes from the execution environment, and an unbound *Parameter* falls back to its default or to null.

An engine that must produce dimension-aware error messages, or resolve a QName to its enumeration, still needs the taxonomy or the DPM. §5.5 records the extension points reserved for enlarging this boundary.

5.3 3 Document Sections

This chapter specifies each section of a *Script Document* normatively. Abstract syntax trees are specified separately in §4.

Throughout, "member" means a JSON object member. Unless a member is marked OPTIONAL, it MUST be present. A member marked *nullable* MAY have the JSON value `null`; a member marked OPTIONAL MUST be omitted rather than set to `null` when it carries no value, except where stated otherwise.

5.3.1 3.1 Serialisation and Root Object

A *Script Document* MUST be a JSON document as defined by RFC 8259, encoded in UTF-8 without a byte-order mark.

The root MUST be a JSON object with **exactly one** member. The member name MUST be the absolute URI of the XBRL taxonomy module entry point. The member value is the *Module Object* (§3.2).

Producers SHOULD name files `<module_code>-<module_version>.json` and MAY serve them with media type `application/json`.

Producers MAY emit whitespace-formatted JSON for legibility. Consumers MUST NOT depend on formatting, member order or key order anywhere in the document.

Every object this part specifies is closed. Where a section gives the members of an object, those are the only members that object may carry: a Producer MUST NOT add any other, and a Consumer MUST reject a document that does (§5.2, requirement 2). This holds uniformly, whether or not a rule of §5.3 restates it for a particular object — several do, because those objects are where the failure was seen in practice, not because the others are open.

Size

A document for a large module runs to several megabytes, dominated by expanded `data` arrays and by the `variables` dictionary. Producers SHOULD offer compressed transfer; the format compresses well, since both are highly repetitive.

5.3.2 3.2 Module Object

Member	Type	Notes
`format_version`	String	Version of this specification the document was generated against. §3.2.1.
`module_code`	String	DPM code of the module. MUST equal *ModuleVersion.Code*.
`module_version`	String	MUST equal *ModuleVersion.VersionNumber*.
`framework_code`	String	DPM code of the owning framework.
`dpm_release`	Object	§3.3
`dates`	Object	§3.4
`variables`	Object	§3.5
`tables`	Object	§3.6
`preconditions`	Object	§3.7
`precondition_variables`	Object	§3.8
`parameters`	Object	§3.9
`operations`	Object	§3.10
`dependency_information`	Object	§3.11
`dependency_modules`	Object	§3.12

All fourteen members are REQUIRED. `operations`, `preconditions`, `tables`, `precondition_variables`, `parameters`, `dependency_modules` and the arrays within `dependency_information` MAY be empty. `variables` MAY be empty too, but only where `tables` is empty as well — it is not chosen independently, but derived as the union of `tables[*].variables` (§3.5, rule V-8).

Consumers MUST reject a document containing an unrecognised member of the *Module Object*. Members are added only by a revision of this specification (§5.6).

3.2.1 `format_version`

`format_version` names the version of **this specification** that the document was generated against — not the DPM release, which `dpm_release` carries (§3.3), and not the module, which `module_version` carries.

It MUST be the `MAJOR.MINOR` version of this specification, as published: `"1.0"` for this version. A Producer MUST emit the version it actually generated against, and MUST NOT emit a version it does not implement in full.

```
"format_version": "1.0"
```

A Consumer MUST reject a document whose `format_version` names a MAJOR version it does not implement, and MAY reject one naming a higher MINOR version of a MAJOR version it does implement. It MUST NOT reject a document naming a *lower* MINOR version, since MINOR revisions are non-breaking (§5.6).

The member exists because the alternative was tried and failed. Without it, a document written to one revision and a malformed document are indistinguishable: a Consumer can report only an unrecognised or missing member, and cannot say which revision it was handed. §5.6 records the two documents in circulation that demonstrate this.

5.3.3 3.3 `dpm_release`

Provenance of the DPM content from which the document was generated.

Member	Type	Notes
`release`	String	DPM release code, e.g. `"4.3"`. MUST equal *Release.Code*.
`publication_date`	String	ISO 8601 calendar date, `YYYY-MM-DD`. Date the *Script Document* content was published.

`publication_date` is the publication date of this generated artefact, which MAY be later than the publication date of the DPM release itself. It is provenance only and MUST NOT affect evaluation.

Documents of the same framework are **not** necessarily generated from the same release. A module version need be regenerated only when the module version itself changes, so a set of documents held together may name several releases. Consumers MUST therefore treat `dpm_release` as a property of the individual document and MUST NOT assume that two documents they hold agree on it.

5.3.4 3.4 `dates`

The reference-period window of the module version: the *reference periods* the module version covers, which are the periods being reported about, not the dates on which data is submitted.

Member	Type	Notes
`from`	String	ISO 8601 date. First reference date covered. MUST equal *ModuleVersion.FromReferenceDate*.
`to`	String, *nullable*	Last reference date covered, inclusive. `null` means the version is open-ended.

If `to` is non-null it MUST be greater than or equal to `from`.

Consumers MUST NOT evaluate a *Script Document* against an instance whose reference date falls outside `[from, to]`.

> **`null` here, `""` in `dependency_information`**
>
> An open-ended window is spelled `null` in `dates.to` but as the **empty string** in `cross_instance_dependencies[*].to_reference_date` (§3.11). The two members mean the same thing and are encoded differently. Consumers MUST accept both spellings in the member where each occurs, and MUST treat the empty string as "no upper bound" rather than as a malformed date.

5.3.5 3.5 `variables`

The *Data Type* dictionary of the fact *Variables* of this module, consolidating the per-table dictionaries of §3.6.

- Member name: the `VariableID`, as the decimal representation of a positive integer in a JSON string.
- Member value: a *Data Type* code (below), as a String.

```
"variables": { "85015": "m", "85083": "b", "85141": "p" }
```

Code	*Data Type*	Code	*Data Type*
`i`	Integer	`m`	Monetary
`r`	Decimal	`p`	Percentage
`s`	String (non-empty)	`d`	Date
`es`	String (including empty)	`dt`	Date-time
`b`	Boolean	`u`	URI
`t`	True	`o`	Ordinals
`e`	Enumeration		

Codes MUST be drawn from this set, which is *DataType.Code* in DPM-ML. A Consumer MUST accept all thirteen; a document is not obliged to use any particular one.

`variables` MUST be exactly the union of the `variables` dictionaries of every entry of `tables` (§5.3, rule V-8). It is therefore keyed by table membership, not by use: it covers every *Variable* of every referenced table, including — typically in the large majority — *Variables* that no *Operation* in the document references.

It is the Consumer's authority for the *Data Type* of an *Operand*, which governs interval arithmetics, null treatment and the admissibility of each *Operator*. Every `datapoint` of a `VarID` whose `table` is a member name of `tables` MUST appear here (§5.3, rule V-9); `datapoints` of *Operands* selecting from foreign modules are typed in `dependency_modules` instead (§3.12).

The *Data Type* is a function of the `VariableID` alone. A *Variable* MAY occur in several tables, and where it does, every dictionary that mentions it MUST assign it the same *Data Type* (§5.3, rule V-26).

This invariant is what makes type integrity enforceable on the loading path. Under the EBA XBRL-CSV architecture a fact is reported as a *Variable* ID, its value, and any open-key values; there is no coordinate in the instance to key on. A Consumer therefore validates the type of an incoming fact with a single lookup on the `VariableID`, before knowing which table or *Operation* will consume it, and MAY treat `variables` as one flat dictionary rather than resolving per table.

5.3.6 3.6 `tables`

Structure of every table version referenced by any `VarID` in `operations`.

- Member name: the table version code, e.g. `"C_28.00"`.
- Member value: an object with the members below.

Member	Type	Notes
`variables`	Object	The *Variables* of this table, in the *Data Type* dictionary form of §3.5.
`open_keys`	Object	*Open Key* dictionary: *Key Component* code → *Data Type* code. MAY be empty.

```
"C_29.00": {
   "variables": { "85015": "m", "85014": "m", "85083": "b" },
   "open_keys": { "GCC": "s", "INC": "s" }
}
```

`variables` serves two purposes, and the second is the one that cannot be obtained elsewhere:

1. it repeats the *Data Type* of each *Variable*, which §3.5 also carries; and
2. it declares **table membership** — which *Variables* belong to this table.

Membership is recorded nowhere else in the document. Under the EBA XBRL-CSV architecture a reported instance is split into **one file per table**, so a Consumer uses this member to determine which *Variables* to load from each file. It MUST NOT attempt to derive membership by collecting the `table` and `datapoint` values of the `VarID` *Nodes* in `operations`. A table contains every *Variable* of its rendering, and only a fraction of those is typically referenced by any *Operation*; membership derived from the rules would omit the rest, and the Consumer would then fail to load data it is required to validate against.

A *Variable* MAY belong to more than one table (§3.5). Its *Data Type* is the same in every table that lists it.

`open_keys` declares the *Key Components* that are open in this table — those whose values are not fixed by the rendering but supplied by the reporter, producing a variable number of *Records*. They are the *Key Components* available to a `GroupingClause` (§4.19), to a `GetClauseOp` (§4.23) and to the condition of a `WhereClauseOp` (§4.21) or `SubClauseOp` (§4.22). Their values are *Data Type* codes drawn from the same set as §3.5; `e`, `s`, `i` and `d` occur.

> ⚠ **Codes are version-scoped**
>
> *Open Key* and *Dimension* codes are properties of the module version, not stable identifiers. The DPM remodelling introduced in release 4.2 renamed many of them with a `q` prefix: `COREP_LE` 3.1.0 groups by `GCC, INC` while 3.3.0 groups by `qGCC, qINC`. Consumers MUST resolve these codes against the `open_keys` of the document in hand and MUST NOT carry them across documents.

5.3.7 3.7 `preconditions`

The preconditions gating the *Operations* of this document. See §2.5.

- Member name: the precondition code. No format is imposed on it. A Producer MAY derive it from the tree — `"p_395095"`, `"p_395096_395097_395098"`, prefixing the `variable_id`s of a filing-indicator precondition, is one such convention — but a Consumer MUST treat the code as opaque and MUST NOT parse anything out of it.
- Member value:

Member	Type	Notes
`code`	String	MUST equal the member name.
`version_id`	Integer	Identifier of this precondition version within the content the Producer generated it from. See the note at §3.10.
`ast`	Object	Root *Node* of the precondition tree.
`affected_operations`	Array of String	Codes of the *Operations* this precondition gates. MUST be non-empty.

The following is `COREP_LE` 3.1.0's `p_395096_395097_395098` verbatim:


```
"p_395096_395097_395098": {
   "code": "p_395096_395097_395098",
   "version_id": 395096,
   "ast": { "class_name": "BinOp", "op": "and",
            "left":  { "class_name": "BinOp", "op": "and",
                       "left":  { "class_name": "PreconditionItem",
                                  "variable_id": 395096, "variable_code": "C_27.00" },
                       "right": { "class_name": "PreconditionItem",
                                  "variable_id": 395097, "variable_code": "C_28.00" } },
            "right": { "class_name": "PreconditionItem",
                       "variable_id": 395098, "variable_code": "C_29.00" } },
   "affected_operations": [ "EGDQ_0056", "EGDQ_0057" ]
}
```


3.7.1 What a precondition tree may contain

A precondition `ast` is an *Operation* tree and is governed by §4 in full. **Any *Node* class permitted in an *Operation* is permitted here**, and no *Node* class is reserved to preconditions or excluded from them. The encoding of a precondition is the encoding of an *Operation*; there is no second grammar of *Nodes* for this position.

What a precondition may *mean* is settled elsewhere: a *Precondition* is an *Operation* type that yields a `Scalar` **Boolean** — exactly one Boolean value per instance under validation — against a Validation's "*Scalar* Boolean value or a *Recordset* with *Scalar* values" (Information Model §5.1). That is a semantic constraint on the expression, not on its serialisation, and it holds of the surface syntax exactly as it holds here.

One consequence of it is structural, and is checked: the tree MUST vary on **no axis**, so `axes(idx(ast))` MUST be empty in the notation of §4.1.5 (§5.3, rule V-12). A tree that varies on an axis yields a *Recordset* of Booleans, and there would be no single verdict on which to gate. It is the **axes** that decide this, not the cardinality of the index set: a single-cell selection has one index whose components are all absent, so it is a *Scalar* and a perfectly good precondition operand — `{tC_26.00, r030, c010} > 0` is well formed — while a selection over a range is not, however few cells the range holds.

A precondition MAY therefore select facts, aggregate them, filter them and compare the result — `sum({tC_29.00, c050}) > 0` is a well-formed precondition. Where it does, its `VarID` *Nodes* are resolved and typed exactly as in an *Operation*: their `table` MUST be declared in `tables` or `dependency_modules[*].tables`, and their `datapoint`s MUST be typed in `variables` (§5.3, rules V-9 and V-10, which apply to `preconditions` as much as to `operations`).

> **In practice, preconditions are simpler than the format allows**
>
> Almost every precondition in circulation is a bare `PreconditionItem` or a conjunction of them, and an implementation built only for that shape will appear to work for as long as no author needs anything more. Since the encoding admits any *Operation* tree, an implementation that dispatches on `class_name` generically — as it already must for `operations` — gets the general case for free and will not be surprised later.

3.7.2 Linkage and evaluation

Every code in `affected_operations` MUST be a member name of `operations`. An *Operation* MAY be gated by more than one precondition; where it is, all of them MUST hold.

Consumers MUST evaluate a precondition before any *Operation* it gates, and MUST skip — not fail — every gated *Operation* when it evaluates to false. A precondition evaluating to null MUST be treated as false, so the gated *Operations* are skipped rather than failed.

A precondition is evaluated **once per instance**, not once per gated *Operation*: the inverted linkage exists precisely so that a Consumer need not re-evaluate it for each rule (§2.5).

5.3.8 3.8 `precondition_variables`

Data Type dictionary for the *filing-indicator Variables* referenced by `PreconditionItem` *Nodes* (§4.9), in the form of §3.5.

```
"precondition_variables": { "395095": "b", "395096": "b", "395097": "b" }
```

Values MUST be `"b"` (Boolean). Identifiers in this dictionary are drawn from a space disjoint from `variables` (§5.3, rule V-11) and MUST NOT be interpreted as fact *Variables*.

The disjointness is structural, not a convention of this format: DPM-ML assigns every *Variable* a type — `fact`, `filingindicator` or `key` — and a filing indicator is a distinct kind of object from a reported fact, with its own identifier. It is this that earns filing indicators a dictionary of their own; it is **not** a consequence of preconditions being restricted to them, which they are not (§3.7.1).

This dictionary therefore does **not** cover a precondition's other *Variables*. A precondition that selects facts types those in `variables` (§3.5), alongside every other fact *Variable* in the document.

5.3.9 3.9 `parameters`

The *Module Parameters* of this module version: named values supplied to the engine at execution time rather than reported in the instance, read by `ParameterRef` *Nodes* (§4.8). They are the serialised counterpart of the Parameter Selection, Information Model §3.1, which is authoritative for their semantics.

- Member name: the *Parameter* code, e.g. `"BM"`.
- Member value: the declared *Data Type*, as a String, from the vocabulary below.

```
"parameters": {
  "BM": "String",
  "FINANCIAL_YEAR_END_MONTH": "Integer",
  "SUBJECT_CONSOLIDATED": "Boolean"
}
```

3.9.1 Declared *Data Types*

Information Model §3.1.1 defines twelve *Parameter* types, six *Scalar* and six *Set*. This part spells each as the surface type word with its initial letter capitalised, and *Set* followed by the capitalised scalar type name:

Surface syntax	`param_type`	Surface syntax	`param_type`
`number`	`"Number"`	`set-number`	`"SetNumber"`
`integer`	`"Integer"`	`set-integer`	`"SetInteger"`
`string`	`"String"`	`set-string`	`"SetString"`
`date`	`"Date"`	`set-date`	`"SetDate"`
`boolean`	`"Boolean"`	`set-boolean`	`"SetBoolean"`
`item`	`"Item"`	`set-item`	`"SetItem"`

The mapping is mechanical, so a Producer and a Consumer arrive at the same spelling independently. It is also the convention `Constant.type_` follows (§4.4), which keeps one vocabulary across the format.

A *Set*-typed *Parameter* takes a set of values of the corresponding *Scalar* type and is usable where a *Scalar Set* is expected — as the right-hand side of `in` (§4.12), for instance — but not where a *Scalar* is. §4.8.1 gives the `default` encoding for all twelve.

3.9.2 Values, defaults and unbound *Parameters*

`parameters` declares that a *Parameter* exists and what type it has. It carries **no value**: a binding is supplied by the execution environment alongside the data instances, and how it is delivered is outside the scope of this part (Information Model §3.1.4).

A **default** is carried per reference, on the `ParameterRef` *Node* (§4.8), not on this declaration — the surface syntax likewise attaches it to the selection, `{p_threshold, number, default: 0}`. Resolution is therefore three-stepped, and a Consumer MUST apply it in this order (Information Model §3.1.2):

1. the value bound by the execution environment for that *Parameter*, if one was supplied;
2. otherwise the `default` of the `ParameterRef` being evaluated, if it is non-null;
3. otherwise **null**.

Step 3 is not an error. An unbound *Parameter* with no default evaluates to null, and the *Operation* proceeds under the null treatment of Information Model §4 like any other null. A Consumer MUST NOT set the *Operation* aside on that ground, and MUST NOT refuse to load a document because a *Parameter* is unbound.

Because the default belongs to the reference, two `ParameterRef` *Nodes* reading the same *Parameter* MAY declare different defaults, and a Consumer MUST honour each where its own reference is evaluated. Producers SHOULD nonetheless keep them consistent within a document, since a *Parameter* that falls back to different values in different rules is difficult to reason about.

Every `ParameterRef.code` in the document MUST be a member name here, and its `param_type` MUST equal the type declared here (§5.3, rule V-29). A Producer SHOULD NOT declare a *Parameter* that no `ParameterRef` reads.

3.9.3 What a *Parameter* is for

A *Parameter* is a property of the reporting entity, of the submission, or of the supervisory regime — not of the reported data. It lets one *Operation* serve executions that differ only in a threshold, a date, a permitted set of codes or a configuration flag. Three kinds arise in practice:

- **Facts about the institution** — its business model, the month its financial year ends, whether it reports on a consolidated basis, its systemic designation.
- **Supervisory permissions and waivers** — which approach to operational risk the institution is authorised to use, whether a reporting waiver applies.
- **Externally set rates and thresholds** — the countercyclical buffer rate, for instance.

`parameters` MAY be empty, and is empty for a module whose *Operations* read no *Parameter*.

5.3.10 3.10 `operations`

The *Operations* of this document.

- Member name: the *Operation* code, e.g. `"EGDQ_0648"`.
- Member value, the *Operation Object*:

Member	Type	Notes
`code`	String	MUST equal the member name.
`version_id`	Integer	Identifier of this *Operation* version within the content the Producer generated it from. See the note below.
`expression`	String	The DPM-XL source expression. Informative; see §4.1.4.
`root_operator_id`	Integer	*OperatorID* of the root *Node*. Redundant; see §4.2.
`ast`	Object	Root *Node* of the tree (§4).
`severity`	String	`"error"`, `"warning"` or `"info"`.
`from_submission_date`	String	ISO 8601 date. The first *submission* date from which this *Operation* applies, which is a different clock from the reference-period window of `dates` (§3.4): a rule added late in a module version's life applies only to submissions from this date onward, including submissions about earlier reference periods. See §2.6.

```
"EGDQ_0052_0": {
   "version_id": 2071,
   "code": "EGDQ_0052_0",
   "expression": "with {r0010-0110, default: 0}: if {tF_32.04.a, c0010} > 0 then {tF_32.04.b,
c0030} > 0 endif",
   "root_operator_id": 30,
   "ast": { "class_name": "CondExpr", "condition": { … }, "then_expr": { … } },
   "from_submission_date": "2026-03-31",
   "severity": "error"
}
```

Identifiers are only as good as the shared provenance

`code` and `version_id` identify the *Operation* within the content the Producer generated it from, which need not be — and in general is not — the validation set of a DPM database. A Consumer MUST therefore treat both members as identifiers local to the document and its generation line, and MUST NOT join on them against a DPM database. The structural members can be checked that way; the rule identifiers cannot. See §5.7.2.

How a Producer obtains the value is outside the scope of this part: it may be an *OperationVID* carried over from a DPM database, or a number derived by other means where there is no database to carry one over from. The format requires only that it is an Integer, and that it is stable for a given *Operation* version within one generation line. It is precisely because the value may or may not be a DPM identifier, while always looking like one, that joining on it is forbidden.

No format is imposed on the *Operation* code. Codes in circulation include `EGDQ_0052_0`, `EGDQ_0071a` and `EGDQ_0928_P04.01`, and also bare decimal numbers such as `"42"`, so no lexical pattern may be assumed. Consumers MUST treat the code as an opaque String and MUST NOT parse it.

Every *Operation* in this section is a validation and is active for this module version (§2.6). The root *Node* MUST be an *Operator Node*, and the *Operation* MUST yield Boolean (a *Scalar* Boolean, or a *Recordset* of Boolean values where the tree expands to several *Position Indices*).

`severity` applies to every failing result of the *Operation*. Where the tree expands to several *Position Indices*, each index yields an independent result and each failing index is reported independently, all at the same severity.

The three severities are `error`, `warning` and `info`, in decreasing order of consequence, and a Consumer MUST implement all three. `info` is a reportable observation, not a defect; a Consumer MUST distinguish it from `warning` rather than folding the two together, and MUST NOT treat any severity as grounds for rejecting a submission on its own.

Consumers MUST NOT evaluate an *Operation* against a submission dated before `from_submission_date`.

5.3.11 3.11 `dependency_information`

Member	Type	Presence	Notes
`intra_instance_validations`	Array of String	REQUIRED	*Operation* codes evaluable from a single instance of this module version.
`cross_instance_dependencies`	Array of Object	REQUIRED	Dependency groups, below. MAY be empty.
`alternative_dependencies`	Array of Array of String	REQUIRED	Groups of interchangeable modules (§3.11.1). MAY be empty.

All three members are REQUIRED. `alternative_dependencies` MUST be emitted as an empty array where no alternatives exist, not omitted — unlike an OPTIONAL member, whose absence would carry meaning (§4.1.3).

Consumers MUST reject a document containing an unrecognised member of `dependency_information` (§5.3, rule V-27).

Each entry of `cross_instance_dependencies`:

Member	Type	Notes
`modules`	Array of Object	The foreign module versions required. MUST be non-empty.
`affected_operations`	Array of String	*Operation* codes requiring them. MUST be non-empty.
`from_reference_date`	String	ISO 8601 date. First reference date the dependency holds for.
`to_reference_date`	String	Last reference date, inclusive; the **empty string** if open-ended (§3.4).

Each entry of `modules`:

Member	Type	Notes
`URI`	String	Taxonomy module URI. MUST be either a member name of `dependency_modules` or the root URI of this document (§3.11.3).
`module_version`	String	Version of that module.
`ref_period`	String	Reference period of the required instance, relative to the instance under validation.

```
"cross_instance_dependencies": [ {
   "modules": [ { "URI": "http://www.eba.europa.eu/.../mod/corep_of",
                  "module_version": "3.1.0", "ref_period": "T" } ],
   "affected_operations": [ "EGDQ_0670", "EGDQ_0671", "EGDQ_0672", "EGDQ_0786" ],
   "from_reference_date": "2023-06-30",
   "to_reference_date": "2025-03-30"
} ]
```

`ref_period` MUST match `T`, `T-<n><p>` or `T+<n><p>`, where `<n>` is a positive integer and `<p>` is a period indicator as defined for `TimeShiftOp` (§4.27). `T` denotes the same reference period as the instance under validation; `T-1A` denotes one year earlier, `T+1Q` one quarter later.

A shifted `ref_period` occurs only on a self-dependency. Where the entry names another module version, `ref_period` MUST be `T`: an *Operation* compares this instance against another module's instance for the *same* reference period, and no *Operator* selects from a foreign module at a shifted period. The shifted forms belong to §3.11.3, where `TimeShiftOp` gives them their use; a Consumer MUST accept a shift in either direction there and MUST NOT assume the backward form.

Forward shifts are not a modelling error. `COREP_LR` declares a `T+1Q` self-dependency because a leverage-ratio figure is checked against the *following* quarter's submission; the *Operation* simply cannot be evaluated until that instance exists.

`intra_instance_validations` and `cross_instance_dependencies` MUST cover `operations` exhaustively and MUST NOT overlap (§5.3, rule V-14): every *Operation* appears in one member or the other, and none in both. Documents violating this are in circulation, and §5.7.1 records the failure mode — a cross-module *Operation* whose dependency entry, and hence whose foreign table structure, was never emitted.

3.11.1 `alternative_dependencies`

Each element is a group of two or more taxonomy module URIs that are **mutually exclusive** sources for the same data: a reporter files **one of them and never more than one**, and a dependency naming any member of the group is satisfied by whichever member was filed.

```
"alternative_dependencies": [
   [ "http://www.eba.europa.eu/eu/fr/xbrl/crr/fws/finrep/4.2/mod/finrep9",
     "http://www.eba.europa.eu/eu/fr/xbrl/crr/fws/finrep/4.2.1/mod/finrep9dp" ]
]
```

Why a group arises. The same table exists in more than one module. Where an *Operation* needs such a table, the module that supplies it is not determined by the rule but by which of those modules the reporter files, so the dependency must name every candidate. In `AE` 1.4.0 all seventeen required tables — `F_01.01`, `F_04.01`, and so on — exist in both `finrep9` 3.3.0 and `finrep9dp` 1.1.0, with identical *Variable* sets in each. `alternative_dependencies` records that those two modules are the alternatives.

Every URI in every group MUST be a member name of `dependency_modules`, and a group MUST have at least two members (§5.3, rule V-28). A document MAY declare several groups, and a group MAY have more than two members, though a pair is the usual case.

3.11.2 Conjunction, and Declared Alternation

Three levels of logic apply, and the distinction is normative:

- The `modules` of a **single entry** are **conjunctive**. Every module version listed MUST be available for the *Operation* to be evaluated.
- **Several entries** naming the same *Operation* are **conjunctive by default**: each entry states a separate requirement, and all of them MUST be satisfied.
- Entries whose modules belong to one `alternative_dependencies` group are **mutually exclusive alternatives**: exactly one of them can ever be satisfied, and satisfying that one satisfies the requirement.

Alternation is **never inferred**. Consumers MUST NOT treat repeated `affected_operations` across entries as evidence of it; a genuine conjunction wrongly read as alternation causes an *Operation* to be evaluated against incomplete data.

In `AE` 1.4.0, eleven *Operations* appear in two entries — one naming `finrep9` 3.3.0, the other `finrep9dp` 1.1.0 — and `alternative_dependencies` declares those two URIs a group. Requiring both would demand that an institution file both FINREP variants, which no reporter does.

A Consumer MUST evaluate an *Operation* listed in `cross_instance_dependencies` when, for **every** entry naming it whose window contains the instance's reference date, each module of that entry's `modules` is available for the required `ref_period` — **counting a module as available when any member of its `alternative_dependencies` group is available**. Otherwise the *Operation* is not evaluable against the submission in hand, and a Consumer MUST NOT evaluate it. How that is then reported is the Consumer's own concern (§1.1); what this part requires is that the absent data is not silently treated as null and the *Operation* thereby passed or failed.

Because the members of a group are mutually exclusive, a Consumer SHOULD expect exactly one of them and MAY treat the presence of two as an error in the submission rather than in the *Script Document*.

An *Operation* MUST be evaluated once regardless of how many entries name it. Which instance supplies a given *Operand* is determined by the `table` of the `VarID` (§4.3), not by entry order; where alternatives are present, by whichever member of the group was filed. Where the alternatives declare the same tables with the same *Variables*, as in `AE` 1.4.0, the loading path is identical whichever was filed.

> ⚠ **Severity may depend on which alternative was filed**
>
> The alternatives are interchangeable as sources of *data*, but not necessarily as *scopes*. DPM-ML attaches severity to a scope covering a *combination* of module versions, so an *Operation* may be an error against one alternative and a warning against another. This document carries a single `severity` per *Operation* (§3.10) and cannot express the difference — a limitation that falls on the module which *depends* on the group, not on the alternatives themselves. See §5.7.2.

3.11.3 Self-Dependencies

An entry of `modules` whose `URI` equals the root URI of the document (§3.1) is a **self-dependency**: the *Operation* requires another reference period of the *same* module version. Such an entry MUST have a `ref_period` other than `T`, and MUST NOT appear in `dependency_modules`, whose purpose is to describe *foreign* structure that the document does not already carry.

Self-dependencies are how `TimeShiftOp` (§4.27) is declared. An *Operation* comparing a figure against its value one year earlier produces an entry such as:

```
{ "modules": [ { "URI": "http://www.eba.europa.eu/.../mod/finrep9",
                 "module_version": "3.3.0", "ref_period": "T-1A" } ],
  "affected_operations": [ "EGDQ_1226", "EGDQ_1245" ],
  "from_reference_date": "2026-03-31",
  "to_reference_date": "2027-09-29" }
```

The `T-1A` self-dependency is the declarative counterpart of the `TimeShiftOp` in the tree of each affected *Operation*.

A shifted `ref_period` is confined to this case: every entry naming another module version carries `T` (§3.11, rule V-16). A temporal shift is something an *Operation* does to its own module's data, so there is no construct that would produce a shifted foreign entry.

5.3.12 3.12 `dependency_modules`

Structural information for the foreign modules named in `cross_instance_dependencies`.

- Member name: the taxonomy module URI of the foreign module.
- Member value:

Member	Type	Notes
`tables`	Object	Table dictionary, in the form of §3.6, restricted to tables **of the foreign module** that cross-module *Operands* reference.
`variables`	Object	*Data Type* dictionary, in the form of §3.5. **See the note below.**

```
"dependency_modules": {
  "http://www.eba.europa.eu/.../mod/corep_of": {
    "tables":     { "C_01.00": { "variables": { "84817": "m" }, "open_keys": {} } },
    "variables": { "84817": "m", "32026": "m", "32024": "m", "32025": "p" }
  }
}
```

Both members are REQUIRED.

> ⚠ **`variables` is not restricted to the foreign module**
>
> Contrary to what the nesting suggests, `variables` here lists the *Data Types* of **all** *Variables* referenced by the *Operations* that depend on this foreign module — those of the local module included. In the example above only `84817` belongs to `C_01.00` in `COREP_OF`; the rest are *Variables* of the local module's own tables. An entry will in general carry identifiers that its own `tables` do not, and a large share of them will be local. The member is a convenience dictionary for evaluating the affected *Operations*, not a description of the foreign module.
>
> Consumers MUST NOT infer module membership from this member; `tables` is the authority for foreign structure. Producers MUST populate it as specified so that the union of `variables` and each entry's `tables[*].variables` covers every `datapoint` reachable from the affected *Operations*.

Every member name of `dependency_modules` MUST occur as some `modules[*].URI`, and every `modules[*].URI` other than the root URI MUST be a member name here (§5.3, rule V-16). The two sides of the correspondence are therefore always in step: a Consumer that finds a `URI` it cannot resolve, or an entry nothing refers to, has a malformed document.

5.4 4 Abstract Syntax Tree Reference

5.4.1 4.1 Common Rules

4.1.1 The `class_name` Discriminator

Every *Node* MUST be a JSON object carrying a `class_name` member whose value is a String drawn from the closed set in §4.1.2. `class_name` is the sole discriminator: a Consumer MUST determine a *Node*'s interpretation from `class_name` alone, and MUST reject a document containing an unrecognised value.

A *Node* MUST NOT carry any member other than those specified for its class.

4.1.2 Node Inventory

Twenty-five classes are defined. The *OperatorID* column gives the DPM-ML *Operator* identifiers the class encodes; `—` means the class encodes no DPM-ML *Operator* (see §4.2).

`class_name`	Kind	*OperatorID*	Section
`VarID`	*Operand* — resolved data selection	—	§4.3
`Constant`	*Operand* — literal	—	§4.4
`Scalar`	*Operand* — DPM item reference	—	§4.5
`Set`	*Operand* — set of items	—	§4.6
`Dimension`	*Operand* — *Component* reference	—	§4.7
`ParameterRef`	*Operand* — *Module Parameter* reference	—	§4.8
`PreconditionItem`	*Operand* — filing indicator	—	§4.9
`ParExpr`	*Operator* — parenthesised expression	37	§4.10
`UnaryOp`	*Operator* — one operand	1, 4, 6, 17, 24	§4.11
`BinOp`	*Operator* — two operands	2, 3, 5, 8, 13–16, 18–23	§4.12
`ComplexNumericOp`	*Operator* — n-ary numeric	7, 9	§4.13
`SetOp`	*Operator* — set construction and algebra	—	§4.14
`SubstrOp`	*Operator* — substring extraction	—	§4.15
`DateConstructorOp`	*Operator* — date construction	—	§4.16
`AnnualiseOp`	*Operator* — `annualise`	—	§4.17
`AggregationOp`	*Operator* — aggregation	11, 12, 26, 27	§4.18
`GroupingClause`	Clause — grouping specification	35	§4.19
`AnalyticClause`	Clause — analytic window specification	—	§4.20
`WhereClauseOp`	*Operator* — `where` clause	28	§4.21
`SubClauseOp`	*Operator* — `sub` clause	38	§4.22
`GetClauseOp`	*Operator* — `get` clause	29	§4.23
`RenameClauseOp`	*Operator* — `rename` clause	33	§4.24
`FilterOp`	*Operator* — `filter`	31	§4.25
`CondExpr`	*Operator* — `if-then-else`	30	§4.26
`TimeShiftOp`	*Operator* — `time_shift`	32	§4.27

The set is closed. A Consumer MUST implement every class in it, and MUST reject a `class_name` outside it (§4.1.1).

4.1.3 Absent Members Versus `null`

The two conventions carry different meanings and MUST both be honoured:

- Where a member is marked **OPTIONAL**, absence is significant and the member MUST be omitted rather than set to `null`. This applies to `VarID.default`, to the axis members of `VarID` and its entries, and to `CondExpr.else_expr`.
- Where a member is marked **nullable**, it MUST be present, with `null` where it carries no value. This applies to `AggregationOp.grouping_clause`, `AggregationOp.analytic_clause`, `AnalyticClause.window` and `ParameterRef.default`.

The asymmetry is normative. Consumers MUST NOT treat an absent nullable member as `null`, nor an explicit `null` as an absent OPTIONAL member.

`CondExpr.else_expr` is the member most likely to be mishandled, because it is the one place where a *semantically* meaningful absence is encoded by omission rather than by `null`. A one-armed conditional is the common case, so a Consumer that treats the member as nullable will misread ordinary documents (§4.26).

4.1.4 Status of `expression`

`expression` (§3.10) is a DPM-XL expression that MUST be semantically equivalent to `ast`.

It is nonetheless **informative**. Consumers MUST NOT parse it, and MUST NOT derive any part of the evaluation from it. Where `expression` and `ast` disagree, `ast` prevails. Its purpose is human review, traceability to the DPM-ML *Expression* column, and diagnostics.

4.1.5 *Position Index* Algebra

Let `idx(N)` denote the set of *Position Indices* produced by evaluating *Node* `N` (§2.4). Then:

Node	`idx`
`VarID`	The set of `(x, y, z)` triples of its `data` entries.
`Constant`, `Scalar`, `Set`, `Dimension`, `ParameterRef`, `PreconditionItem`	∅ — *Scalar*; broadcast over any index set.
`ParExpr`, `UnaryOp`, `SubstrOp`	`idx` of the operand.
`BinOp`, `ComplexNumericOp`, `CondExpr`	The combined index set, per the rule below.
`AggregationOp`	∅ when both clauses are `null`; with a `GroupingClause`, one index per distinct group; with an `AnalyticClause`, `idx` of the operand.
`WhereClauseOp`, `SubClauseOp`, `FilterOp`	`idx` of the operand or selection; membership is narrowed, indices are preserved.
`GetClauseOp`	`idx` of the operand; the *Component* value replaces the *Fact*, positions are unchanged.
`SetOp`	∅ — the result is a *Scalar Set*, which carries no *Components* and no positions.
`RenameClauseOp`, `TimeShiftOp`	`idx` of the operand.

Combination rule. *Position Indices* stand in for the *Key Components* that align *Records* in the Information Model (§2.4), so operands combine the way *Recordsets* do: by **natural join on the axes they share**. An axis present in one operand and absent from another is not a mismatch — the operand lacking it broadcasts along it.

Write `axes(S)` for the set of axes on which the indices of `S` carry a component, and `S↓A` for the projection of `S` onto the axes `A`. For an *Operator* over operands $N_1 \dots N_n$, let $S = \{ idx(N_i) : idx(N_i) \neq \varnothing \}$.

1. If `S` is empty, the result is a *Scalar*.
2. Otherwise there MUST be a member `M` of `S` with $axes(T) \subseteq axes(M)$ for every `T` in `S`. `M` is the widest operand, and the result carries $idx = M$.
3. Every `T` in `S` MUST satisfy $M{\downarrow}\,axes(T) = T{\downarrow}\,axes(T)$ — on the axes it does have, it must range over exactly the positions the widest operand ranges over.
4. An *Operator* meeting 2 and 3 is combined by joining each operand's *Record* to every index of `M` that agrees with it on $axes(T)$, producing one result per index of `M`.
5. Otherwise the *Operator* is **ill-formed**. Producers MUST NOT emit it, and rule V-17 rejects it. A Consumer MUST NOT evaluate it over the intersection, the union, or a positional truncation of the operand index sets: an ill-formed *Operator* has no result, and inventing one silently converts a document defect into a validation outcome.

   Two familiar shapes are special cases and need no separate rule:

   - **Broadcast of a single value.** An operand whose one index is `(-, -, -)` has $axes = \varnothing$, so condition 3 is vacuous and it broadcasts across every index of `M`. This is by far the commonest departure from equal index sets.
   - **Equal index sets.** Where every operand has the same axes, condition 3 reduces to equality, and the *Operator* combines index by index.

   The general case does occur. In `EGDQ_0379` of `COREP_OF` 3.1.0,

```
with {tC_14.01, s*}: if isnull({{c0460, c0470}}) then {c0440, default: 0} <= 12.5 *
sum({{c0310, … , c0420), default: 0}) endif
```

   the condition ranges over two columns and six sheets — twelve indices on `(y, z)` — while `{c0440}` ranges over the six sheets alone, six indices on `(z)`. Their `z` sets are identical, so the six-index operand joins on `z` and broadcasts along `y`, and the *Operation* yields twelve results: for each sheet, one check per column tested. The `sum` is a full reduction and contributes no index at all.

   Reading the index as an opaque triple instead of as a set of axis components would reject this *Operation* as a cardinality mismatch. That reading is wrong: it is the axes, not the arity, that must line up.

   Case 5 is what a genuine mismatch looks like — two ranges over the *same* axis of unequal extent, such as a ten-row range compared against an explicit twelve-row list. Silently truncating is the principal way two engines can disagree while both appearing to succeed, which is why it is prohibited rather than defined.

   Index identity. Two indices are equal when their `x`, `y` and `z` components are pairwise equal, an absent component being equal only to an absent component. An index component MUST be a positive integer, and indices within one `VarID` MUST be distinct.

5.4.2 4.2 Operator Encoding Map

The DPM-ML *Operator* table has 38 rows, identical in every release checked (4.2.1, 4.3 and 4.4). The table below maps each to its encoding. It is the bridge between the Operators part, the DPM-ML *Operator* table and this part.

ID	Symbol	Category	Encoding	Semantics
1	`+`	Numeric	`UnaryOp op: "+"`	Operators §4
2	`+`	Numeric	`BinOp op: "+"`	Operators §4
3	`/`	Numeric	`BinOp op: "/"`	Operators §4
4	`-`	Numeric	`UnaryOp op: "-"`	Operators §4
5	`-`	Numeric	`BinOp op: "-"`	Operators §4
6	`abs`	Numeric	`UnaryOp op: "abs"`	Operators §4
7	`min`	Numeric	`ComplexNumericOp op: "min"`	Operators §4
8	`*`	Numeric	`BinOp op: "*"`	Operators §4
9	`max`	Numeric	`ComplexNumericOp op: "max"`	Operators §4
10	`sqrt`	Numeric	`UnaryOp op: "sqrt"`	Operators §4.2.6
11	`max_aggr`	Aggregate	`AggregationOp op: "max_aggr"`	Operators §7
12	`min_aggr`	Aggregate	`AggregationOp op: "min_aggr"`	Operators §7
13	`=`	Comparison	`BinOp op: "="`	Operators §5
14	`<=`	Comparison	`BinOp op: "<="`	Operators §5
15	`>=`	Comparison	`BinOp op: ">="`	Operators §5
16	`in`	Comparison	`BinOp op: "in"`	Operators §13
17	`isnull`	Comparison	`UnaryOp op: "isnull"`	Operators §5
18	`>`	Comparison	`BinOp op: ">"`	Operators §5
19	`<`	Comparison	`BinOp op: "<"`	Operators §5
20	`!=`	Comparison	`BinOp op: "!="`	Operators §5
21	`match`	Comparison	`BinOp op: "match"`	Operators §5.9
22	`and`	Logical	`BinOp op: "and"`	Operators §6
23	`or`	Logical	`BinOp op: "or"`	Operators §6
24	`not`	Logical	`UnaryOp op: "not"`	Operators §6
25	`xor`	Logical	`BinOp op: "xor"`	Operators §6.3
26	`sum`	Aggregate	`AggregationOp op: "sum"`	Operators §7
27	`count`	Aggregate	`AggregationOp op: "count"`	Operators §7
28	`where`	Clause	`WhereClauseOp`	Operators §11
29	`get`	Clause	`GetClauseOp`	Operators §11
30		Conditional	`CondExpr`	Operators §8

ID	Symbol	Category	Encoding	Semantics
31	`filter`	Conditional	`FilterOp`	Operators §8
32	`time_shift`	Time	`TimeShiftOp`	Operators §10
33, 34	`rename`, `node`	Clause	`RenameClauseOp`	Operators §11
35	`group by`	Clause	`GroupingClause`	Operators §7
36	`<-`	Assignment	**Unassigned** — §5.4 (Calculations)	Operators §12
37	`()`	Logical	`ParExpr`	—
38	`sub`	Clause	`SubClauseOp`	Operators §11

4.2.1 *Operators* With No *OperatorID*

The DPM-ML *Operator* table is narrower than the language. The Operators part specifies a further twenty *Operators* that the table does not list, and each has an encoding here; a Consumer MUST implement them all:

`op`	Encoding	Semantics
`exp`	`UnaryOp` `op: "exp"`	Operators §4.2.4 — exponential.
`ln`	`UnaryOp` `op: "ln"`	Operators §4.2.5 — natural logarithm.
`log`	`BinOp` `op: "log"`	Operators §4.2.7 — logarithm in a given base.
`power`	`BinOp` `op: "power"`	Operators §4.2 — exponentiation.
`nvl`	`BinOp` `op: "nvl"`	Operators §8 — substitutes `right` where `left` is null.
`&`	`BinOp` `op: "&"`	Operators §9.2 — string concatenation.
`len`	`UnaryOp` `op: "len"`	Operators §9.1 — string length.
`substr`	`SubstrOp`	§4.15 — substring by start and length.
`year`, `semester`, `quarter`, `month`, `week`, `day`	`UnaryOp` with that `op`	Operators §10.2 — date-component extraction.
`date`	`DateConstructorOp`	§4.16 — a Date from three Integers.
`annualise`	`AnnualiseOp`	§4.17 — scale to a full-year equivalent.
`avg`	`AggregationOp` `op: "avg"`	Operators §7.6 — arithmetic mean.
`median`	`AggregationOp` `op: "median"`	Operators §7.7 — median.
`rank`	`AggregationOp` `op: "rank"`	§4.18, §4.20 — analytic rank over an ordered window.
`set_of`	`SetOp` `op: "set_of"`	Operators §13.2 — the set of an operand's values.
`union`	`SetOp` `op: "union"`	Operators §13.3 — set union.
`intersect`	`SetOp` `op: "intersect"`	Operators §13.4 — set intersection.
`setdiff`	`SetOp` `op: "setdiff"`	Operators §13.5 — set difference.
`symdiff`	`SetOp` `op: "symdiff"`	Operators §13.6 — symmetric difference.

The divergence follows from how a *Script Document* tree is built: from the DPM-XL `expression`, not by walking DPM-ML *OperationNode* rows. The encodable vocabulary is therefore the language's, not the *Operator* table's. `rank` alone is specified nowhere but here, in §4.18 and §4.20.

A (`class_name`, `op`) pair is consequently the identity of an *Operator* in this format, and the *OperatorID* is a cross-reference available only for the subset the table covers.

Consequently `root_operator_id` is well-defined only where the root *Node* encodes a DPM-ML *Operator*. Where it does, it MUST equal that *OperatorID* (§5.3, rule V-4 checks this): it is fully derivable from `ast` and is retained as a checksum, so a Producer MUST NOT emit a document in which the two disagree. A Consumer needs neither to verify it nor to read it — the root *Node* carries its own `class_name`

and `op`, and those are what a Consumer MUST dispatch on. Because an *Operation* must yield Boolean (§3.10), the *Operators* that can appear at a root are in practice the comparisons, the logical *Operators* and `if-then-else`.

> **Categories differ from *Node* classes**
>
> The *Node* classes group *Operators* by arity and encoding shape, not by the categories used in the Operators part. `min` and `max` are `ComplexNumericOp` because they are n-ary; `+` appears as both `UnaryOp` and `BinOp`; `in` and `nvl` are `BinOp` although they are specified under Set and Numeric operators respectively. Consumers MUST dispatch on `class_name` and `op` jointly, never on `op` alone.

5.4.3 4.3 `VarID`

A resolved data selection. The *Operand* of the Selection *Operator*, with all ranges expanded and all context applied.

4.3.1 Fields

Member	Type	Presence	Notes
`table`	String	REQUIRED	Table version code. MUST be a member name of `tables` (§3.6) or of some `dependency_modules[*].tables` (§3.12).
`row`	String	OPTIONAL	Row code, when the selection fixes the row axis.
`column`	String	OPTIONAL	Column code, when the selection fixes the column axis.
`sheet`	String	OPTIONAL	Sheet code, when the selection fixes the sheet axis.
`interval`	Boolean	REQUIRED	Interval arithmetics setting.
`default`	any	OPTIONAL	Fallback value for nulls. A JSON integer, number, boolean or string — a Date written `YYYY-MM-DD`, an *Item* as its signature. Absent means none.
`data`	Array	REQUIRED	*Datapoint References*. MUST be non-empty.

Each entry of `data` is a *Datapoint Reference*:

Member	Type	Presence	Notes
`datapoint`	Integer	REQUIRED	`VariableID`. MUST be non-null and MUST appear in the applicable *Data Type* dictionary.
`operand_reference_id`	Integer	REQUIRED	Identifier of the resolved cell reference. See §4.3.4.
`row`	String	OPTIONAL	Row code of this cell, on every entry where the table has a row axis.
`column`	String	OPTIONAL	Column code of this cell, on every entry where the table has a column axis.
`sheet`	String	OPTIONAL	Sheet code of this cell, on every entry where the table has a sheet axis.
`x`	Integer	OPTIONAL	Row *Position Index*, when the row axis varies within the *Operand*.
`y`	Integer	OPTIONAL	Column *Position Index*, when the column axis varies.
`z`	Integer	OPTIONAL	Sheet *Position Index*, when the sheet axis varies.

4.3.2 Axis Placement

Coordinates and indices are placed by two independent rules. The distinction matters, and the two are easy to conflate:

- **A `VarID`-level coordinate records what the *selection* fixed**, after `with` push-down (§2.3). It is present only where the source expression pinned that axis to a single code.
- **An entry carries the resolved coordinate of its own cell** on every axis the table version has, *except* where that axis is already fixed at the `VarID` level, in which case repeating it is permitted but not required.
- **An index member is present exactly where the axis varies within the *Operand***, that is, where the entries carry more than one distinct code for it.

Formally, for each axis independently — `row`/`x`, `column`/`y`, `sheet`/`z` — a `VarID` MUST be in exactly one of these states:

State	Coordinate on `VarID`	Coordinate on entries	Index on entries	Distinct codes
Fixed by selection	present	on **every** entry or on **none**; where present, equal to it	absent from every entry	1
Fixed, not pinned	absent	present on **every** entry	absent from every entry	1
Varying	absent	present on **every** entry	present on **every** entry	$\geqslant 2$
Not applicable	absent	absent from every entry	absent from every entry	0

Partial placement MUST NOT occur: an index on some entries only, an index without its coordinate, or a `VarID`-level coordinate that some entry contradicts. At least one axis MUST be in a state other than Not applicable.

> **Producers disagree about the redundant coordinate**
>
> In the **Fixed by selection** state alone, the entry coordinate repeats a value already fixed one level up. Both conventions are in circulation and neither is a defect: one Producer repeats it on every entry, another omits it throughout. Since a Consumer must read the entry first in the other three states, the rule that covers both is simply: **take the coordinate from the entry, and where the entry does not carry it, from the `VarID`**. Earlier versions of this part required the repetition; requiring it would invalidate documents that are otherwise correct, so it is now permitted instead.

Within one document, whether an axis is Not applicable is a property of the table version, not of the individual *Operand*: every `VarID` over the same table agrees. Across documents it does not, because table versions differ.

The **Fixed, not pinned** state is what a wildcard produces when it happens to resolve to one code. `{tC_68.00.a, c0010, s*}` writes `s*`, so nothing is pinned onto the `VarID`; where the sheet axis of that table has a single sheet, the entries all carry `sheet: "0010"` and no `z`. It arises most often on the sheet axis, for the simple reason that single-sheet tables are common. Consumers MUST NOT treat the `VarID`-level coordinate as the authority for an entry's position — the entry is — and MUST NOT infer from its absence that the axis varies.

WORKED EXAMPLE

Take the *Operand* `{tC_17.01.a, (r0060, r0160, …), c0080}` — ten rows named explicitly, one column. The row axis varies and the column axis is fixed by the selection. Three of its ten entries:


```
{ "class_name": "VarID", "table": "C_17.01.a",
  "interval": true, "default": 0,
  "column": "0080",
  "data": [
    { "datapoint": 51416, "operand_reference_id": 50735, "x": 1, "row": "0060", "column":
"0080" },
    { "datapoint": 51421, "operand_reference_id": 50744, "x": 2, "row": "0160", "column":
"0080" },
    { "datapoint": 51420, "operand_reference_id": 50753, "x": 3, "row": "0260", "column":
```

```
"0080" }
  ] }
```

`column` appears twice over — once on the `VarID` because the selection fixed it, once on each entry because that is where the cell's own coordinates live. There is no `y`, because the column axis does not vary. `sheet` and `z` are absent throughout because `C_17.01.a` has no sheet axis.

4.3.3 Constraints

1. Indices within one `VarID` MUST be pairwise distinct (§4.1.5).
2. A `VarID` referencing a table of a foreign module MUST occur only within an *Operation* listed in a `cross_instance_dependencies` entry naming that module (§3.11). A foreign selection with no such entry cannot be resolved; see §5.7.1.
3. Where the *Data Types* of the `datapoints` of one `VarID` differ, the *Operators* applied to it must be admissible for all of them — a semantic condition, and one this part does not check.

   Which *Data Types* admit `interval`, and which values may serve as a `default`, are questions about what a selection means rather than about how it is written. They are settled where *Data Types* are defined, and this part neither restates nor validates them (§1.1).

   Both reference implementations resolve such a `VarID`'s *Data Type* per-*Record* rather than once for the whole *Recordset*, and give that per-*Record* state a dedicated internal type, `Mixed`, undocumented here but present in both (`dpmcore/src/dpmcore/dpm_xl/types/promotion.py`, `cve/Code/DataTypes/ScalarTypes.py:239`). Applying an *Operator* to a `Mixed` operand checks and promotes types row by row instead of once. Where the *Operator* has a fixed result type regardless of its operands (a comparison always yields Boolean), the result of the whole *Recordset* is that type; where it does not (arithmetic, `nvl`, ...), the result stays `Mixed`, carrying a per-*Record* type onward rather than collapsing to one. The practical consequence is that restriction 3 above is checked per *Record* rather than once for the whole selection — again a matter for the type system, noted here only because the encoding gives no hint that a `VarID` is of mixed type.

4.3.4 Semantics

The `VarID` denotes a *Recordset* containing, for each *Datapoint Reference*, the reported value of the identified *Variable*, positioned at its *Position Index*. Where the value is absent from the instance, `default` supplies the value if present; otherwise null treatment applies as specified in Information Model §4 Null handling. Where `interval` is `true`, values are treated as intervals per Operators §3.1.2.

`operand_reference_id` carries no evaluation semantics. It identifies a **resolved cell reference**: within one document the identifier MUST stand in a one-to-one correspondence with the tuple (`table`, `datapoint`, `row`, `column`, `sheet`), and every *Operand* that selects that cell repeats the same identifier (§5.3, rule V-15).

Two consequences follow, and both matter:

- The identifier is **not unique within the document**. A Consumer MUST NOT use it as a primary key for an *Operand* occurrence, and MUST NOT assume that two entries bearing it belong to the same `VarID`, the same *Operation*, or the same *Position Index* — the same cell may be indexed differently in different *Operands*. Repetition is the normal case, not the exception, since rules routinely test the same cell.
- It is **not a key into a DPM database**. The member is named after the DPM-ML *OperandReference* table, but DPM-ML assigns one identifier per operand *occurrence*, keyed on *NodeID*, whereas the values here are one per cell. The two numberings are unrelated, and because they occupy overlapping ranges a join will appear to succeed while returning the wrong row. A Consumer MUST NOT join on this member across the two artefacts. To check an entry against a DPM database, use its `table`, coordinates and `datapoint` (§5.7.2).

Within the document it is the RECOMMENDED identifier for reporting a failure against a specific cell.

The *Operand* `{tF_25.01.a, r0120, (c0030, c0040)}`, whose column axis varies over two codes:

```
{ "class_name": "VarID",
  "table": "F_25.01.a", "row": "0120", "default": 0, "interval": true,
  "data": [ { "datapoint": 433544, "operand_reference_id": 131100, "y": 1, "row": "0120",
"column": "0030" },
            { "datapoint": 433422, "operand_reference_id": 131109, "y": 2, "row": "0120",
"column": "0040" } ] }
```

5.4.4 4.4 `Constant`

A literal *Scalar*.

Member	Type	Presence	Notes
`type_`	String	REQUIRED	`"Integer"`, `"Number"`, `"Boolean"`, `"String"`, `"Date"` or `"Null"`.
`value`	Integer, Number, Boolean, String or null	REQUIRED	The literal.

The JSON type of `value` MUST correspond to `type_`: `"Integer"` with a JSON integer, `"Number"` with a JSON number, `"Boolean"` with a JSON boolean, `"String"` with a JSON string, `"Date"` with a JSON string holding an ISO 8601 calendar date, `YYYY-MM-DD`, and `"Null"` with a JSON `null`.

`type_` is spelled with a trailing underscore. This is normative.

A surface percentage literal has no `type_` of its own: `%` divides by 100 before this *Node* is built, so `5%` and the decimal literal `0.05` produce the identical `Constant`, with `type_: "Number"`. Nothing on the wire distinguishes a value that was written as a percentage from one that was always a plain decimal; where that distinction matters it is carried by the *Data Type* of the *Operand* the value feeds (`p`, Percentage, §3.5), never by `Constant` itself.

```
{ "class_name": "Constant", "type_": "Integer", "value": 1 }
{ "class_name": "Constant", "type_": "Date", "value": "2011-01-01" }
{ "class_name": "Constant", "type_": "Number", "value": 0.05 }
```

5.4.5 4.5 `Scalar`

A reference to a DPM item, used as the right-hand operand of a comparison inside a clause condition, and as an element of a `Set`.

Member	Type	Presence	Notes
`item`	String	REQUIRED	The item QName, in `prefix:localName` form, e.g. `"eba_GA:x1"`.
`scalar_type`	String	REQUIRED	`"Item"`.

`scalar_type` distinguishes the kind of DPM object referenced. `"Item"` is the only value this version of the specification assigns; §5.5 reserves the member for further kinds.

The QName is not resolved within the document: the enumeration it belongs to is not serialised (§2.8). Consumers requiring the enumeration MUST obtain it from the taxonomy or the DPM.

```
{ "class_name": "Scalar", "item": "eba_GA:x1", "scalar_type": "Item" }
```

5.4.6 4.6 `Set`

A set literal — the right-hand operand of `in`, or the empty set compared against a `SetOp` result.

Member	Type	Presence	Notes
`children`	Array	REQUIRED	Elements of the set: `Scalar` *Nodes* (§4.5) or `Constant` *Nodes* (§4.4), not both. MAY be empty.

Order is not significant. See Operators §13 Set Operators.

A `Set` is homogeneous. Its elements are all `Scalar` — enumerated DPM items — or all `Constant` — literals of one primitive type. Both forms are in circulation and both are correct; what does not occur, and would not mean anything, is a set mixing an *Item* with a Number, since every *Operator* that consumes a set compares its elements against a single value.

> **The `class_name` on each child is redundant, and will not survive**
>
> A homogeneous set repeats its element kind on every child, and repeats `scalar_type` or `type_` there too, when both belong to the set as a whole. The encoding that would say this once — `scalar_type` on the `Set` itself and bare values in `children` — is the one this format should have. It is not changed here because documents in circulation use the shape above and this version describes them; the redesign is tracked as future work.

An **empty** `children` array is the encoding of the DPM-XL empty-set literal `{}`, and is not a defect. Its usual place is the right operand of an equality against a `SetOp` — the idiom for asserting that a set difference is empty:

```
{ "class_name": "BinOp", "op": "=",
  "left":  { "class_name": "SetOp", "op": "setdiff", "operands": [ … ] },
  "right": { "class_name": "Set", "children": [] } }
```

```
{ "class_name": "Set",
  "children": [ { "class_name": "Scalar", "item": "eba_RS:x1", "scalar_type": "Item" },
                { "class_name": "Scalar", "item": "eba_RS:x6", "scalar_type": "Item" } ] }
```

5.4.7 4.7 `Dimension`

A reference to a *Component* of the operand *Recordset*, by code.

Member	Type	Presence	Notes
`dimension_code`	String	REQUIRED	The *Component* code.

Unlike `VarID`, a `Dimension` carries no numeric identifier; only the code is serialised.

4.7.1 The three *Component* roles

A *Recordset* has *Components* in three roles — *Key*, *Fact* and *Attribute* (Information Model §2.2.3) — and `dimension_code` reaches **all three**. The role is not encoded: it follows from the code, and a Consumer determines it by where the code resolves.

Role	How the code resolves	Examples
Key — open	A member name of `open_keys` for the operand's table (§3.6), **after** any `RenameClauseOp` (§4.24).	`GCC`, `INC`, `LGS`, `qEEA`
Key — standard	The axis tokens `r`, `c`, `s`.	`r`, `c`, `s`
Key — contextual	The reserved identifiers `refPeriod` and `entityID`, latent in every *Recordset* and never declared.	`entityID`
Fact	The single reserved code `f`.	`f`
Attribute	The *Property* code of an *Attribute Variable* associated with the selected *Variables*. Not declared in the document — see below.	`baseCurrency` (the only *Attribute* code in circulation)

Two consequences a Consumer MUST honour:

1. **`open_keys` is not the whole vocabulary.** A Consumer that resolves every `dimension_code` against `open_keys` alone will reject documents it should accept, because `f`, the axis tokens, the contextual identifiers and every *Attribute* code lie outside it. `baseCurrency` is common enough that such a Consumer fails almost at once.
2. **Producers MUST NOT declare an *Open Key* named `f`, `r`, `c`, `s`, `refPeriod` or `entityID`.** These are reserved (Information Model §2.4.3), and a table declaring one would make its own *Components* unaddressable.

4.7.2 Reserved contextual codes, and *Attribute Components*

Three codes are **contextual**: they are present on every *Recordset*, are declared by no `open_keys`, and a Producer MUST NOT declare an *Open Key* with any of them.

Code	*Data Type*	Meaning
`entityID`	String	The reporting entity of the instance under validation.
`refPeriod`	Date	The reference period of the instance under validation. Reached through `GetClauseOp` (§4.23) rather than as a `Dimension`.
`baseCurrency`	Item	The currency a monetary fact is reported in. An *Attribute Component* rather than a *Key Component*, but contextual in the same way: always present, never declared.

A `dimension_code` therefore resolves against exactly two things: the three codes above, and the `open_keys` of the operand's table (§3.6), after any rename (§4.24). **A code that resolves against neither is a malformed document**, and a Consumer MUST reject it (§5.2, requirement 2) exactly as it would an unrecognised `class_name` — not treat it as an *Attribute* reference and carry on. Nothing outside this list is inferred.

What an implementation does when the *reported instance* does not carry a value for a code that does resolve is a different question, and not one for this part: that is a property of the data, not of the encoding (§1.1).

Attribute Components in general arise when a selected *Variable* has an associated *Attribute Variable*, the name being the *Property* code of that attribute (Information Model §2.2.3.3). §3.6 declares a table's *Variables* and its *Open Keys*, but not its *Attributes*; `baseCurrency` is the only one that occurs, and it is covered above as a contextual code. Declaring *Attributes* generally would require a new member of `tables[*]`, which this version does not define.

4.7.3 Placement

A `Dimension` *Node* occurs in four positions, and nowhere else:

- within the `condition` of a `WhereClauseOp` (§4.21) or a `SubClauseOp` (§4.22) — the usual case, as the `left` of a comparison `BinOp`, or as the operand of `not`/`isnull`, or as the whole condition where it is the Boolean *Fact Component* `f`;
- within the `condition` of a `FilterOp` (§4.25);
- as the `operand` of a `SubstrOp` (§4.15);
- directly as the `condition` of a `WhereClauseOp` or `SubClauseOp`, where the *Component* is itself Boolean — in practice `f`.

```
{ "class_name": "Dimension", "dimension_code": "baseCurrency" }
```

`refPeriod` is the one reserved code that does not appear as a `Dimension`: it is reached through `GetClauseOp` instead (§4.23), because the *Operations* using it want its value in the *Fact* rather than a test on it.

5.4.8 4.8 `ParameterRef`

A reference to a *Module Parameter* declared in `parameters` (§3.9). It is the encoding of the Parameter Selection, Information Model §3.1 and Operators §3.1.1, Form 5, written `{p_BM, string}` or `{p_threshold, number, default: 0}` in surface syntax.

Member	Type	Presence	Notes
`code`	String	REQUIRED	*Parameter* code. MUST be a member name of `parameters` (§3.9).
`param_type`	String	REQUIRED	MUST equal the type declared in `parameters` for `code`, in the spelling of §3.9.1.
`default`	any	OPTIONAL, *nullable*	Fallback value for this reference. Absent and `null` both mean none is declared.

`param_type` duplicates the declaration and is retained so that a *Node* remains self-describing (§2.3). A Consumer MUST reject a document where the two disagree (§5.3, rule V-29).

Note the case difference between the two forms: the surface syntax spells the type in lower case (`string`, `boolean`, `integer`), while `param_type` and `parameters` use the initial-capital spelling (§3.9.1).

4.8.1 `default`

`default` is the fallback for **this reference**, and it is where a *Parameter* default lives — the surface syntax attaches it to the selection, not to the declaration, and so does this format. It is OPTIONAL and nullable, on the same terms as `VarID.default` (§4.3.1): the two members answer the same question in their respective domains, and this part draws no distinction between them. A Producer MAY omit the member or emit `null` where the selection declares no default; a Consumer MUST treat the two alike. This is the one place where an absent member and an explicit `null` are equivalent, and it is stated here because §4.1.3 makes the opposite the general rule.

Any value of the declared type is admissible, **including `null`**, which is a value of every *Data Type*. For a *Set* type an empty set — `[]` — is admissible and distinct from `null`: it declares a default that is the empty set, where `null` declares no default at all.

The encoding follows the three shapes the surface syntax admits in this position — a literal, an item reference, or a set literal — each mapped to the *Node* class this part already uses for it:

`param_type`	`default` encoding
`"Integer"`, `"Number"`, `"Boolean"`, `"String"`, `"Date"`	A JSON literal, related to `param_type` exactly as `Constant.value` is to `Constant.type_` (§4.4): a JSON integer, number, boolean or string, with `"Date"` a `YYYY-MM-DD` string.
`"Item"`	A bare String: the item signature (the same value as a `Scalar` *Node*'s `item`, §4.5), not a `Scalar` *Node*.
`"SetNumber"`, `"SetInteger"`, `"SetString"`, `"SetDate"`, `"SetBoolean"`	A JSON array of bare literals of the element type, not a `Set` *Node*.
`"SetItem"`	A JSON array of bare item-signature Strings, not a `Set` *Node*.

A default is always a bare value or an array of them, never wrapped in the `Scalar`/`Set` *Node* this part defines for the same value elsewhere in the format: `param_type` already fixes the element type, so a wrapper would only repeat it, the same reasoning that makes `SubstrOp.start` a bare integer (§4.15).

4.8.2 Semantics

A `ParameterRef` evaluates to a *Scalar* for the six *Scalar* types, and to a *Scalar Set* for the six *Set* types. Either way it carries no *Position Index*, so `idx(ParameterRef) = ∅` and it broadcasts over any index set it is combined with (§4.1.5).

Its value is resolved in the three steps of §3.9.2 — supplied binding, then this reference's `default`, then null. The third step is ordinary null, not a failure: the *Operation* is evaluated with a null *Parameter* under the null treatment of Information Model §4.

Using a *Parameter* changes the contract between the *Operation* and its execution environment (Information Model §3.1.4): an engine that cannot obtain bindings will still evaluate the document, but the results of the affected *Operations* will reflect a null where a value was intended. A Consumer SHOULD therefore report which *Parameters* it resolved and from where — binding or default — alongside its results, so that such an outcome is visible rather than silent.

```
{ "class_name": "ParameterRef", "code": "BM", "param_type": "String", "default": null }
```

With a declared default, encoding `{p_threshold, number, default: 0}`:

```
{ "class_name": "ParameterRef", "code": "threshold", "param_type": "Number", "default": 0 }
```

An item-typed default, encoding `{p_ccy, item, default: [eba_CU:EUR]}`:

```
{ "class_name": "ParameterRef", "code": "ccy", "param_type": "Item", "default": "eba_CU:EUR" }
```

A set-typed default, encoding `{p_ccys, set-item, default: {[eba_CU:EUR]}}`:

```
{ "class_name": "ParameterRef", "code": "ccys", "param_type": "SetItem", "default": [
"eba_CU:EUR" ] }
```

5.4.9 4.9 `PreconditionItem`

A reference to a *filing-indicator Variable* — a Boolean *Operand*, and a *Scalar* for the purposes of §4.1.5.

Member	Type	Presence	Notes
`variable_id`	Integer	REQUIRED	`VariableID`. MUST be a member name of `precondition_variables` (§3.8).
`variable_code`	String	REQUIRED	The filing indicator code, which is the code of the template it governs, e.g. `"C_28.00"`.

A filing indicator is a distinct kind of *Variable* in DPM-ML, not a fact, which is why it is referenced by its own *Node* class and typed in its own dictionary (§3.8) rather than through a `VarID`. It carries no coordinates: an indicator is reported once per instance, not per cell.

Semantics

Evaluates to true when the instance under validation reports the corresponding filing indicator as true, and to false otherwise — including when the indicator is absent.

Placement

A precondition tree (§3.7) is where a `PreconditionItem` is expected. That is not, however, the converse of §3.7.1: nothing in this part reserves the class to preconditions or forbids a Validation from testing a filing indicator. A Consumer SHOULD evaluate the *Node* wherever a Boolean *Operand* is admissible, rather than rejecting the document on its position.

```
{ "class_name": "PreconditionItem", "variable_id": 395097, "variable_code": "C_28.00" }
```

5.4.10 4.10 `ParExpr`

An explicit parenthesisation, retained from the source expression.

Member	Type	Presence	Notes
`expression`	*Node*	REQUIRED	The parenthesised *Node*.

`ParExpr` is semantically transparent: it evaluates to its `expression` and preserves its index set. It carries no precedence information, because precedence is already resolved by the tree structure.

Producers MUST preserve the parenthesisation of the source expression so that `expression` (§3.10) can be reconstructed. A `ParExpr` MAY nest directly within another `ParExpr`, and MAY be the root *Node* of an *Operation*. Consumers MUST accept `ParExpr` at any position where a *Node* is permitted, and MUST NOT rely on its presence or absence.

```
{ "class_name": "ParExpr", "expression": { "class_name": "BinOp", "op": "or", … } }
```

5.4.11 4.11 `UnaryOp`

Member	Type	Presence	Notes
`op`	String	REQUIRED	One of the fifteen values below.
`operand`	*Node*	REQUIRED	The operand.

`op`	Category	Operand type	Result	Semantics
`+`, `-`, `abs`	Numeric	Numeric	Numeric	Operators §4.2
`sqrt`, `exp`, `ln`	Numeric	Numeric	Number	Operators §4.2
`isnull`	Comparison	any	Boolean	Operators §5.10
`not`	Logical	Boolean	Boolean	Operators §6
`len`	String	String	Integer	Operators §9.1
`year`, `semester`, `quarter`, `month`, `week`, `day`	Time	Date	Integer	Operators §10.2

The six date-component extractors share one signature and differ only in the component returned (Operators §10.2.1), so they take one `op` value each on the same *Node* shape. `sqrt`, `exp` and `ln` are likewise unary numeric functions, alongside `abs`.

Note that `sqrt`, `exp` and `ln` always yield Number, never Integer, even for an Integer operand (Operators §4.1); and that they reject *Number Interval* operands, so a Consumer MUST NOT apply interval arithmetics to them even where the operand `VarID` carries `interval: true`.

A date-component extractor takes any Date-valued *Node*. The commonest is `[get refPeriod]`, the reference period of the instance read as a *Component* (§4.23), since a reported Date *Variable* is comparatively rare.

```
{ "class_name": "UnaryOp", "op": "isnull",
  "operand": { "class_name": "VarID", … } }
```

5.4.12 4.12 `BinOp`

Member	Type	Presence	Notes
`op`	String	REQUIRED	One of the symbols below.
`left`	*Node*	REQUIRED	Left operand.
`right`	*Node*	REQUIRED	Right operand.

`op`	Category	Operand types	Result	Semantics
`+`, `-`, `*`, `/`	Numeric	Numeric	Numeric	Operators §4.3
`power`, `log`	Numeric	`left` Numeric; `right` a Numeric *Scalar*	Number	Operators §4.2.7
`nvl`	Conditional	`left` any, `right` of `left`'s type	`left`'s type	Operators §8
`=`, `!=`, `<`, `<=`, `>`, `>=`	Comparison	Comparable, of compatible *Data Type*	Boolean	Operators §5
`match`	Comparison	`left` String; `right` a String *Scalar* pattern	Boolean	Operators §5.9
`in`	Set	`left` item- or value-valued; `right` a `Set` or `SetOp`	Boolean	Operators §13
`and`, `or`, `xor`	Logical	Boolean	Boolean	Operators §6
`&`	String	String	String	Operators §9.2

Additional constraints:

1. Where `op` is `"in"`, `right` MUST be a `Set` *Node* (§4.6) or a `SetOp` *Node* (§4.14).
2. Where `op` is `"and"`, `"or"` or `"xor"`, both operands MUST yield Boolean. No further restriction applies inside a precondition tree: a precondition is an *Operation* tree and admits every *Node* class (§3.7.1).
3. Where `op` is `"match"`, `right` MUST be a `Constant` with `type_: "String"` holding a regular expression in Python syntax (Operators §5.9.1.5). The grammar admits only a literal in this position, so a Producer MUST NOT emit a selection or a `ParameterRef` there.
4. Where `op` is `"log"`, `right` is the base and MUST be a numeric *Scalar*; where it is `"power"`, `right` is the exponent. Both are asymmetric, so a Consumer MUST NOT commute their operands.
5. Operand index sets MUST combine per §4.1.5.

`power` and `log` are binary numeric *functions* in the surface syntax — `log(op, base)`, `power(op, exponent)` — rather than infix *Operators*, but their encoding is a `BinOp` like any other two-operand *Operator*: the *Node* classes group by arity and shape, not by surface form (§4.2).

`nvl` is the null-substitution *Operator*: it yields `left` where `left` is non-null and `right` otherwise. It is a `BinOp` although the Operators part specifies it under Numeric *Operators*, and it has no DPM-ML *OperatorID* (§4.2.1). All nine occurrences guard an `AggregationOp` against an empty aggregation, as `nvl(count(…), 0)`.

Associativity is expressed by nesting: `a + b + c` is encoded as one `BinOp` whose `left` is another `BinOp`. Consumers MUST NOT assume a particular nesting direction.

```
{ "class_name": "BinOp", "op": ">=",
  "left": { "class_name": "VarID", "table": "C_28.00", "column": "040", … },
  "right": { "class_name": "VarID", "table": "C_28.00", "column": "050", … } }
```

5.4.13 4.13 `ComplexNumericOp`

The n-ary numeric *Operators* `min` and `max`.

Member	Type	Presence	Notes
`op`	String	REQUIRED	`"min"` or `"max"`.
`operands`	Array of *Node*	REQUIRED	MUST contain at least two *Nodes*.

Operands MUST be numeric and MUST combine per §4.1.5. The result is the least or greatest operand value at each *Position Index*. These are the element-wise numeric *Operators*, distinct from the aggregate `min_aggr` and `max_aggr` (§4.18), which reduce a *Recordset*. Arities of 2, 3 and 7 occur.

```
{ "class_name": "ComplexNumericOp", "op": "max",
  "operands": [ { "class_name": "VarID", … }, { "class_name": "VarID", … } ] }
```

5.4.14 4.14 `SetOp`

Set construction and set algebra. The result is a ***Scalar Set*** as defined in Operators §13 — unordered, duplicate-free, null-free, carrying no *Components* — so `idx(SetOp) = ∅` (§4.1.5).

Member	Type	Presence	Notes
`op`	String	REQUIRED	One of the five values below.
`operands`	Array of *Node*	REQUIRED	Arity per `op`, below.

`op`	Arity	Meaning
`set_of`	exactly 1	The *Scalar Set* of the distinct non-null *Fact* values of its operand.
`union`	2 or more	Members of any operand.
`intersect`	2 or more	Members of every operand.
`setdiff`	exactly 2	Members of the first operand that are not members of the second.
`symdiff`	exactly 2	Members of exactly one of the two operands.

The arities follow the surface syntax (Operators §13): `union` and `intersect` are variadic, `setdiff` and `symdiff` strictly binary because neither is associative.

Semantics are those of Operators §13 Set Operators, which is authoritative. None of the three has a DPM-ML *OperatorID* (§4.2.1).

An operand that is a *Recordset* rather than a *Scalar Set* is **implicitly read as the set of its *Fact* values**, exactly as if wrapped in `set_of` (Operators §13.1.5). It is what lets `union` and `setdiff` apply directly to a `GetClauseOp` without an intervening `set_of`: `[get INC]` moves the *Key Component* into the *Fact* (§4.23), and the set *Operator* then takes the set of those values. A Consumer MUST perform that implicit conversion rather than rejecting the operand.

A `SetOp` occurs as the `right` of a `BinOp` `in`, or as an operand of an equality against a `Set` — usually the empty `Set`, to assert that a difference is empty:

```
{ "class_name": "SetOp", "op": "setdiff",
  "operands": [
    { "class_name": "SetOp", "op": "union", "operands": [ { "class_name": "GetClauseOp", … },
                                                          { "class_name": "GetClauseOp",
… } ] },
    { "class_name": "GetClauseOp", … } ] }
```

5.4.15 4.15 `SubstrOp`

Substring extraction. `idx(SubstrOp) = idx(operand)`.

Member	Type	Presence	Notes
`op`	String	REQUIRED	`"substr"`.
`operand`	*Node*	REQUIRED	The String-valued *Node* to cut.
`start`	Integer or null	OPTIONAL, *nullable*	1-based index of the first character taken. A **plain JSON integer** or `null`.
`length`	Integer or null	OPTIONAL, *nullable*	Number of characters taken. A **plain JSON integer** or `null`.

`start` and `length` are encoded as bare JSON numbers (or `null`) rather than as `Constant` *Nodes*. If omitted or `null`, extraction begins at the first character and runs to the end of the string respectively (Operators §9.3.4). This is normative; a Consumer MUST NOT expect a *Node* here.

Its characteristic use is cutting the institution's LEI out of the entity identifier — `substr(entityID, 4, 20)` — so the `operand` is typically a `Dimension`, though any String-valued *Node* is admissible. `substr` has no DPM-ML *OperatorID* (§4.1.2).

```
{ "class_name": "SubstrOp", "op": "substr",
  "operand": { "class_name": "Dimension", "dimension_code": "entityID" },
  "start": 4, "length": 20 }
```

5.4.16 4.16 `DateConstructorOp`

The `date` constructor: builds a Date from three Integer parts. See Operators §10.3.

Member	Type	Presence	Notes
`op`	String	REQUIRED	`"date"`.
`year_node`	*Node*	REQUIRED	Integer-valued.
`month_node`	*Node*	REQUIRED	Integer-valued.
`day_node`	*Node*	REQUIRED	Integer-valued.

The three parts are named members rather than an `operands` array, because they are positionally significant and not interchangeable — the same reason `SubstrOp` names `start` and `length` and `TimeShiftOp` names its arguments. The `_node` suffix distinguishes them from the same-named *Operators* of §4.11, `year`, `month` and `day`, which extract a part from a Date rather than supplying one. They are full *Nodes*, not bare integers: Operators §10.3.5 admits a *Recordset* in any of the three positions.

No document in circulation uses this class, so the member names follow the only implementation that has them.

All three MUST combine per §4.1.5, and the result carries the combined index set. This is the sanctioned way to obtain a Date from Integers, since Integer → Date is not a permitted cast.

```
{ "class_name": "DateConstructorOp", "op": "date",
  "year_node":  { "class_name": "UnaryOp", "op": "year", "operand": { … } },
  "month_node": { "class_name": "Constant", "type_": "Integer", "value": 12 },
  "day_node":   { "class_name": "Constant", "type_": "Integer", "value": 31 } }
```

5.4.17 4.17 `AnnualiseOp`

Scales each value to its full-year equivalent, given the month a financial year ends. See Operators §10.4.

Member	Type	Presence	Notes
`op`	String	REQUIRED	`"annualise"`.
`operand`	*Node*	REQUIRED	The numeric *Node* to scale.
`fy_end`	*Node*	REQUIRED	Integer *Scalar*, the month (1–12) the financial year ends.
`component`	String	REQUIRED	*Component* code supplying each *Record*'s reference date.

`fy_end` is a *Node* because it is typically a `ParameterRef` — `FINANCIAL_YEAR_END_MONTH` is exactly this quantity — rather than a literal. It MUST evaluate to an Integer *Scalar*.

`component` is a bare String naming a *Component*, on the same terms as `GetClauseOp.component` (§4.23) and `TimeShiftOp.reference_period` (§4.27): the surface syntax's third argument is a `propertyCode`, not an expression, so the encoding carries a code rather than a *Node*. `"refPeriod"` is the expected value.

`idx(AnnualiseOp) = idx(operand)`; `fy_end` is a *Scalar* and broadcasts.

```
{ "class_name": "AnnualiseOp", "op": "annualise",
  "operand": { "class_name": "VarID", … },
  "fy_end":  { "class_name": "ParameterRef", "code": "FINANCIAL_YEAR_END_MONTH",
               "param_type": "Integer", "default": null },
  "component": "refPeriod" }
```

5.4.18 4.18 `AggregationOp`

Member	Type	Presence	Notes
`op`	String	REQUIRED	`"sum"`, `"count"`, `"avg"`, `"median"`, `"max_aggr"`, `"min_aggr"` or `"rank"`.
`operand`	*Node*	REQUIRED	The *Recordset* to aggregate.
`grouping_clause`	*Node* or `null`	OPTIONAL, *nullable*	A `GroupingClause` (§4.19). Absent and `null` both mean "no grouping".
`analytic_clause`	*Node* or `null`	OPTIONAL, *nullable*	An `AnalyticClause` (§4.20). Absent and `null` both mean "no analytic clause".

Both clause members are REQUIRED and both are nullable; a Consumer MUST NOT treat either as optional (§4.1.3). They are mutually exclusive: at most one MAY be non-null (§5.3, rule V-30).

`op`	*OperatorID*	Reduction	Result type
`sum`	26	Sum of the operand values.	the operand's
`count`	27	Number of *Records*, or the cardinality of a *Scalar Set*.	Integer
`avg`	—	Arithmetic mean of the operand values.	Number
`median`	—	Median of the operand values.	Number
`max_aggr`	11	Greatest operand value.	the operand's
`min_aggr`	12	Least operand value.	the operand's
`rank`	—	Ordinal position of each value within its window.	Integer

`avg` and `median` take the same *Node* shape as `sum` (Operators §7.6, §7.7) and admit a `GroupingClause` on the same terms. `count` differs in one respect: its operand MAY be a *Scalar Set* rather than a *Recordset*, in which case it returns the set's cardinality. A *Scalar Set* carries no *Components* to group by, so no `grouping_clause` accompanies it — a consequence of what a set is, specified in Operators §7.3, not an additional rule of this part.

`rank` MUST take an `AnalyticClause` (and its `window` MUST be `null`). Other aggregate operators (`sum`, `avg`, `count`, `min_aggr`, `max_aggr`, `median`) MAY take an `AnalyticClause` instead of a `GroupingClause`.

Semantics

With both clauses `null`, the *Operator* reduces the whole operand to a single *Scalar*, and `idx` becomes ∅.

With a `GroupingClause`, the operand is partitioned by the listed components and reduced within each partition, yielding one result per group. See Operators §7 Aggregate Operators.

With an `AnalyticClause`, the *Operator* is not a reduction: it yields one value per input *Record*, computed over an ordered window or partition, and preserves the operand's index set. When `op` is `rank`, `window` MUST be `null` (rank cannot take a window frame). For other aggregate operators, a `window` MAY be specified.

Null treatment follows Information Model §4 Null handling; note that a `default` on the operand `VarID` is applied before aggregation, so a fallback of `0` and no fallback are not equivalent.

```
{ "class_name": "AggregationOp", "op": "sum",
  "operand": { "class_name": "VarID", "table": "C_29.00", … },
  "grouping_clause": { "class_name": "GroupingClause", "components": [ "GCC" ] },
  "analytic_clause": null }
```

5.4.19 4.19 `GroupingClause`

Member	Type	Presence	Notes
`components`	Array of String	REQUIRED	MUST be non-empty.

Each component MUST be either:

- an **axis token** — `"r"`, `"c"` or `"s"`, denoting the row, column or sheet axis of the operand; or
- a ***Component* code** available on the operand.

The two namespaces are distinct but share one array. Consumers MUST test for the three axis tokens first and treat any other value as a *Component* code. Because the tokens are single lower-case letters and *Component* codes are not, the two cannot collide; Producers MUST NOT emit a *Component* code equal to `"r"`, `"c"` or `"s"`.

Grouping by an axis token partitions on *Position Index*: `group by c` produces one group per distinct `y`. Grouping by a *Component* code partitions on the reported values of that *Component*.

Resolution is after renaming. A *Component* code MUST be a member name of `open_keys` for the operand's table (§3.6) **as the operand presents it** — that is, after applying any `RenameClauseOp` between the table and the `AggregationOp` (§4.24). A rule that groups `{tC_27.00, c050}[rename INC to GCC]` by `GCC` is well-formed even though `GCC` is not an *Open Key* of `C_27.00`, because the rename put it there. A Consumer that resolves against the unrenamed table will reject such a rule.

A `GroupingClause` MUST occur only as the `grouping_clause` of an `AggregationOp`.

Components observed: the axis tokens `c` (63), `r` (8), `s` (12) and the pair `r, c` (12), and *Key Component* codes including `CEG`, `SIC`, `GCC`, `INC`, `CUS` and their `q`-prefixed forms.

```
{ "class_name": "GroupingClause", "components": [ "c" ] }
```

5.4.20 4.20 `AnalyticClause`

The window specification of an analytic `AggregationOp` (§4.18).

Member	Type	Presence	Notes
`partition_by`	Array of String	REQUIRED	*Component* codes or axis tokens partitioning the window. MAY be empty, meaning one window over the whole operand.
`order_by`	Array of Object	REQUIRED	Sort keys, below. REQUIRED and non-empty when `window` is present or `op` is `"rank"`; otherwise MAY be empty (`[]`).
`window`	Object or `null`	REQUIRED, *nullable*	Frame bounds. `null` means the whole partition. MUST be `null` when `op` is `"rank"`.

Each entry of `order_by`:

Member	Type	Notes
`key_name`	String	*Component* code, axis token, or the *Fact Component* `f` (§4.7).
`direction`	String	`"asc"` or `"desc"`.

An `AnalyticClause` MUST occur only as the `analytic_clause` of an `AggregationOp`. When `op` is `"rank"`, `order_by` MUST be non-empty and `window` MUST be `null` (rank cannot accept a window frame). For general aggregations without a window (e.g. `sum({…} over (partition by …))`), `order_by` MAY be empty. Where a `window` is present, `order_by` MUST be non-empty.

A non-null `window` is an object with the members below. It narrows the window from the whole partition to a frame around each *Record*, in the manner of an SQL window frame:

Member	Type	Presence	Notes
`frame_type`	String	REQUIRED	`"data_points"` to count *Records*, `"range"` to compare `order_by` values.
`start`	Object	REQUIRED	Lower bound, below.
`end`	Object	REQUIRED	Upper bound, below.

Each bound is an object:

Member	Type	Presence	Notes
`bound_type`	String	REQUIRED	`"unbounded_preceding"`, `"n_preceding"`, `"current_data_point"`, `"n_following"` or `"unbounded_following"`.
`n`	Integer	OPTIONAL	Distance from the current *Record*. Present only where `bound_type` is `"n_preceding"` or `"n_following"`; absent for the other three, which carry unboundedness or the current position in `bound_type` itself.

`start` MUST not be later than `end` in window order — `unbounded_preceding` < `n_preceding` < `current_data_point` < `n_following` < `unbounded_following`: a `start` of `"unbounded_following"` with an `end` of `"unbounded_preceding"` describes an empty frame and MUST NOT be emitted. Where `frame_type` is `"range"`, `n` is a distance

in the *Data Type* of the sole `order_by` key, so a `"range"` frame with an `n_preceding`/`n_following` bound requires exactly one `order_by` entry.

An unpartitioned window ordered by the fact value descending, with a `null` frame, is what `rank({…} over (order by f desc))` produces:

```
{ "class_name": "AnalyticClause",
  "partition_by": [],
  "order_by": [ { "key_name": "f", "direction": "desc" } ],
  "window": null }
```

A `null` `window` is the common case, and a Consumer MUST NOT treat the frame as optional on that account: where one is present it MUST be evaluated as specified rather than ignored, since ignoring it widens the window and changes every rank it produces.

`rank(…) over (…)` has no DPM-ML *OperatorID* (§4.1.2).

5.4.21 4.21 `WhereClauseOp`

Restricts a selection by a condition on its *Components*.

Member	Type	Presence	Notes
`operand`	*Node*	REQUIRED	The selection to restrict.
`condition`	*Node*	REQUIRED	A `BinOp`, a `UnaryOp` or a `Dimension` *Node*.

The `condition` compares *Components* to values: its `left` is normally a `Dimension` *Node* (§4.7) and its `right` a `Scalar` (§4.5), a `Set` (§4.6) or a `SubstrOp` (§4.15), combined with `=`, `!=` or `in`, and composed with `and`/`or`. It MAY also be a `UnaryOp` — `not(isnull(f))` is the common form — or, where the *Fact Component* is Boolean, the bare `Dimension` `f`. All three shapes occur: 399 `BinOp`, 11 `UnaryOp`, 6 `Dimension`.

`WhereClauseOp` narrows which *Records* the operand contributes but does not renumber them: `idx(WhereClauseOp) = idx(operand)`. See Operators §11 Clause Operators.

```
{ "class_name": "WhereClauseOp",
  "operand":   { "class_name": "VarID", "table": "C_09.04", "row": "0070", "column": "0010",
… },
  "condition": { "class_name": "BinOp", "op": "=",
                 "left":  { "class_name": "Dimension", "dimension_code": "RIO" },
                 "right": { "class_name": "Scalar", "item": "eba_GA:qx2014", "scalar_type":
"Item" } } }
```

5.4.22 4.22 `SubClauseOp`

Selects a **sub-*Record*** of the operand by fixing a *Component* the rendering leaves open. Encodes DPM-ML *Operator* 38, `sub`.

Member	Type	Presence	Notes
`operand`	*Node*	REQUIRED	The selection to narrow.
`condition`	*Node*	REQUIRED	Any *Node*. In practice always a `BinOp` `op: "="` equating a *Component* to a value, which is the form Operators §11 defines.

`sub` differs from `where` in what it does to the *Recordset*, not in what it tests: `where` filters *Records* and leaves the *Component* in place, while `sub` fixes the *Component* to the given value and drops it, so the result is the single *Record* at that value. `idx(SubClauseOp) = idx(operand)`. See Operators §11 Clause Operators.

```
{ "class_name": "SubClauseOp",
  "operand":   { "class_name": "VarID", "table": "C_07.00.a", "row": "0010", "column": "0200",
… },
  "condition": { "class_name": "BinOp", "op": "=",
                 "left":  { "class_name": "Dimension", "dimension_code": "qEBB" },
                 "right": { "class_name": "Scalar", "item": "eba_qEC:qx19", "scalar_type":
"Item" } } }
```

5.4.23 4.23 `GetClauseOp`

Moves a *Component* of the operand into the *Fact* position. Encodes DPM-ML *Operator* 29, `get`. The result is a *Recordset* with the same structure as the operand, so `idx(GetClauseOp) = idx(operand)`; it is not a set. Where a set is wanted, a `SetOp` supplies it, explicitly or implicitly (§4.14).

Member	Type	Presence	Notes
`operand`	*Node*	REQUIRED	The selection to project. Any *Node* yielding a *Recordset* that carries the named *Component*.
`component`	String	REQUIRED	*Component* code to project. A **plain String**, not a `Dimension` *Node*.

`component` is a bare String even though `Dimension` (§4.7) exists for the same purpose elsewhere. This is normative; a Consumer MUST NOT expect a *Node* here.

The code is resolved as in §4.7.2: a member name of `open_keys` for the operand's table (§3.6) after any rename, or one of the three reserved contextual codes. Which *Components* `get` may be applied to is a semantic question, settled in Operators §11.3.5 — the *Fact Component* and the standard *Key Components* `r`, `c` and `s` are excluded there — and this part adds nothing to it.

`refPeriod` is the commonest thing a `GetClauseOp` projects: the reference period of the instance under validation, moved into the *Fact* as a Date, where it feeds `month` and `year` (§4.11).

See Operators §11 Clause Operators.

```
{ "class_name": "GetClauseOp",
  "operand": { "class_name": "VarID", "table": "C_29.00", "column": "030", … },
  "component": "INC" }
```

5.4.24 4.24 `RenameClauseOp`

Renames *Components* of the operand. Encodes DPM-ML *Operator* 33, `rename`. `idx(RenameClauseOp) = idx(operand)`. *Operator* 34 (`node`) has no encoding in this version and no defined semantics anywhere in this specification; see §5.4.

Member	Type	Presence	Notes
`operand`	*Node*	REQUIRED	The *Recordset* whose *Components* are renamed.
`clauses`	Array of Object	REQUIRED	Rename pairs, below. MUST be non-empty.

Each entry of `clauses`:

Member	Type	Notes
`from_component`	String	Existing *Component* code on the operand.
`to_component`	String	New code.

The renames of one *Node* apply **simultaneously**, not in sequence: a Consumer MUST compute the new *Component* set in one step, so that `[rename qEGS to qCIN, qLGS to qLIN]` — the most common form, 30 occurrences — cannot be affected by the order of its entries. Producers MUST NOT emit a `clauses` array whose `from_component` and `to_component` sets overlap.

Renaming is what makes an aggregation across two tables alignable, by giving the same *Component* the same code on both sides. It has consequences for resolution elsewhere: a *Component* code in a `GroupingClause` (§4.19), a `GetClauseOp` (§4.23) or a `Dimension` (§4.7) resolves against the *Component* set **as renamed**, not against the `open_keys` of the underlying table.

```
{ "class_name": "RenameClauseOp",
  "operand": { "class_name": "AggregationOp", "op": "sum", … },
  "clauses": [ { "from_component": "GCC", "to_component": "INC" } ] }
```

5.4.25 4.25 `FilterOp`

Restricts a selection by a condition on reported **data** rather than on *Components*.

Member	Type	Presence	Notes
`selection`	*Node*	REQUIRED	The selection to filter.
`condition`	*Node*	REQUIRED	A `BinOp` or `UnaryOp` *Node* yielding Boolean.

The `condition` may reference `VarID` *Nodes*, which are themselves resolved and MUST be co-located with `selection` — that is, they select from the same table and the same non-varying axes — so that the condition is evaluable per *Record*. As with `WhereClauseOp`, indices are preserved. See Operators §8 Conditional Operators.

`FilterOp` differs from `WhereClauseOp` in the subject of the test: `where` tests metadata, `filter` tests values.

```
{ "class_name": "FilterOp",
  "selection": { "class_name": "VarID", "table": "C_08.02", "column": "0020", "sheet": "0003",
… },
  "condition": { "class_name": "BinOp", "op": "=",
                 "left":  { "class_name": "VarID", "table": "C_08.02", "column": "0010",
"sheet": "0003", … },
                 "right": { "class_name": "Constant", "type_": "Integer", "value": 1 } } }
```

5.4.26 4.26 `CondExpr`

The `if-then-else` *Operator*.

Member	Type	Presence	Notes
`condition`	*Node*	REQUIRED	MUST yield Boolean.
`then_expr`	*Node*	REQUIRED	Evaluated where `condition` is true.
`else_expr`	*Node*	OPTIONAL	Evaluated where `condition` is false. **Omitted**, not `null`, where the *Operator* has no else branch.

`else_expr` is OPTIONAL and its absence is significant (§4.1.3). A Producer MUST omit it rather than emit `null`; a Consumer MUST treat absence, not `null`, as the one-armed form. The one-armed conditional is the commoner shape of the two, so this is not an edge case a Consumer can defer.

Where `else_expr` is absent and `condition` is false, the *Operation* yields no result for that *Position Index*. A Consumer MUST report such an index as not applicable, distinctly from passed and failed. This is the encoding of a one-armed conditional validation — "if these cells are reported, that cell must be reported too" — where the absence of the trigger is not itself a finding.

Null treatment of `condition` follows Information Model §4 Null handling: a null condition selects neither branch.

```
{ "class_name": "CondExpr",
  "condition": { "class_name": "BinOp", "op": ">", … },
  "then_expr": { "class_name": "BinOp", "op": ">", … } }
```

5.4.27 4.27 `TimeShiftOp`

Selects the operand from a different reference period.

Member	Type	Presence	Notes
`operand`	*Node*	REQUIRED	The selection to shift.
`period_indicator`	*Node*	REQUIRED	MUST be a `Constant` with `type_: "String"`.
`shift_number`	*Node*	REQUIRED	A `Constant` with `type_: "Integer"`, or a `UnaryOp` `op: "-"` over one.
`reference_period`	String	REQUIRED	`"refPeriod"`.

`period_indicator` gives the period unit and `shift_number` the number of units to shift. `"A"` with `1` denotes one year earlier, `"M"` with `12` the same span expressed in months. Permitted period indicators are those defined in Operators §10 Time Operators; `A`, `Q` and `M` occur.

A shift may be forward. `shift_number` is not restricted to a positive `Constant`: three occurrences encode `-1` as a `UnaryOp` `op: "-"` wrapping `Constant 1`, giving a shift of one quarter **later**. A Consumer MUST evaluate `shift_number` as a *Node* and honour its sign, and MUST NOT read the `Constant` beneath a negation as if it were the value.

`reference_period` names the temporal anchor of the shift. `"refPeriod"` — the reference period of the instance under validation — is the only value currently assigned; §5.5 reserves the member for further anchors.

An *Operation* containing a `TimeShiftOp` requires an instance for the shifted period and MUST therefore appear in a `cross_instance_dependencies` entry whose `modules` include an entry with the corresponding `ref_period` (§3.11), which for a forward shift is of the form `T+<n><p>`.

```
{ "class_name": "TimeShiftOp",
  "operand": { "class_name": "VarID", "table": "F_25.01.a", "row": "0120", … },
  "period_indicator": { "class_name": "Constant", "type_": "String", "value": "M" },
  "shift_number":     { "class_name": "Constant", "type_": "Integer", "value": 12 },
  "reference_period": "refPeriod" }
```

A shift of one quarter forward:

```
{ "class_name": "TimeShiftOp",
  "operand": { "class_name": "VarID", … },
  "period_indicator": { "class_name": "Constant", "type_": "String", "value": "Q" },
  "shift_number":     { "class_name": "UnaryOp", "op": "-",
                        "operand": { "class_name": "Constant", "type_": "Integer", "value":
1 } },
  "reference_period": "refPeriod" }
```

5.5 5 Conformance

5.5.1 5.1 Producer Requirements

A conforming Producer MUST:

1. Emit exactly one *Script Document* per module version, containing every *Operation* whose scope for that module version is active, and no *Operation* whose scope is inactive.
2. Resolve every `with` context into every `VarID` *Node*, so that no *Node* depends on an enclosing context (§2.3).
3. Expand every range, wildcard and enumerated selection into explicit *Datapoint References*, carrying each cell's resolved coordinates on every axis the selection did not pin, and assigning *Position Indices* on the row, column and sheet axes as `x`, `y` and `z` only where the axis varies (§2.4, §4.3.2).
4. Emit a document satisfying every rule of §5.3.
5. Keep `expression`, `root_operator_id` and `ast` mutually consistent (§4.1.4, §4.2).
6. Refrain from emitting any construct listed as Unassigned in §5.4.
7. Declare in `parameters` every *Module Parameter* that any `ParameterRef` reads, with the type that `ParameterRef` carries, and emit each `ParameterRef.default` explicitly — the declared default, or `null` (§3.9, §4.8.1).
8. Emit only preconditions that yield a *Scalar* Boolean (§3.7.1), and declare the *Variables* a precondition selects in the same dictionaries as those of an *Operation*.
9. Emit a `cross_instance_dependencies` entry for every *Operation* that selects from a table of another module version or another reference period, and a `dependency_modules` entry for the structure of each such module. This is the requirement most often missed in practice; see §5.7.1. Producers SHOULD regenerate all documents of a framework together for a given `dpm_release`, so that cross-module dependencies are mutually consistent. Where they do not — a module version need only be regenerated when it changes, so documents held together may name different releases (§3.3) — each document's `dpm_release` MUST still record the release it was actually generated from.

5.5.2 5.2 Consumer Requirements

A conforming Consumer is one that **reads a *Script Document* correctly**. What it then computes is governed by the Operators and Information Model parts, and what it reports is its own concern (§1.1); neither is a requirement of this part.

A conforming Consumer MUST:

1. Dispatch solely on `class_name`, and jointly on `class_name` and `op` where a class carries several *Operators* (§4.1.1, §4.2).
2. Reject a document containing an unrecognised *Module Object* member, an unrecognised `class_name`, or an unrecognised `op` for a recognised class.
3. Evaluate `ast`, never `expression` (§4.1.4).
4. Honour the distinction between an absent OPTIONAL member and an explicit `null` (§4.1.3) — in particular, treat an absent `CondExpr.else_expr` as the one-armed form (§4.26).
5. Read each *Datapoint Reference*'s position from the entry itself, never from the `VarID`-level coordinates, which record only what the selection pinned (§4.3.2).
6. Resolve *Data Types* from `variables`, `tables[*].variables`, `precondition_variables` and `dependency_modules[*]` only, never by inference from a value found in an instance.
7. Determine which *Variables* to load from each table of an instance from `tables[*].variables`, never by collecting the `table` and `datapoint` values found in `operations` (§3.6).
8. Resolve *Component* codes only against the document in hand — the `open_keys` of §3.6, after any `RenameClauseOp` (§4.24) — together with the *Fact Component* `f` and the three reserved contextual codes `entityID`, `refPeriod` and `baseCurrency`, which no `open_keys` declares (§4.7.2, §4.23).
9. Resolve every *Module Parameter* an *Operation* reads in the order supplied binding, then that reference's `default`, then null (§3.9.2, §4.8.2). The third step is a value, not a failure: a Consumer MUST NOT refuse to load a document, or set an *Operation* aside, because a *Parameter* is unbound.
10. Apply the applicability rules: reference date within `dates` (§3.4), submission date at or after `from_submission_date` (§3.10), and, for a cross-instance *Operation*, every in-window dependency entry satisfied — counting a module as available when any member of its `alternative_dependencies` group is (§3.11.2).
11. Accept both spellings of an open-ended upper bound: `null` in `dates.to`, the empty string in `to_reference_date` (§3.4).

Consumers MAY cache resolved *Node* results, since *Nodes* are context-free (§2.3). Consumers MUST NOT cache across documents keyed on codes, since codes are version-scoped (§3.6).

What is deliberately not required here

Earlier drafts of this part also told a Consumer how to combine *Operands* across *Position Indices*, how to treat `interval` on a non-numeric *Operand*, when to skip an *Operation* whose precondition is false, and which outcome categories to report. The first three are semantics, specified in the Operators and Information Model parts and identical whether an *Operation* reaches an engine as a document or as surface syntax; restating them here could only duplicate or contradict them. The fourth is output, which this part does not standardise (§1.1).

A Consumer that satisfies the eleven requirements above has read the document as intended. Whether it then evaluates it correctly is a question about the Operators part, and this part defers to it entirely (§1.3).

5.5.3 5.3 Structural Validation Rules

A *Script Document* is **valid** if and only if all of the following hold. These rules are mechanically checkable without reference to any external artefact, and a Producer MUST NOT publish a document that violates any of them.

#	Rule
V-1	The root object has exactly one member, whose name is an absolute URI.
V-2	The *Module Object* has all fourteen REQUIRED members (§3.2) and no other member; `format_version` matches `MAJOR.MINOR` (§3.2.1).
V-3	For every entry of `operations` and `preconditions`, `code` equals the member name, and the entry carries exactly the members specified in §3.10 and §3.7 respectively.
V-4	For every *Operation Object* whose root *Node* encodes a DPM-ML *Operator*, `root_operator_id` equals that *OperatorID* per §4.2.
V-5	Every *Node* has a `class_name` from §4.1.2, and carries only the members specified for that class.
V-6	Every `VarID` axis is in exactly one of the four states of §4.3.2; no partial placement occurs; where an entry carries a coordinate whose axis is also fixed at the `VarID` level, the two are equal; at least one axis is not Not applicable.
V-7	Within every `VarID`, `data` is non-empty, every `datapoint` is non-null, all index components are positive integers, and all *Position Indices* are pairwise distinct.
V-8	`variables` equals the union of `tables[*].variables`.
V-9	Every `datapoint` of every `VarID` in `operations` **or** `preconditions` appears in `variables` if the `VarID`'s `table` is a member name of `tables`, and otherwise in the `variables` or `tables[*].variables` of some `dependency_modules` entry.
V-10	Every `table` of every `VarID` in `operations` or `preconditions` is a member name of `tables` or of some `dependency_modules[*].tables`.
V-11	The key sets of `variables` and `precondition_variables` are disjoint, and every value of `precondition_variables` is `"b"`.
V-12	Every precondition `ast` has `idx(ast)` = ∅ under §4.1.5, so it does not vary on any axis (§3.7.1). Every `variable_id` of every `PreconditionItem`, wherever it occurs, appears in `precondition_variables`.
V-13	Every code in `affected_operations`, in both `preconditions` and `cross_instance_dependencies`, is a member name of `operations`; every such array is non-empty.
V-14	`intra_instance_validations` and the union of all `cross_instance_dependencies[*].affected_operations` cover the member names of `operations` exhaustively and do not intersect each other. An *Operation* MAY appear in several `cross_instance_dependencies` entries (§3.11.2).
V-15	Every `operand_reference_id` denotes exactly one (`table`, `datapoint`, `row`, `column`, `sheet`) combination throughout the document. It need not be unique (§4.3.4).
V-16	Every `modules[*].URI` is a member name of `dependency_modules` or the root URI; every member name of `dependency_modules` occurs as some `modules[*].URI`; every self-referencing entry has a `ref_period` other than `T`, and every entry naming another module has `ref_period` `T`; every `ref_period` matches `T`, `T-<n><p>` or `T+<n><p>` (§3.11.3).
V-17	Every *Operator*'s operands combine under §4.1.5: one of them has an axis set containing every other's, and each ranges over exactly that operand's positions on the axes it shares.
V-18	Every date member is a well-formed ISO 8601 calendar date, except that `dates.to` MAY be `null` and `to_reference_date` MAY be the empty string; and `dates.from` ≤ `dates.to` and `from_reference_date` ≤ `to_reference_date` hold wherever the upper bound is present and bounded.

#	Rule
V-19	`severity` is `"error"`, `"warning"` or `"info"`.
V-20	For every `Constant`, the JSON type of `value` corresponds to `type_`, `"Date"` is a well-formed `YYYY-MM-DD` string, and `"Null"` has value `null` (§4.4).
V-21	Every `BinOp` with `op: "in"` has a `Set` or `SetOp` *Node* as `right`; every `Set` has a `children` array containing only `Scalar` or `Constant` *Nodes*, possibly empty (§4.6).
V-22	Every `Dimension` *Node* occurs in one of the four positions of §4.7; every `GroupingClause` occurs as the `grouping_clause` of an `AggregationOp`; every `AnalyticClause` occurs as its `analytic_clause`. Checked in `preconditions` as well as `operations`.
V-23	Every `GroupingClause` component is `"r"`, `"c"`, `"s"`, or a *Component* code available on the operand — a member name of `open_keys` for the operand's table, or a `to_component` introduced by a `RenameClauseOp` between the table and the `AggregationOp` (§4.19).
V-25	The root *Node* of every *Operation Object* is an *Operator Node*.
V-26	Every dictionary in the document that mentions a given `VariableID` assigns it the same *Data Type* (§3.5).
V-27	`dependency_information` has exactly `intra_instance_validations`, `cross_instance_dependencies` and `alternative_dependencies`, and no other member.
V-28	Every group of `alternative_dependencies` has at least two members, every URI in it is a member name of `dependency_modules`, and no URI appears in two groups.
V-29	Every value of `parameters` is one of the twelve type names of §3.9.1; every `ParameterRef.code` is a member name of `parameters`; every `ParameterRef.param_type` equals the declared type; every non-null `ParameterRef.default` has the JSON type its `param_type` requires — a bare item-signature String for `"Item"`, and a JSON array of such values, possibly empty, for a *Set* type (§3.9, §4.8.1).
V-30	Every `AggregationOp` has at most one clause present and non-null; `rank` has an `AnalyticClause` with `window` absent or `null` and a non-empty `order_by`; an `AnalyticClause` with a non-null `window` has a non-empty `order_by`, and its window has a `frame_type` of `"data_points"` or `"range"` with well-formed `start` and `end` bounds (§4.18, §4.20).
V-31	Every `SetOp` has the arity its `op` requires — one for `set_of`, two for `setdiff` and `symdiff`, two or more for `union` and `intersect`; every `SubstrOp` has integer or null `start` and `length`; every `RenameClauseOp` has a non-empty `clauses` array whose `from_component` and `to_component` sets do not overlap; every `DateConstructorOp` carries `year_node`, `month_node` and `day_node`, and every `AnnualiseOp` its `operand`, `fy_end` and `component` (§4.14–§4.17, §4.24).
V-32	Every `dependency_modules` entry has exactly `tables` and `variables`, and no other member (§3.12).

Rules V-9, V-10 and V-23 resolve against dictionaries carried in the same document, so every rule above remains checkable from the document alone.

~~V-24~~ is withdrawn

An earlier version of this part carried a rule V-24, requiring `VarID.default` to agree with the *Data Type* of its `datapoints` and `interval: true` to appear only on a numeric selection. Both are constraints on what a selection may **mean** — a fallback of the wrong type and interval arithmetics over an enumeration are type errors, settled by the Information Model and the Operators part before a document is produced — and neither can be decided without knowing what a *Data Type* is for. They are therefore semantic analysis, not serialisation, and this part no longer states them (§1.1).

The identifier is retired rather than reused, and the rules after it keep the numbers they had, so references to V-25 and beyond remain valid.

5.3.1 Reference Validator

A reference implementation of all 31 rules is published alongside this part: `validate_script.py`. It depends only on the Python standard library, requires neither the DPM database nor the taxonomy, and stands to this part as the `.g4` files stand to the Grammar.

```
python validate_script.py COREP_LE-3.3.0.json            # human-readable
python validate_script.py --json FINREP9-3.3.0.json      # machine-readable
python validate_script.py --quiet *.json                 # exit code only
```

It exits `0` when every document is valid, `1` on any violation, and `2` when a document cannot be read or parsed. Violations are grouped by rule identifier.

The validator is a **conformance aid, not the definition**: where its behaviour and the rules above disagree, the rules prevail.

One limitation is inherent rather than incidental. The *Position Index* set of a grouped `AggregationOp` depends on reported data (§4.1.5), so V-17 cannot be decided statically for any *Operator* above one. The validator counts these separately as *not statically verifiable* rather than passing them silently.

5.5.4 5.4 Unassigned Constructs

Three constructs have **no encoding** in this version of the specification. The first two are whole *Operation* types, not gaps in the expression language, and both need a design decision rather than an inference:

- **Calculations.** Every *Operation* in a *Script Document* is a validation. DPM-ML defines Calculation as an *Operation* type (Information Model §5.1) and links calculated *Variables* through *VariableCalculation* (Information Model §5.4); neither has an encoding here. Persistent assignment, *Operator* 36 `<-`, is unassigned for the same reason: it is the *Operator* that writes a calculated result, so it has nothing to act on until Calculations are encoded. Encoding them requires more than a *Node* class — a document would have to say which *Operations* produce data, in what order they run, and how a produced *Variable* becomes available to the *Operations* that consume it.
- **Conditional severity.** `severity` is a String on the *Operation Object* (§3.10). DPM-ML defines Conditional_severity as an *Operation* type whose result determines the severity of a failing case; there is no member for it, and adding one means deciding whether the severity-determining *Operation* is carried inline, referenced like a precondition, or resolved by the Producer before publication.

The third is a single *Operator*:

- ***Operator* 34, `node`.** It has no semantics anywhere in this specification — not in the Operators part, which describes `rename` (*Operator* 33) without ever mentioning `node`, and not in the Grammar. An earlier version of this part assigned it a JSON shape on `RenameClauseOp` so that a document using it would be recognised rather than rejected; that was a mistake, because a shape without a meaning cannot be implemented, and no document has ever carried it. `RenameClauseOp` now encodes *Operator* 33 only (§4.24).

A Producer MUST NOT emit any of the three in any form. A Consumer encountering an unrecognised `op` value or `class_name` MUST reject the document (§5.2, requirement 2) rather than guess an interpretation. Assigning an encoding is a revision of this specification (§5.6); implementations MUST NOT anticipate one.

5.5.5 5.5 Reserved Extension Points

The following members and values are deliberately shaped to admit extension without a breaking change. Producers MUST NOT use them for any other purpose; Consumers MUST NOT assume the value this version assigns is the only one they will ever meet.

Location	Current	Reserved for
`Scalar.scalar_type`	`"Item"`	Other kinds of DPM object reference — *Property*, *SubCategory*.
`TimeShiftOp.reference_period`	`"refPeriod"`	Any Date-typed *Component* as the temporal anchor, rather than only the instance's own reference period.
`AnnualiseOp.component`	`"refPeriod"`	The same, on the same terms (§4.17).
`AnalyticClause.window.frame_type`	`"data_points"`, `"range"`	Further frame modes, e.g. `groups` (§4.20).
`Constant.type_`	Integer, Number, Boolean, String, Date, Null	Item-valued and date-time literals.
`modules[*].ref_period`	`T`, `T±<n>{A,Q,M}`	Further period indicators.

Two DPM-ML attributes have **no** slot and would require a new member: *OperationNode.AbsoluteTolerance* and *OperationNode.RelativeTolerance*. Were a tolerance ever authored, comparison *Operators* would need to carry it, and a Consumer built against this version would silently apply exact comparison instead — a wrong answer rather than a rejected document, which is the dangerous kind of gap. Producers MUST therefore treat a non-null tolerance in DPM-ML as a condition requiring a revision of this specification, not as a value to discard.

5.5.6 5.6 Versioning

This specification is versioned with the documentation set it belongs to. A change is **breaking** if a document valid under the previous version becomes invalid, or is validly interpreted differently. Assigning an encoding to an Unassigned construct (§5.4), or a new value at a Reserved Extension Point (§5.5), is not breaking for Producers but requires a Consumer update to be usable.

Every *Script Document* carries the revision it was written to, in `format_version` (§3.2.1). A Consumer therefore always knows which rules a document was generated under, rather than having to infer it from which members happen to be present. A Consumer SHOULD also report the specification version it implements alongside its results, so that a mismatch is visible from both ends.

`format_version` is `MAJOR.MINOR`. A **MAJOR** increment marks a breaking change under the rule above; a **MINOR** increment marks a change a Consumer built for an earlier MINOR of the same MAJOR can still read — a new value at a Reserved Extension Point (§5.5), an encoding assigned to an Unassigned construct (§5.4), or a requirement relaxed. Relaxing a requirement is never breaking for a Consumer, but it is for a Producer's expectations of one, which is why it moves MINOR rather than nothing at all.

Why the member exists

Earlier versions of this part carried no version member, on the reasoning that `dpm_release` plus the Consumer's own version determined compatibility. That reasoning does not survive contact with real documents. Two of them in circulation carry the same `module_code` and `module_version` — `AE` 1.4.0 — from two different Producers, and differ in whether `parameters` is present, in whether an `AggregationOp` carries `analytic_clause` at all, and in whether a `VarID.data` entry repeats the coordinate fixed at the `VarID` level. Nothing in either document said which convention it was written to; the only way to tell was to open it and look, and a Consumer meeting one could report only an unrecognised or missing member without being able to say which revision it had been handed.

Adding the member is itself a breaking change under the rule above, and is the reason this version is 1.0.

5.5.7 5.7 Known Gaps and Limitations

This section is **informative**. It records two things an implementer will otherwise discover the hard way: a failure mode seen in documents already in circulation, and what a DPM database can and cannot be used to check a document against.

Nothing here relaxes a rule. A document exhibiting §5.7.1 is invalid under §5.3, and a Producer MUST fix it at source.

5.7.1 Incomplete cross-instance dependency information

Documents are in circulation in which some *Operations* appear in **neither** `intra_instance_validations` nor any `cross_instance_dependencies` entry, contrary to rule V-14. Every such *Operation* is a cross-module rule: it selects from a table of another module version, and because its dependency entry was never emitted, neither was that module's structure in `dependency_modules`.

One omission therefore shows up as several apparently separate failures:

Rule	Symptom
V-14	The *Operation* is in neither dependency group.
V-10	Its `VarID` names a `table` that is in neither `tables` nor `dependency_modules[*].tables`.
V-9	Its `datapoint`s are consequently in no *Data Type* dictionary.
V-13	A precondition may gate an *Operation* absent from `operations` altogether.

The undeclared tables are ordinary tables of the other module — not stale or misspelled codes; they exist in the DPM release the document names. What is missing is the declaration, not the table.

A Consumer MUST NOT guess. An *Operation* whose `VarID` names an undeclared table cannot be resolved — its *Data Types* are absent too — and MUST NOT be evaluated as though the missing table held no data.

The failure is confined to the dependency section. Where an entry does exist it is complete, and the correspondence between `dependency_modules` and `modules[*].URI` holds (V-16), so a Consumer can rely on what is declared and need only handle what is missing.

5.7.2 What a DPM database can and cannot confirm

A *Script Document* has two halves, and only one has a counterpart in a DPM database. The distinction is worth stating because the natural assumption — that a document can be checked against the DPM it was generated from — is half right, and the wrong half fails silently.

The structural half does correspond, exactly. A `datapoint`, a table version code, a *Data Type* code and a filing-indicator identifier are all DPM objects, and each resolves: the tuple (`table`, `row`, `column`, `sheet`) of a *Datapoint Reference* identifies a *TableVersionCell* whose *Variable* is that entry's `datapoint`, reading an axis the document leaves open as the DPM's `r*`, `c*` or `s*`. This is a strong check — it exercises range expansion, coordinate resolution and *Variable* assignment at once — and it is the right way to check a document suspected of being malformed.

The validation half does not correspond, and is not meant to. `code`, `version_id` and `operand_reference_id` identify *Operations* and operand occurrences within the content the Producer generated from, which need not be the validation set of any DPM database. Worse, all three fall in the same numeric or lexical ranges as their DPM namesakes, so a join **appears** to succeed and returns an unrelated rule. A Consumer or tool MUST NOT join on them (§3.10, §4.3.4).

This is the format doing its job rather than a shortcoming. §1.2 exists so that an engine needs no DPM at run time: an *Operation*'s tree, coordinates, *Data Types* and applicability are all in the document. A database can confirm the structure a document asserts; it cannot confirm the rules, because the rules are not in it.